

5. En la sección 6.1.3. se presentó un algoritmo para implementar una estrategia de reemplazo de páginas FIFO. Idee uno más eficiente. *Sugerencia:* Es posible actualizar el contador en la página recién cargada y no modificar los otros.
6. En los sistemas paginados que describimos en el texto el manejador de fallos de página formaba parte del nivel ISA y por tanto no estaba presente en el espacio de direcciones de ningún programa del nivel OSM. En realidad, el manejador de fallos de página también ocupa páginas, y en ciertas circunstancias (por ejemplo, política de reemplazo de páginas FIFO) podría ser sacado de la memoria. ¿Qué sucedería si el manejador de fallos de página no estuviera presente al ocurrir un fallo de página? ¿Cómo podría corregirse esto?
7. No todas las computadoras tienen un bit en hardware que se enciende automáticamente cuando se escribe en una página. No obstante, resulta útil de seguir la pista a las páginas modificadas, para no tener que suponer el peor de los casos y escribir todas las páginas de vuelta al disco después de usarlas. Suponiendo que cada página tiene bits en hardware que de forma independiente habilitan el acceso para lectura, escritura y ejecución, ¿cómo puede el sistema operativo saber cuáles páginas están limpias y cuáles están sucias?
8. Una memoria segmentada tiene segmentos paginados. Cada dirección virtual tiene un número de segmento de 2 bits, un número de página de 2 bits y una distancia dentro de la página de 11 bits. La memoria principal contiene 32 KB, divididos en páginas de 2 KB. Cada segmento es sólo de lectura, de lectura/ejecución, de lectura/escritura, o de lectura/escritura/ejecución. Las tablas de páginas y de protección tienen este aspecto:

Segmento 0		Segmento 1		Segmento 2	Segmento 3	
Sólo lectura		Leer/ejecutar		Leer/escribir/ejecutar	Leer/escribir	
Página virtual	Marco de página	Página virtual	Marco de página		Página virtual	Marco de página
0	9	0	En disco	Tabla de páginas no en la memoria principal	0	14
1	3	1	0		1	1
2	En disco	2	15		2	6
3	12	3	8		3	En disco

Para cada uno de los siguientes accesos a la memoria virtual, indique qué dirección física se calcula. Si ocurre un fallo, indique de qué tipo es.

Acceso	Segmento	Página	Distancia dentro de la página
1. buscar datos	0	1	1
2. buscar datos	1	1	10
3. buscar datos	3	3	2047
4. guardar datos	0	1	4
5. guardar datos	3	1	2
6. guardar datos	3	0	14
7. saltar a ellos	1	3	100
8. buscar datos	0	2	50
9. buscar datos	2	0	5
10. saltar a ellos	3	0	60

9. Algunas computadoras permiten E/S directamente al espacio de usuario. Por ejemplo, un programa podría iniciar una transferencia de disco a un buffer dentro de un proceso de usuario. ¿Causa esto problemas si se usa compactación para implementar la memoria virtual? Comente.

10. Los sistemas operativos que permiten archivos con correspondencia en la memoria siempre requieren que los archivos se mapeen en fronteras de página. Por ejemplo, con páginas de 4K, un archivo puede mapearse de modo que comience en la dirección virtual 4096, pero no en la dirección virtual 5000. ¿Por qué?
11. Cuando un registro de segmento se carga con un valor en el Pentium II, se trae el descriptor correspondiente y se carga en una parte invisible del registro de segmento. ¿Por qué cree que los diseñadores de Intel hayan decidido hacer esto?
12. Un programa en el Pentium II hace referencia al segmento local 10 con distancia 8000. El campo BASE del segmento LDT 10 contiene 10000. ¿Cuál entrada de directorio usa el Pentium II? ¿Cuál es el número de página? ¿Cuál es la distancia?
13. Comente algunos posibles algoritmos para eliminar segmentos en una memoria segmentada no paginada.
14. Compare la fragmentación interna con la externa. ¿Qué puede hacerse para aliviar cada una?
15. Los supermercados enfrentan constantemente un problema similar al reemplazo de páginas en los sistemas con memoria virtual. Hay una cantidad fija de espacio de anaquel para exhibir un número cada vez mayor de productos. Si llega un producto nuevo importante, digamos alimento para perros con eficiencia del 100%, algún producto existente tendrá que sacarse del inventario a fin de hacer un hueco para aquél. Los algoritmos de reemplazo obvios son LRU y FIFO. ¿Cuál de éstos preferiría usted?
16. ¿Por qué los bloques de caché siempre son mucho más pequeños que las páginas de memoria virtual, a menudo cien veces más pequeños?
17. ¿Por qué muchos sistemas de archivos exigen que un archivo se abra explícitamente con una llamada al sistema `open` antes de leerse?
18. Compare los métodos de mapa de bits y lista de huecos para contabilizar el espacio libre de un disco que tiene 800 cilindros, cada uno de los cuales tiene 5 pistas de 32 sectores. ¿Cuántos huecos tendría que haber para que la lista de huecos fuera más grande que el mapa de bits? Suponga que la unidad de asignación es el sector y que un hueco requiere una entrada de tabla de 32 bits.
19. Si se quiere predecir el desempeño de un disco, resulta útil tener un modelo de asignación del espacio. Suponga que el disco se ve como un espacio de direcciones lineal de $N \gg 1$ sectores, que consiste en una serie de bloques de datos, luego un hueco, luego otra serie de bloques de datos, y así. Si mediciones empíricas muestran que las distribuciones de probabilidad para las longitudes de las series de datos y los huecos son iguales, y la probabilidad de que cualquiera de ellos sea de i sectores es de 2^{-i} , ¿cuál será el número esperado de huecos en el disco?
20. En cierta computadora un programa puede crear tantos archivos como necesite, y todos los archivos pueden crecer dinámicamente durante la ejecución sin proporcionar al sistema operativo información por adelantado acerca de su tamaño final. ¿Cree que los archivos se almacenarán en sectores consecutivos? Explique.
21. Considere el siguiente método con el que un sistema operativo podría implementar instrucciones de semáforos. Cada vez que la CPU está a punto de ejecutar `up` o `down` con un semáforo (una variable entera en la memoria), primero ajusta los bits de prioridad o máscara de la CPU de modo que se inhabiliten todas las interrupciones. Luego la CPU trae el semáforo, lo modifica y toma la rama correcta. Por último, la CPU vuelve a habilitar las interrupciones. ¿Este método funciona si

- a. hay una sola CPU que conmuta entre procesos cada 100 ms?
b. dos CPU comparten una memoria común en la que se encuentra el semáforo?
22. La Compañía de Sistemas Operativos Quenuncaseaen ha estado recibiendo quejas de algunos de sus clientes relacionadas con su más reciente versión, que incluye operaciones con semáforos. Los clientes creen que es inmoral que los procesos se bloqueen (lo llaman "dormirse en el trabajo"). Dado que es política de la compañía dar a los clientes lo que piden, se ha propuesto añadir una tercera operación, **peek**, para complementar **up** y **down**. Lo que hace **peek** es examinar el semáforo sin modificarlo ni bloquear el proceso. De este modo, los programas que sientan que es inmoral bloquearse pueden examinar primero el semáforo para ver si pueden ejecutar **down** sin peligro. ¿Funcionará esta idea si tres o más procesos usan el semáforo? ¿Y si dos procesos usan el semáforo?
23. Prepare una tabla que muestre cuáles de los procesos P1, P2 Y P3 se están ejecutando y cuáles están bloqueados, en función del tiempo desde 0 hasta 1000 ms. Los tres procesos ejecutan instrucciones **up** y **down** con el mismo semáforo. Cuando dos procesos están bloqueados y se ejecuta una **up**, se reinicia el proceso que tiene el número más bajo, es decir, P1 tiene preferencia sobre P2 y P3, etc. Inicialmente, los tres procesos están en ejecución y el semáforo es 1.

En $t = 100$ P1 ejecuta **down**
 En $t = 200$ P1 ejecuta **down**
 En $t = 300$ P2 ejecuta **up**
 En $t = 400$ P3 ejecuta **down**
 En $t = 500$ P1 ejecuta **down**
 En $t = 600$ P2 ejecuta **up**
 En $t = 700$ P2 ejecuta **down**
 En $t = 800$ P1 ejecuta **up**
 En $t = 900$ P1 ejecuta **up**

24. En un sistema de reservaciones de líneas aéreas es necesario asegurar que mientras un proceso está ocupado usando un archivo, ningún otro proceso puede usarlo también. De lo contrario, dos procesos distintos, trabajando para dos agentes de viajes distintos, podrían sin darse cuenta vender el último asiento de algún vuelo. Idee un método de sincronización empleando semáforos que asegure que sólo un proceso a la vez accederá a cada archivo (suponiendo que los procesos obedecen las reglas).
25. Para poder implementar semáforos en una computadora con varias CPU que comparten una memoria común, los arquitectos de computadoras a menudo incluyen una instrucción de Probar y Poner Candado (*Test and Set Lock*) **TSL X** prueba la posición X. Si el contenido es 0, se pone en 1 en un solo ciclo de memoria indivisible, y se pasa por alto la siguiente instrucción. Si el contenido no es 0, la **TSL** actúa como **nop** (ninguna operación). Con **TSL** es posible escribir procedimientos **lock** y **unlock** con las siguientes propiedades. **lock** (x) verifica si x tienen candado; si no, pone un candado a x y devuelve el control; si x ya tiene un candado, el procedimiento espera hasta que se abre el candado y entonces pone un candado a x y devuelve el control. **unlock** abre un candado existente. Si todos los procesos ponen un candado a la tabla de semáforos antes de usarla, sólo un proceso a la vez podrá meterse con las variables y apuntadores, y se evitarán las competencias. Escriba **lock** y **unlock** en lenguaje ensamblador. (Haga cualquier supuesto razonable que necesite.)

26. Indique los valores de *in* y *out* para un buffer circular de 65 palabras de largo después de cada una de las operaciones siguientes. Ambas variables valen inicialmente 0.
- Se meten 22 palabras
 - Se sacan 9 palabras
 - Se meten 40 palabras
 - Se sacan 17 palabras
 - Se meten 12 palabras
 - Se sacan 45 palabras
 - Se meten 8 palabras
 - Se sacan 11 palabras
27. Suponga que una versión de UNIX usa bloques de disco de 2K y guarda 512 direcciones de disco en cada bloque indirecto (sencillo, doble y triple). ¿Qué tamaño máximo podría tener un archivo? (Suponga que los apuntadores de archivo son de 64 bits.)
28. Suponga que la llamada al sistema UNIX
- ```
unlink("/usr/ast/bin/game3")
```
- se ejecuta en el contexto de la figura 6-37. Describa cuidadosamente los cambios que se efectúan en el sistema de directorios.
29. Imagine que tiene que implementar el sistema UNIX en una microcomputadora con muy escasa memoria principal. Por más que trata de meter a la fuerza el sistema en el espacio disponible, no logra hacer que quepa, así que escoge una llamada al sistema al azar, la cual sacrificará por el bien general. Se escogió **pipe**, que crea los tubos empleados para enviar flujos de bytes de un proceso a otro. ¿Sigue siendo posible implementar la redirección de E/S de alguna manera? ¿Y las filas de procesamiento? Comente los problemas y posibles soluciones.
30. El Comité para el Trato Justo de los Descriptores de Archivo está organizando una protesta contra el sistema UNIX porque cada vez que éste devuelve un descriptor de archivo, siempre devuelve el número más bajo que no se está usando actualmente. Por consiguiente, los descriptores de archivo con números altos pocas veces se usan. Su plan es devolver el número más bajo que el programa todavía no ha usado, en lugar del número más bajo que no se está usando actualmente. Se asegura que la implementación es trivial, que no afectará a los programas existentes, y que es más justo. ¿Qué piensa usted?
31. En NT es posible configurar una lista de control de acceso de tal manera que Roberta no tenga ningún acceso a un archivo, pero todos los demás tengan pleno acceso a él. ¿Cómo cree que se implemente esto?
32. Describa dos formas de programar problemas de productor-consumidor empleando buffers compartidos y semáforos en NT. Piense en cómo implementar el buffer compartido en cada caso.
33. Es común probar los algoritmos de reemplazo de páginas por simulación. Para este ejercicio, escriba un simulador para una memoria virtual basada en páginas en una máquina con 64 páginas de 1 KB. El simulador debe mantener una sola tabla de 64 entradas, una por página, que contengan el número de página física que corresponde a esa página virtual. El simulador deberá leer un archivo que contiene direcciones virtuales en decimal, una dirección por línea. Si la página correspondiente está en la memoria, se registra un acierto de página; si no, se invoca un procedimiento de

reemplazo de páginas que escoja una página para expulsarla (es decir, una entrada de la tabla para sobrescribirla) y se registra un fallo de página. No ocurre realmente transporte de páginas. Genere un archivo que consista en direcciones aleatorias y pruebe el desempeño con LRU y con FIFO. Ahora genere un archivo de direcciones en el que  $x\%$  de las direcciones sean cuatro bytes más altas que la dirección anterior (a fin de simular localidad). Realice pruebas con diversos valores de  $x$  e informe sus resultados.

34. Escriba un programa para UNIX o NT que acepte como entrada el nombre de un directorio. El programa deberá imprimir una lista de los archivos del directorio, una línea por archivo, y después del nombre del archivo imprimir el tamaño del archivo. Imprima los nombres de archivo en el orden en que aparezcan en el directorio. Las ranuras no utilizadas del directorio deberán indicarse con esta expresión: (no utilizado).

# 7

## EL NIVEL DE LENGUAJE ENSAMBLADOR

En los capítulos 4, 5 y 6 examinamos tres niveles que están presentes en casi todas las computadoras contemporáneas. Este capítulo se ocupa primordialmente de otro nivel que también está presente en prácticamente todas las computadoras modernas: el nivel de lenguaje ensamblador. El nivel de lenguaje ensamblador difiere en un aspecto importante de los niveles de microarquitectura, ISA y sistema operativo: se implementa por traducción en lugar de interpretación.

Los programas que convierten un programa de usuario escrito en algún lenguaje, a otro lenguaje, se llaman **traductores**. El lenguaje en el que está escrito el programa original se llama **lenguaje fuente**, y el lenguaje al que se convierte se llama **lenguaje objetivo**. Ambos lenguajes definen niveles. Si se cuenta con un procesador capaz de ejecutar directamente programas escritos en el lenguaje fuente, no hay necesidad de traducir el programa fuente al lenguaje objetivo.

Se usa traducción cuando se cuenta con un procesador (sea hardware o un intérprete) para el lenguaje objetivo pero no para el lenguaje fuente. Si la traducción se efectuó correctamente, la ejecución del programa traducido dará exactamente los mismos resultados que habría dado la ejecución del programa fuente si se hubiera contado con un procesador para él. Por tanto, es posible implementar un nuevo nivel para el que no hay un procesador traduciendo primero los programas escritos para ese nivel a un nivel objetivo, y ejecutando después los programas resultantes en el nivel objetivo.

Es importante señalar la diferencia entre traducción e interpretación. En la traducción no se ejecuta directamente el programa original en el lenguaje fuente; en vez de ello, se convier-

te en un programa equivalente llamado **programa objeto** o **programa binario ejecutable** que se ejecuta sólo después de que se ha llevado a cabo la traducción. En la traducción hay dos pasos bien definidos:

1. Generación de un programa equivalente en el lenguaje objetivo.
2. Ejecución del programa recién generado.

Estos dos pasos no ocurren simultáneamente. El segundo paso no se inicia hasta que el primero ha finalizado. En la interpretación sólo hay un paso: ejecutar el programa fuente original. No es necesario generar primero un programa equivalente, aunque a veces el programa fuente se convierte en una forma intermedia (por ejemplo, código de bytes Java) para facilitar la interpretación.

Mientras el programa objeto se está ejecutando, sólo son evidentes tres niveles: el nivel de microarquitectura, el nivel ISA y el nivel de máquina del sistema operativo. Por tanto, en el momento de la ejecución pueden encontrarse tres programas —el programa objeto del usuario, el sistema operativo y el microprograma (si lo hay)— en la memoria de la computadora. Todos los rastros del programa fuente original han desaparecido. Así pues, el número de niveles presentes en el momento de la ejecución podría diferir del número de niveles presentes antes de la traducción. No obstante, cabe señalar que, si bien definimos un nivel por las instrucciones y construcciones lingüísticas con que cuentan sus programadores (y no por la técnica de implementación), otros autores a veces hacen una distinción mayor entre los niveles implementados con intérpretes en el momento de la ejecución y los niveles implementados por traducción.

## 7.1 INTRODUCCIÓN AL LENGUAJE ENSAMBLADOR

Los traductores pueden dividirse a grandes rasgos en dos grupos, dependiendo de la relación entre el lenguaje fuente y el lenguaje objetivo. Si el lenguaje fuente es en lo esencial una representación simbólica de un lenguaje de máquina numérico, el traductor se llama **ensamblador** y el lenguaje fuente se llama **lenguaje ensamblador**. Si el lenguaje fuente es un lenguaje de alto nivel como Java o C y el lenguaje objetivo es un lenguaje de máquina numérico o una representación simbólica de tal lenguaje, el traductor se llama **compilador**.

### 7.1.1 ¿Qué es un lenguaje ensamblador?

Un lenguaje ensamblador puro es un lenguaje en el que cada enunciado produce exactamente una instrucción de máquina. En otras palabras, existe una correspondencia uno a uno entre las instrucciones de máquina y los enunciados del programa en ensamblador. Si cada línea del programa en lenguaje ensamblador contiene exactamente un enunciado y cada palabra de máquina contiene exactamente una instrucción de máquina, entonces un programa en ensamblador de  $n$  líneas producirá un programa en lenguaje de máquina de  $n$  palabras.

La razón por la que la gente usa lenguaje ensamblador en lugar de programar en lenguaje de máquina (en hexadecimal) es que es mucho más fácil programar en lenguaje ensamblador. El uso de nombres simbólicos y direcciones simbólicas en lugar de direcciones binarias u octales representa una gran diferencia. Casi toda la gente puede recordar que las abreviaturas para sumar, restar, multiplicar y dividir son ADD, SUB, MUL y DIV, pero pocos pueden recordar los valores numéricos correspondientes que la máquina usa. El programador en lenguaje ensamblador sólo tiene que recordar los nombres simbólicos porque el ensamblador los traduce a instrucciones de máquina.

Los mismos comentarios aplican a las direcciones. El programador en lenguaje ensamblador puede asignar nombres simbólicos a posiciones de memoria y dejar que el ensamblador se preocupe por insertar los valores numéricos correctos. El programador en lenguaje de máquina siempre debe trabajar con los valores numéricos de las direcciones. Es por esto que ya nadie programa en lenguaje de máquina, aunque se hacía hace algunas décadas, antes de inventarse los ensambladores.

Los lenguajes ensambladores tienen otra propiedad, además de la correspondencia uno a uno entre enunciados en lenguaje ensamblador e instrucciones de máquina, que los distingue de los lenguajes de alto nivel. El programador en ensamblador tiene acceso a todas las características e instrucciones disponibles en la máquina objetivo. El programador en lenguaje de alto nivel no. Por ejemplo, si la máquina objetivo tiene un bit de desbordamiento, un programa en lenguaje ensamblador puede probarlo, pero un programa en Java no puede hacerlo directamente. Un programa en lenguaje ensamblador puede ejecutar todas las instrucciones del conjunto de instrucciones de la máquina objetivo, pero el programa en lenguaje de alto nivel no puede hacerlo. En síntesis, todo lo que puede hacerse en lenguaje de máquina puede hacerse en lenguaje ensamblador, pero muchas instrucciones, registros, y características similares no están disponibles para el programador en lenguaje de alto nivel. Los lenguajes para programación de sistemas, como C, suelen ser un híbrido entre estos dos tipos, con la sintaxis del lenguaje de alto nivel pero con una buena parte del acceso a la máquina que tiene un lenguaje ensamblador.

Una diferencia final que vale la pena dejar en claro es que un programa en lenguaje ensamblador sólo puede ejecutarse en una familia de máquinas, mientras que un programa escrito en un lenguaje de alto nivel tiene el potencial para ejecutarse en muchas máquinas. En muchas aplicaciones, esta posibilidad de trasladar software de una máquina a otra tiene gran importancia práctica.

### 7.1.2 ¿Por qué usar lenguaje ensamblador?

No nos engañemos: la programación en lenguaje ensamblador es difícil; no es para los débiles ni los cobardes. Además, escribir un programa en lenguaje ensamblador toma mucho más tiempo que escribir el mismo programa en un lenguaje de alto nivel; y también tarda mucho más en depurarse y es mucho más difícil de mantener.

En tales condiciones, ¿por qué alguien habría de programar en lenguaje ensamblador? Hay dos razones: desempeño y acceso a la máquina. Antes que nada, un programador **experto** en lenguaje ensamblador a menudo puede producir código que es mucho más pequeño y

rápido que el que puede producir un programador en lenguaje de alto nivel. En algunas aplicaciones la rapidez y el tamaño son críticas. Muchas aplicaciones incorporadas, como el código de una tarjeta inteligente, el código de un teléfono celular, los controladores de dispositivos, las rutinas de BIOS y los ciclos interiores de aplicaciones para las que el desempeño es crítico pertenecen a esta categoría.

En segundo lugar, algunos procedimientos requieren acceso total al hardware, cosa que suele ser imposible en lenguajes de alto nivel. Por ejemplo, los manejadores de interrupciones y trampas de bajo nivel de un sistema operativo, y los controladores de dispositivos de muchos sistemas incorporados de tiempo real pertenecen a esta categoría.

La primera razón para programar en lenguaje ensamblador (obtener un buen desempeño) suele ser la más importante, así que la examinaremos con mayor detenimiento. En casi todos los programas, un porcentaje pequeño del código ocupa un porcentaje elevado del tiempo de ejecución. Es común que el 1% del programa ocupe el 50% del tiempo de ejecución y que el 10% del programa ocupe el 90% del tiempo de ejecución.

Supongamos, por ejemplo, que se requieren 10 años-programador para escribir cierto programa en un lenguaje de alto nivel y que el programa resultante requiere 100 s para ejecutar cierto programa de prueba estándar. (Un **programa de prueba estándar** se usa para comparar computadoras, compiladores, etc.). Escribir todo el programa en lenguaje ensamblador podría requerir 50 años-programador, debido a la menor productividad de los programadores en lenguaje ensamblador. El programa final podría ejecutar el programa de prueba estándar en unos 33 s, ya que un programador ingenioso puede superar a un compilador ingenioso en un factor de 3 (aunque nadie se pone de acuerdo en estas proporciones). La situación se ilustra en la figura 7-1.

Con base en la observación de que sólo una diminuta fracción del código ocupa casi todo el tiempo de ejecución, puede adoptarse otra estrategia. El programa se escribe primero en lenguaje de alto nivel. Luego se efectúa una serie de mediciones para determinar cuáles partes del programa ocupan la mayor parte del tiempo de ejecución. Tales mediciones normalmente incluirían usar el reloj del sistema para calcular el tiempo que se invierte en cada procedimiento, contabilizar el número de veces que cada ciclo se ejecuta, y acciones similares.

Por ejemplo, supongamos que el 10% del programa da cuenta del 90% del tiempo de ejecución. Esto implica que, de un trabajo de 100 s, 90 s se dedican a este 10% crítico y 10 s se dedican al 90% restante del programa. Ahora puede mejorarse el 10% crítico reescribiéndolo en lenguaje ensamblador. Este proceso se llama **afinación** y se ilustra en la figura 7-1. Aquí se requieren otros 5 años-programador para reescribir los procedimientos críticos pero su tiempo de ejecución se reduce de 90 s a 30 s.

Es provechoso comparar la estrategia mixta de lenguaje de alto nivel/lenguaje ensamblador con la versión únicamente en lenguaje ensamblador (vea la figura 7-1). La segunda es aproximadamente 20% más rápida (33 s *versus* 40 s) pero con un costo de más del triple (50 años-programador *versus* 15 años-programador). Además, la ventaja de la estrategia mixta es en realidad mayor que la indicada, porque la recodificación en lenguaje ensamblador de un procedimiento en lenguaje de alto nivel ya depurado es en realidad mucho más fácil que escribir el mismo procedimiento en lenguaje ensamblador desde cero. En otras palabras, el

|                                    | Años-programador para producir el programa | Tiempo de ejecución del programa en segundos |
|------------------------------------|--------------------------------------------|----------------------------------------------|
| Lenguaje ensamblador               | 50                                         | 33                                           |
| Lenguaje de alto nivel             | 10                                         | 100                                          |
| Estrategia mixta antes de afinar   |                                            |                                              |
| Crítico 10%                        | 1                                          | 90                                           |
| Otro 90%                           | 9                                          | 10                                           |
| Total                              | 10                                         | 100                                          |
| Estrategia mixta después de afinar |                                            |                                              |
| Crítico 10%                        | 6                                          | 30                                           |
| Otro 90%                           | 9                                          | 10                                           |
| Total                              | 15                                         | 40                                           |

**Figura 7-1.** Comparación de programación en lenguaje ensamblador y lenguaje de alto nivel, con y sin afinación.

estimado de 5 años-programador para reescribir los procedimientos críticos en excesivamente conservador. Si esta recodificación en realidad sólo requiriera un año-programador, la relación de costos entre la estrategia mixta y la estrategia de lenguaje ensamblador pura sería de más de 4 a 1 en favor de la estrategia mixta.

Un programador que usa un lenguaje de alto nivel no está absorto en pasar bits de aquí para acá y a veces ve aspectos del problema que permiten efectuar mejoras *reales* en el desempeño. Esta situación pocas veces se presenta con los programadores de lenguaje ensamblador, quienes por lo regular se la pasan haciendo juegos malabares con las instrucciones a fin de ahorrarse unos cuantos ciclos.

Dos experiencias bien documentadas que ocurrieron durante la creación de MULTICS destacan este punto. Graham (1970) informó de un procedimiento en PL/I que se reescribió en tres meses; la nueva versión era 26 veces más pequeña que el original y 50 veces más rápida. Otro procedimiento se volvió 20 veces más pequeño y 40 veces más rápido con dos meses de trabajo. Corbató (1969) describió un procedimiento en PL/I que se redujo de 50,000 a 10,000 palabras de código compilado en menos de un mes, y un controlador de E/S que se encogió de 65,000 a 30,000 palabras de código compilado, con una mejora de la rapidez de un factor de 8 en cuatro meses. La cuestión aquí es que, como los programadores en lenguaje de alto nivel tienen una perspectiva más global de lo que están haciendo, tienen más posibilidades de generar ideas que conduzcan a algoritmos totalmente distintos e incomparablemente mejores.

A pesar de estas experiencias (y muchas otras similares), sigue habiendo por lo menos cuatro buenas razones para estudiar lenguaje ensamblador. Primera, dado que el éxito o fracaso de un proyecto grande podría depender de la capacidad para expresar algún procedi-

miento crítico hasta mejorar su desempeño en un factor de 2 o 3, es importante poder escribir buen código en lenguaje ensamblador cuando realmente es necesario.

Segunda, el código de ensamblador a veces es la única alternativa debido a la escasez de memoria. Las tarjetas inteligentes contienen una CPU, pero pocas tienen un megabyte de memoria, y más pocas aún tienen un disco duro para paginar. A pesar de ello, tales tarjetas tienen que efectuar complejos cálculos criptográficos con recursos limitados. Los procesadores incorporados en los aparatos domésticos a menudo tienen un mínimo de memoria por razones de costo. Los asistentes digitales personales y otros dispositivos electrónicos inalámbricos de baterías suelen tener memorias pequeñas a fin de ahorrar energía, por lo que aquí también es indispensable un código pequeño y eficiente.

Tercera, un compilador debe producir salidas que un ensamblador pueda usar, o bien realizar el proceso de ensamblado él mismo. Por ello, entender el lenguaje ensamblador es indispensable para entender cómo funcionan los compiladores. Después de todo, alguien tiene que escribir el compilador (y su ensamblador).

Por último, el estudio del lenguaje ensamblador pone al descubierto la máquina real. Para los estudiantes de arquitectura de las computadoras, escribir algo de código de ensamblador es la única forma de ver realmente cómo son las máquinas en el nivel de arquitectura.

7.1.3 Formato de un enunciado en lenguaje ensamblador

Aunque la estructura de un enunciado en lenguaje ensamblador refleja la estructura de la instrucción de máquina que representa, los lenguajes de ensamblador para diferentes máquinas y diferentes niveles se parecen lo suficiente unos a otros como para poder hablar del lenguaje ensamblador en general. La figura 7-2 muestra fragmentos de programas en lenguaje ensamblador para el Pentium II, el Motorola 680x0 y el (Ultra)SPARC que realizan el cálculo  $N = I + J$ . En los tres ejemplos los enunciados que están arriba de la línea en blanco efectúan el cálculo. Los enunciados que están abajo de la línea en blanco son comandos para el ensamblador, que le ordenan reservar memoria para las variables *I*, *J* y *N*, y no son representaciones simbólicas de instrucciones de máquina.

Existen varios ensambladores para la familia Intel, cada uno con una sintaxis distinta. En este capítulo usaremos el lenguaje ensamblador MASM de Microsoft para nuestros ejemplos. Aunque nos estamos concentrando en el Pentium II, todo lo que digamos al respecto aplica igualmente al 386, 486, Pentium y Pentium Pro. Para el SPARC basaremos nuestros ejemplos en el ensamblador de Sun. Aquí también todo lo que digamos aplica también a las versiones anteriores (de 32 bits) de SPARC. Por uniformidad, usaremos mayúsculas para los códigos de operación y los registros en todos los casos (convención de Pentium II) aunque el ensamblador de Sun espera minúsculas.

Los enunciados en lenguaje ensamblador tienen cuatro partes: un campo de etiqueta, un campo de operación (código de operación), un campo de operandos y un campo de comentarios. Las etiquetas, que sirven para asignar nombres simbólicos a direcciones de memoria, se necesitan en enunciados ejecutables para poder saltar a esos enunciados. También se necesitan en las palabras de datos para poder acceder con un nombre simbólico a los datos ahí almacenados. Si un enunciado está etiquetado, la etiqueta (generalmente) inicia en la columna 1.

| Etiqueta | Cód. op. | Operandos        | Comentarios                               |
|----------|----------|------------------|-------------------------------------------|
| FÓRMULA: | MOV      | EAX,I            | ; registro EAX = I                        |
|          | ADD      | EAX,J            | ; registro EAX = I + J                    |
|          | MOV      | N,EAX            | ; N = I + J                               |
| I        | DW       | 3                | ; reservar 4 bytes inicializados a 3      |
| J        | DW       | 4                | ; reservar 4 bytes inicializados a 4      |
| N        | DW       | 0                | ; reservar 4 bytes inicializados a 0      |
| (a)      |          |                  |                                           |
| Etiqueta | Cód. op. | Operandos        | Comentarios                               |
| FÓRMULA  | MOVE.L   | I, D0            | ; registro D0 = I                         |
|          | ADD.L    | J, D0            | ; registro D0 = I + J                     |
|          | MOVE.L   | D0, N            | ; N = I + J                               |
| I        | DC.L     | 3                | ; reservar 4 bytes inicializados a 3      |
| J        | DC.L     | 4                | ; reservar 4 bytes inicializados a 4      |
| N        | DC.L     | 0                | ; reservar 4 bytes inicializados a 0      |
| (b)      |          |                  |                                           |
| Etiqueta | Cód. op. | Operandos        | Comentarios                               |
| FÓRMULA: | SETHI    | %HI(I),%R1       | ! R1 = bits de orden alto de la dir. de I |
|          | LD       | [%R1+%LO(I)],%R1 | ! R1 = I                                  |
|          | SETHI    | %HI(J),%R2       | ! R2 = bits de orden alto de la dir. de J |
|          | LD       | [%R2+%LO(J)],%R2 | ! R2 = J                                  |
|          | NOP      |                  | ! esperar que J llegue de la memoria      |
|          | ADD      | %R1,%R2,%R2      | ! R2 = R1 + R2                            |
|          | SETHI    | %HI(N),%R1       | ! R1 = bits de orden alto de la dir. de N |
|          | ST       | %R2,[%R1+%LO(N)] |                                           |
| I:       | .WORD    | 3                | ! reservar 4 bytes inicializados a 3      |
| J:       | .WORD    | 4                | ! reservar 4 bytes inicializados a 4      |
| N:       | .WORD    | 0                | ! reservar 4 bytes inicializados a 0      |
| (c)      |          |                  |                                           |

Figura 7-2. Cálculo de  $N = I + J$ . (a) Pentium II. (b) Motorola 680x0. (c) SPARC.

Cada una de las tres partes de la figura 7-2 tiene cuatro etiquetas: *FÓRMULA*, *I*, *J* y *N*. Observe que el lenguaje ensamblador de SPARC requiere un signo de dos puntos después de cada etiqueta, pero el de Motorola no. El de Intel requiere los dos puntos en las etiquetas de código pero no en las de datos. Esta diferencia nada tiene de fundamental. Los diseñadores de diferentes ensambladores suelen tener diferentes gustos. Nada en la arquitectura subyacente sugiere una opción o la otra. Una ventaja de la notación con dos puntos es que cuando se usa una etiqueta puede aparecer sola en una línea, con el código de operación en la columna 1 de la siguiente línea.

Tal estilo a veces es cómodo para los compiladores. Sin los dos puntos, no habría forma de distinguir un rótulo solo en una línea de un código de operación solo en una línea.

Una característica deplorable de algunos ensambladores es que los rótulos están restringidos a seis u ocho caracteres. En contraste, la mayor parte de los lenguajes de alto nivel permite usar nombres arbitrariamente largos. Los nombres largos bien elegidos hacen que los programas sean mucho más fáciles de entender por otras personas.

Cada una de las máquinas tiene algunos registros, pero reciben nombres muy diferentes. Los registros del Pentium II tienen nombres como **EAX**, **EBX**, **ECX**, etc. Los registros del Motorola se llaman **D0**, **D1**, **D2**, entre otros nombres. Los registros del SPARC tienen diversos nombres. Aquí usamos **%R1** y **%R2** para ellos.

El campo de código de operación contiene una abreviatura simbólica del código de operación —si el enunciado es una representación simbólica de una instrucción de máquina— o un comando para el ensamblador. La selección de un nombre apropiado es cuestión de gusto, y diferentes diseñadores de lenguaje ensamblador a menudo toman decisiones distintas. Los diseñadores del ensamblador de Intel decidieron usar **MOV** tanto para cargar un registro con el contenido de la memoria como para almacenar el contenido de un registro en la memoria. Los diseñadores del ensamblador de Motorola escogieron **MOVE** para las dos operaciones. En contraste, los diseñadores del ensamblador de SPARC decidieron usar **LD** para la primera y **ST** para la segunda. En este caso, también, las decisiones nada tienen que ver con la máquina subyacente.

En cambio, la necesidad de usar dos instrucciones de máquina, comenzando con **SETHI**, para acceder a la memoria, es una propiedad inherente a la arquitectura SPARC porque las direcciones virtuales tienen 32 bits (SPARC Versión 8) o 44 bits (SPARC Versión 9) y las instrucciones pueden contener cuando más 22 bits de datos intermedios. Por ello, siempre se requieren dos instrucciones para proporcionar todos los bits de una dirección virtual completa. Lo que

**SETHI %HI(I),%R1**

hace es poner en ceros los 32 bits superiores y los 10 bits inferiores del registro (de 64 bits) **R1** y luego colocar los 22 bits superiores de la dirección de 32 bits de *I* en las posiciones de bit 10 a 31 de **R1**. La siguiente instrucción,

**LD [%R1+%LO(I)],%R1**

suma **R1** y los 10 bits de orden bajo de la dirección de *I* para formar la dirección completa de *I*, obtener esa palabra de la memoria y colocarla en **R1**. En un concurso de belleza en el que se usara una escala del 1 al 10, estas instrucciones obtendrían una puntuación de aproximadamente -20, pero el SPARC no se diseñó pensando en la belleza de su lenguaje ensamblador; se diseñó pensando en la rapidez de ejecución, y esa meta sí se alcanza.

Las familias Pentium, 680x0 y SPARC permiten operandos de byte, palabra y largos. ¿Cómo sabe el ensamblador qué longitud debe usar? Una vez más, los diseñadores de ensambladores escogieron diferentes soluciones. En el Pentium II los registros con diferente longitud tienen nombres distintos; así, **EAX** se usa para elementos de 32 bits, **AX** sirve para mover elementos de 16 bits y, **AL** y **AH** se usan para elementos de 8 bits. Los diseñadores del ensamblador Motorola, en cambio, decidieron añadir a cada código de operación un sufijo: *.L* para largo, *.W* para palabra o *.B* para byte, en lugar de dar a subconjuntos de **D0**, etc., diferen-

tes nombres. El SPARC usa diferentes códigos de operación para las distintas longitudes (por ejemplo, **LDSB**, **LDSH** y **LDSW** para cargar bytes con signo, medias palabras y palabras en un registro de 64 bits, respectivamente). Las tres técnicas son válidas, y destacan una vez más la naturaleza arbitraria del diseño de lenguajes.

Los tres ensambladores también difieren en la forma en que reservan espacio para los datos. Los diseñadores del lenguaje ensamblador de Intel escogieron **DW** (*Define Word*, definir palabra), aunque posteriormente se añadió **.WORD** como alternativa. A los de Motorola les gustó **DC** (*Definir Constante*). La gente de SPARC prefirió **.WORD** desde el principio. Una vez más, las diferencias son arbitrarias.

El campo de operandos de un enunciado en lenguaje ensamblador sirve para especificar las direcciones y registros que la instrucción de máquina usará como operandos. El campo de operandos de una instrucción de suma de enteros indica qué se sumará a qué. El campo de operandos de una instrucción de ramificación indica a dónde se saltará. Los operandos pueden ser registros, constantes, localidades de memoria, etcétera.

El campo de comentarios ofrece a los programadores un lugar para incluir explicaciones de cómo funciona el programa, para beneficio de otros programadores que podrían usar o modificar posteriormente el programa (o para el beneficio del programador original un año después). Un programa en lenguaje ensamblador sin tal documentación es casi incomprensible para todos los programadores, a menudo incluido el autor. El campo de comentarios es exclusivamente para consumo humano; no afecta el proceso de ensamblado ni el programa que se genera.

### 7.1.4 Seudoinstrucciones

Además de especificar qué instrucciones de máquina deben ejecutarse, un programa en lenguaje ensamblador puede contener comandos para el ensamblador mismo, por ejemplo para pedirle que asigne espacio en la memoria o que saque una nueva página de listado. Los comandos para el ensamblador se llaman **seudoinstrucciones** o a veces **directrices de ensamblador**. Ya vimos una pseudoinstrucción típica en la figura 7-2(a): **DW**. En la figura 7-3 se dan algunas otras pseudoinstrucciones tomadas del ensamblador Microsoft MASM para la familia Intel.

La pseudoinstrucción **SEGMENT** inicia un nuevo segmento, y **ENDS** termina uno. Está permitido iniciar un segmento de texto, con código, luego iniciar un segmento de datos, luego volver al segmento de texto, etcétera.

**ALIGN** hace que la siguiente línea, casi siempre de datos, se coloque en una dirección que sea un múltiplo de su argumento. Por ejemplo, si el segmento actual ya tiene 61 bytes de datos, después de **ALIGN 4** la siguiente dirección asignada será la 64.

**EQU** sirve para asignar un nombre simbólico a una expresión. Por ejemplo, después de la pseudoinstrucción

**BASE EQU 1000**

puede usarse el símbolo **BASE** en cualquier punto en vez de 1000. La expresión que sigue a **EQU** puede contener varios símbolos definidos combinados con operadores aritméticos o de otro tipo, como en

| Seudoinstrucción | Significado                                                               |
|------------------|---------------------------------------------------------------------------|
| SEGMENT          | Iniciar un nuevo segmento (texto, datos, etc.) con ciertos atributos      |
| ENDS             | Terminar el segmento actual                                               |
| ALIGN            | Controlar la alineación de la siguiente instrucción o datos               |
| EQU              | Definir un nuevo símbolo igual a una expresión dada                       |
| DB               | Asignar memoria para uno o más bytes (inicializados)                      |
| DD               | Asignar memoria para una o más medias palabras de 16 bits (inicializadas) |
| DW               | Asignar memoria para una o más palabras de 32 bits (inicializadas)        |
| DQ               | Asignar memoria para una o más palabras dobles de 64 bits (inicializadas) |
| PROC             | Iniciar un procedimiento                                                  |
| ENDP             | Terminar un procedimiento                                                 |
| MACRO            | Iniciar una definición de macro                                           |
| ENDM             | Terminar una definición de macro                                          |
| PUBLIC           | Exportar un nombre definido en este módulo                                |
| EXTERN           | Importar un nombre de otro módulo                                         |
| INCLUDE          | Traer e incluir otro archivo                                              |
| IF               | Iniciar ensamblado condicional con base en una expresión dada             |
| ELSE             | Iniciar ensamblado condicional si la condición IF previa fue falsa        |
| ENDIF            | Terminar ensamblado condicional                                           |
| COMMENT          | Definir nuevo carácter de inicio de comentario                            |
| PAGE             | Generar un salto de página en el listado                                  |
| END              | Terminar el programa de ensamblador                                       |

Figura 7-3. Algunas de las seudoinstrucciones con que cuenta el ensamblador de Pentium II (MASM).

LIMIT EQU 4 \* BASE + 2000

Casi todos los ensambladores, incluido MASM, exigen que un símbolo se defina antes de usarse en una expresión como ésta.

Las siguientes cuatro seudoinstrucciones, DB, DD, DW y DQ, asignan espacio para una o más variables de 1, 2, 4 u 8 bytes, respectivamente. Por ejemplo,

TABLE DB 11, 23, 49

asigna espacio para 3 bytes y los inicializa con los valores 11, 23 y 49, respectivamente. También define el símbolo TABLE y lo hace igual a la dirección donde está almacenado 11.

Las seudoinstrucciones PROC y ENDP definen el principio y el final de procedimientos en lenguaje ensamblador, respectivamente. Los procedimientos en lenguaje ensamblador

tienen la misma función que en otros lenguajes de programación. Así mismo, MACRO y ENDM delimitan una definición de macro. Estudiaremos las macros más adelante en este capítulo.

Las siguientes dos seudoinstrucciones, PUBLIC y EXTERN, controlan la visibilidad de los símbolos. Es común escribir programas como una colección de archivos. Con frecuencia, un procedimiento de un archivo necesita invocar un procedimiento o acceder una palabra de datos definidos en otro archivo. Para hacer posibles estas referencias de un archivo a otro, un símbolo que estará disponible para otros archivos se exporta con PUBLIC. Así mismo, para evitar que el ensamblador proteste por el uso de un símbolo que no está definido en el archivo en curso, dicho símbolo puede declararse como EXTERN, lo que indica al ensamblador que se definirá en algún otro archivo. Los símbolos que no se declaran en ninguna de estas seudoinstrucciones tienen como alcance el archivo local. Este default implica que usar, digamos, FOO en varios archivos no genera conflictos porque cada definición es local respecto a su propio archivo.

La seudoinstrucción INCLUDE hace que el ensamblador traiga otro archivo y lo incluya entero dentro del archivo actual. Tales archivos incluidos a menudo contienen definiciones, macros y otras cosas que se necesitan en múltiples archivos.

Muchos ensambladores, incluido MASM, manejan ensamblado condicional. Por ejemplo,

```
WORDSIZE EQU 16
IF WORDSIZE GT 16
WSIZE: DW 32
ELSE
WSIZE: DW 16
ENDIF
```

asigna una sola palabra de 32 bits y llama a su dirección WSIZE. La palabra recibe el valor inicial 32 o 16, dependiendo del valor de WORDSIZE, que en este caso es 16. Esta construcción se usaría típicamente para escribir un programa que podría ensamblarse en máquinas de 16 bits (como el 8088) o de 32 bits (como el Pentium II). Al encerrar todo el código dependiente de la máquina entre IF y ENDIF, bastará con modificar una sola definición, la de WORDSIZE, para que el programa se ensamble automáticamente para cualquiera de los dos tamaños de palabra. Con este enfoque es posible mantener un programa fuente para varias máquinas objetivo (distintas), lo que facilita el desarrollo y mantenimiento del software. En muchos casos, todas las definiciones que dependen de la máquina, como WORDSIZE, se reúnen en un solo archivo, con diferentes versiones para diferentes máquinas. Si incluimos el archivo de definiciones correcto, podremos recompilar fácilmente el programa para diferentes máquinas.

La seudoinstrucción COMMENT permite al usuario cambiar el delimitador de comentarios a algo distinto del signo de punto y coma. PAGE sirve para controlar el listado que el ensamblador puede producir si se le solicita. Por último, END marca el final del programa.

Hay muchas otras seudoinstrucciones en MASM. Otros ensambladores para Pentium II cuentan con un conjunto diferente de seudoinstrucciones porque éstas no dependen de la arquitectura subyacente, sino del gusto de quien escribió el ensamblador.



## 7.2 MACROS

Los programadores en lenguaje ensamblador a menudo necesitan repetir sucesiones de instrucciones varias veces dentro de un programa. La forma más fácil de hacerlo es simplemente escribir las instrucciones requeridas donde sea que se necesiten. Sin embargo, si una secuencia es larga, o debe usarse muchas veces, escribirla una y otra vez resulta tedioso.

Una estrategia alternativa sería convertir la secuencia en un procedimiento e invocarlo cada vez que se necesite. Esta estrategia tiene la desventaja de que requiere la ejecución de una instrucción de llamada a procedimiento y una de retorno cada vez que se necesita la secuencia. Si la secuencia es corta —digamos, dos instrucciones— pero se usa con mucha frecuencia, el gasto extra de la llamada a procedimiento podría hacer al programa apreciablemente más lento. Las macros son una solución fácil y eficiente al problema de necesitar repetidamente la misma o casi la misma secuencia de instrucciones.

### 7.2.1 Definición, llamada y expansión de macros

Una **definición de macro** es una forma de asignar un nombre a un fragmento de texto. Una vez definida una macro, el programador puede escribir el nombre de la macro en lugar del fragmento de programa. Una macro es, de hecho, una abreviatura para un fragmento de texto. La figura 7-4(a) muestra un programa en lenguaje ensamblador para el Pentium II que intercambia dos veces el contenido de las variables *p* y *q*. Estas sucesiones podrían definirse como una macro, como se hace en la figura 7-4(b). Después de su definición, cada ocurrencia de *SWAP* hace que sea sustituida por las cuatro líneas:

```
MOV EAX,P
MOV EBX,Q
MOV Q,EAX
MOV P,EBX
```

El programador definió *SWAP* como una abreviatura de los cuatro enunciados anteriores.

Aunque diferentes ensambladores tienen notaciones un poco distintas para definir macros, todas requieren las mismas partes básicas en una definición de macro:

1. Una cabecera de macro que da el nombre de la macro que se está definiendo.
2. El texto que constituye el cuerpo de la macro.
3. Una seudoinstrucción que marca el final de la definición (por ejemplo, *ENDM*).

Cuando el ensamblador se topa con una definición de macro, la guarda en una tabla de definición de macros para usarla posteriormente. A partir de ese punto, cada vez que el nombre de la macro (*SWAP* en el ejemplo de la figura 7-4) aparezca como código de operación, el ensamblador la sustituirá por el cuerpo de la macro. El uso de un nombre de macro como código de operación se denomina **llamada a macro** y su sustitución por el cuerpo de la macro se denomina **expansión de macro**.

|           |      |           |
|-----------|------|-----------|
| MOV EAX,P | SWAP | MACRO     |
| MOV EBX,Q |      | MOV EAX,P |
| MOV Q,EAX |      | MOV EBX,Q |
| MOV P,EBX |      | MOV Q,EAX |
|           |      | MOV P,EBX |
|           |      | ENDM      |
| MOV EAX,P |      |           |
| MOV EBX,Q |      |           |
| MOV Q,EAX |      | SWAP      |
| MOV P,EBX |      |           |
|           |      | SWAP      |

(a) (b)

**Figura 7-4.** Código en lenguaje ensamblador para intercambiar P y Q dos veces. (a) Sin una macro. (b) Con una macro.

La expansión de macros se efectúa durante el proceso de ensamblado y no durante la ejecución del programa. Este punto es importante. El programa de la figura 7-4(a) y el de la figura 7-4(b) producen exactamente el mismo código en lenguaje de máquina. Si examinamos sólo el programa en lenguaje de máquina, es imposible saber si se usaron o no macros para generarlo. La razón es que una vez finalizada la expansión de macros, el ensamblador desecha las definiciones de macros. En el programa generado no queda ni rastro de ellas.

Las llamadas a macros no deben confundirse con llamadas a procedimientos. La diferencia básica es que una llamada a macro es una instrucción dirigida al ensamblador para que sustituya el nombre de la macro por el cuerpo de la misma. Una llamada a procedimiento es una instrucción de máquina que se inserta en el programa objeto y que posteriormente se ejecutará para llamar al procedimiento. En la figura 7-5 se comparan las llamadas a macros con las llamadas a procedimientos.

| Aspecto                                                                                          | Llamada a macro            | Llamada a procedimiento   |
|--------------------------------------------------------------------------------------------------|----------------------------|---------------------------|
| ¿Cuándo se hace la llamada?                                                                      | Al ensamblar               | Al ejecutarse el programa |
| ¿El cuerpo se inserta en el programa objeto en cada lugar en el que se efectúa la llamada?       | Sí                         | No                        |
| ¿Se inserta una instrucción de llamada a procedimiento en el programa objeto y luego se ejecuta? | No                         | Sí                        |
| ¿Se debe usar una instrucción de retorno después de efectuarse la llamada?                       | No                         | Sí                        |
| ¿Cuántas copias del cuerpo aparecen en el programa objeto?                                       | Una por llamada a la macro | 1                         |

**Figura 7-5.** Comparación de llamadas a macros y llamadas a procedimientos.

En lo conceptual, es mejor pensar que el proceso de ensamblado ocurre en dos pasadas. En la primera pasada, todas las definiciones de macros se guardan y las llamadas a macros se expanden. En la segunda pasada, el texto resultante se procesa como si estuviera en el programa original. Desde este punto de vista, el programa fuente se lee y se transforma en otro programa en el cual se han eliminado todas las definiciones de macros y en el que todas las llamadas a macros han sido sustituidas por sus cuerpos. La salida resultante, un programa en lenguaje ensamblador que no contiene macros, se alimenta al ensamblador.

Es importante tener en cuenta que un programa es una cadena de caracteres que incluye letras, dígitos, espacios, signos de puntuación y “retornos de carro” (cambio a una nueva línea). La expansión de macros consiste en sustituir ciertas subcadenas de esta cadena por otras cadenas de caracteres, sin fijarse en su significado.

### 7.2.2 Macros con parámetros

El recurso de macros que acabamos de describir puede servir para acortar programas en los que ocurre la misma secuencia exacta de instrucciones una y otra vez. Sin embargo, es común que un programa contenga varias secuencias de instrucciones que son casi idénticas pero no idénticas, como se ilustra en la figura 7-6(a). Aquí la primera secuencia intercambia *P* y *Q*, y la segunda secuencia intercambia *R* y *S*.

|     |       |         |              |
|-----|-------|---------|--------------|
| MOV | EAX,P | CAMBIAR | MACRO P1, P2 |
| MOV | EBX,Q |         | MOV EAX,P1   |
| MOV | Q,EAX |         | MOV EBX,P2   |
| MOV | P,EBX |         | MOV P2,EAX   |
|     |       |         | MOV P1,EBX   |
| MOV | EAX,R |         | ENDM         |
| MOV | EBX,S |         |              |
| MOV | S,EAX |         | CAMBIAR P, Q |
| MOV | R,EBX |         | CAMBIAR R, S |
| (a) |       | (b)     |              |

**Figura 7-6.** Sucesiones casi idénticas de enunciados. (a) Sin una macro. (b) Con una macro.

Los ensambladores de macros manejan el caso de secuencias casi idénticas permitiendo que las definiciones de macros incluyan **parámetros formales** y que las llamadas a macros incluyan **parámetros reales**. Cuando una macro se expande, cada parámetro formal que aparece en el cuerpo de la macro es sustituido por el parámetro real correspondiente. Los parámetros reales se colocan en el campo de operandos de la llamada a la macro. En la figura 7-6(b) se muestra el programa de la figura 7-6(a) reescrito empleando una macro con dos parámetros. Los símbolos *P1* y *P2* son los parámetros formales. Cada ocurrencia de *P1* den-

tro del cuerpo de una macro es sustituida por el primer parámetro real durante la expansión de la macro. Así mismo, *P2* es sustituido por el segundo parámetro real. En la llamada a macro

#### CAMBIAR P, Q

*P* es el primer parámetro real y *Q* es el segundo parámetro real. Así, los programas ejecutables que ambas partes de la figura 7-6 producen son idénticos.

### 7.2.3 Características avanzadas

Casi todos los procesadores de macros tienen una serie de características avanzadas que facilitan el trabajo del programador en lenguaje ensamblador. En esta sección examinaremos unas cuantas de las características avanzadas de MASM. Un problema que ocurre con todos los ensambladores que manejan macros es la duplicación de etiquetas. Suponga que una macro contiene una instrucción de ramificación condicional y una etiqueta a la cual se salta. Si la macro se llama dos o más veces, la etiqueta se duplicará y causará un error de ensamblado. Una solución es hacer que el programador proporcione como parámetro una etiqueta distinta en cada llamada. Una solución distinta (empleada por MASM) es permitir que una etiqueta se declare como **LOCAL**, y el ensamblador generará automáticamente una distinta en cada expansión de la macro. Algunos otros ensambladores tienen la regla de que las etiquetas numéricas son automáticamente locales.

MASM y casi todos los demás ensambladores permiten definir macros dentro de otras macros. Esta característica es útil sobre todo en combinación con el ensamblado condicional. Por lo regular, se define la misma macro en ambas partes de un enunciado **IF**, así:

```

M1 MACRO
 IF WORDSIZE GT 16
M2 MACRO
 ...
 ENDM
ELSE
M2 MACRO
 ...
 ENDM
ENDIF
ENDM

```

En cualquier caso, se definirá la macro *M2*, pero la definición dependerá de si el programa se está ensamblando en una máquina de 16 bits o en una de 32 bits. Si no se invoca *M1*, *M2* no se definirá.

Por último, las macros pueden llamar a otras macros, incluidas ellas mismas. Si una macro es recursiva, es decir, si se llama a sí misma, deberá pasarse a sí misma un parámetro que se modificará en cada expansión, y la macro deberá probar el parámetro y terminar la recursión cuando llegue a cierto valor. En caso contrario, el ensamblador podría ingresar en un ciclo infinito. Si esto sucede, el usuario deberá suspender explícitamente el ensamblador.

### 7.2.4 Implementación de un recurso de macros en un ensamblador

Para implementar un recurso de macros, un ensamblador debe poder realizar dos funciones: guardar definiciones de macros y expandir llamadas de macros. Examinaremos ambas funciones por turno.

El ensamblador debe mantener una tabla de todos los nombres de macros y, junto con cada nombre, un apuntador a su definición almacenada para poder recuperarla cuando la necesite. Algunos ensambladores tienen una tabla aparte para nombres de macros y algunos tienen una tabla de códigos de operación combinada en la que se guardan todas las instrucciones de máquina, pseudoinstrucciones y nombres de macros.

Cuando se llega a una definición de macro, se crea una entrada de tabla que da el nombre de la macro, el número de parámetros formales y un apuntador a otra tabla —la tabla de definiciones de macros— en la que se guardará el cuerpo de la macro. En este momento también se construye una lista de los parámetros formales que se usarán al procesar la definición. Luego se lee el cuerpo de la macro y se guarda en la tabla de definiciones de macros. Los parámetros formales que ocurren dentro del cuerpo se indican con algún símbolo especial. Por ejemplo, la representación interna de la definición de la macro *CAMBIAR* con un signo de punto y coma como “retorno de carro” y el signo “&” como símbolo de parámetro formal es:

```
MOV EAX,&P1; MOV EBX,&P2; MOV &P2,EAX; MOV &P1,EBX;
```

Dentro de la tabla de definiciones de macros el cuerpo de la macro no es más que una cadena de caracteres.

Durante la primera pasada del ensamblador, se buscan en tablas los códigos de operación y las macros se expanden. Cada vez que se encuentra una definición de macro, se guarda en la tabla de macros. Cuando se invoca una macro, el ensamblador suspende temporalmente la lectura del dispositivo de entrada y comienza a leer del cuerpo almacenado de la macro. Los parámetros formales extraídos del cuerpo almacenado de la macro se sustituyen por los parámetros reales proporcionados en la llamada. La presencia de un “&” antes de los parámetros formales permite al ensamblador reconocerlos fácilmente.

## 7.3 EL PROCESO DE ENSAMBLADO

En las secciones que siguen describiremos brevemente cómo funciona un ensamblador. Aunque cada máquina tiene un lenguaje ensamblador distinto, el proceso de ensamblado tiene suficientes similitudes en las distintas máquinas que es posible describirlo en términos generales.

### 7.3.1 Ensambladores de dos pasadas

Dado que un programa en lenguaje ensamblador consta de una serie de enunciados de una sola línea, al principio podría parecer natural tener un ensamblador que lea un enunciado, lo traduzca a lenguaje de máquina y luego escriba la instrucción de lenguaje de máquina gene-

rada en un archivo, escribiendo al mismo tiempo la línea correspondiente del listado, si se va a producir, en otro archivo. Este proceso se repetiría hasta haber traducido todo el programa. Lo malo es que tal estrategia no funciona.

Considere la situación en la que el primer enunciado es un salto a *L*. El ensamblador no podrá ensamblar este enunciado hasta que no conozca la dirección del enunciado *L*. Dicho enunciado podría estar cerca del final del programa, y para el ensamblador sería imposible encontrar la dirección sin leer antes casi todo el programa. Esto se conoce como el **problema de la referencia adelantada (hacia adelante)**, porque se ha usado un símbolo, *L*, antes de definirse; es decir, se ha hecho referencia a un símbolo cuya definición ocurrirá posteriormente.

Las referencias adelantadas se pueden manejar de dos maneras. Primera, el ensamblador podría leer el programa fuente dos veces. Cada lectura del programa fuente es una **pasada**; cualquier traductor que lee dos veces el programa de entrada es un **traductor de dos pasadas**. En la primera pasada de un ensamblador de dos pasadas, se reúnen las definiciones de símbolos, incluidas las etiquetas de enunciados, y se guardan en una tabla. Para cuando se inicia la segunda pasada, ya se conocen los valores de todos los símbolos, así que no quedan ya referencias adelantadas y cada enunciado puede leerse, ensamblarse y enviarse hacia la salida. Aunque este enfoque requiere una pasada extra por las entradas, es conceptualmente sencillo.

La segunda estrategia consiste en leer el programa en lenguaje ensamblador una vez, convertirlo en una forma intermedia, y guardar esta forma intermedia en una tabla en la memoria. Luego se efectúa otra pasada, pero ahora por la tabla, no por el programa fuente. Si hay suficiente memoria (o memoria virtual), este enfoque ahorra tiempo de E/S. Si se va a producir un listado, entonces hay que guardar todo el enunciado fuente, incluidos los comentarios. Si no se requiere un listado, la forma intermedia puede reducirse a lo indispensable.

En cualquier caso, otra tarea de la primera pasada es guardar todas las definiciones de macros y expandir las llamadas conforme se llega a ellas. Por ello, la definición de los símbolos y la expansión de las macros generalmente se combina en una sola pasada.

### 7.3.2 Primera pasada

La principal función de la primera pasada es construir una tabla llamada **tabla de símbolos**, que contiene los valores de todos los símbolos. Un símbolo es una etiqueta o bien un valor al que se le asigna un nombre simbólico mediante una pseudoinstrucción como

```
BUFSIZE EQU 8192
```

Al asignar un valor a un símbolo en el campo de etiqueta de una instrucción, el ensamblador debe saber qué dirección tendrá esa instrucción durante la ejecución del programa. Para seguir la pista a la dirección de tiempo de ejecución de la instrucción que se está ensamblando, el ensamblador mantiene una variable llamada **contador de posiciones de instrucciones** (*ILC, Instruction Location Counter*). Esta variable se pone en 0 al principio de la primera pasada y se incrementa en una cantidad igual a la longitud de la instrucción cada vez que se procesa una instrucción, como se muestra en la figura 7-7. Este ejemplo es para el **Pentium II**.

En adelante no daremos ejemplos de SPARC (ni Motorola) porque las diferencias entre los lenguajes ensambladores no son muy importantes, y deberá bastar con un ejemplo. Además, si hubiera un concurso para hallar el lenguaje ensamblador más incomprensible del mundo, el SPARC sería uno de los favoritos.

| Etiqueta  | Cód. op. | Operandos | Comentarios                 | Longitud | ILC |
|-----------|----------|-----------|-----------------------------|----------|-----|
| MARÍA:    | MOV      | EAX, I    | EAX = I                     | 5        | 100 |
|           | MOV      | EBX, J    | EBX = J                     | 6        | 105 |
| ROBERTA:  | MOV      | ECX, K    | ECX = K                     | 6        | 111 |
|           | IMUL     | EAX, EAX  | EAX = I * I                 | 2        | 117 |
|           | IMUL     | EBX, EBX  | EBX = J * J                 | 3        | 119 |
|           | IMUL     | ECX, ECX  | ECX = K * K                 | 3        | 122 |
| MARILYN:  | ADD      | EAX, EBX  | EAX = I * I + J * J         | 2        | 125 |
|           | ADD      | EAX, ECX  | EAX = I * I + J * J + K * K | 2        | 127 |
| STEPHANY: | JMP      | DONE      | Saltar a DONE               | 5        | 129 |

Figura 7-7. El contador de localidades de instrucciones (ILC) sigue la pista a las direcciones donde las instrucciones se cargarán en la memoria. En este ejemplo, los enunciados anteriores a MARÍA ocupan 100 bytes.

La primera pasada de casi todos los ensambladores utiliza por lo menos tres tablas: la tabla de símbolos, la tabla de seudoinstrucciones y la tabla de códigos de operación. Si se necesita, también se mantiene una tabla de literales. La tabla de símbolos tiene una entrada para cada símbolo, como se muestra en la figura 7-8. Los símbolos se definen ya sea usándolos como etiquetas o por definición explícita (por ejemplo, EQU). Cada entrada de la tabla de símbolos contiene el símbolo mismo (o un apuntador a él), su valor numérico y a veces otra información. Esta información adicional podría incluir

- 1. La longitud del campo de datos asociado al símbolo.
- 2. Los bits de reubicación. (¿El símbolo cambia de valor si el programa se carga en una dirección diferente de la que el ensamblador supuso?)
- 3. Si el símbolo estará o no accesible afuera del procedimiento.

| Símbolo  | Valor | Otra información |
|----------|-------|------------------|
| MARÍA    | 100   |                  |
| ROBERTA  | 111   |                  |
| MARILYN  | 125   |                  |
| STEPHANY | 129   |                  |

Figura 7-8. Tabla de símbolos para el programa de la figura 7-7.

La tabla de códigos de operación contiene al menos una entrada para cada código de operación simbólico (mnemónico) del lenguaje ensamblador. En la figura 7-9 se muestra parte de una tabla de códigos de operación. Cada entrada contiene el código de operación simbólico, dos operandos, el valor numérico del código de operación, la longitud de la ins-

trucción y un número de tipo que divide los códigos de operación en grupos dependiendo del número y el tipo de operandos.

| Cód. op. | Primer operando | Segundo operando | Cód. op. hexadecimal | Longitud de instrucción | Clase de instrucción |
|----------|-----------------|------------------|----------------------|-------------------------|----------------------|
| AAA      | —               | —                | 37                   | 1                       | 6                    |
| ADD      | EAX             | immed32          | 05                   | 5                       | 4                    |
| ADD      | reg             | reg              | 01                   | 2                       | 19                   |
| AND      | EAX             | immed32          | 25                   | 5                       | 4                    |
| AND      | reg             | reg              | 21                   | 2                       | 19                   |

Figura 7-9. Extractos de la tabla de códigos de operación para un ensamblador de Pentium II.

Por ejemplo, consideremos el código de operación ADD. Si una instrucción ADD contiene EAX como primer operando y una constante de 32 bits (immed32) como segundo operando, entonces se usa el código de operación 0x05 y la longitud de la instrucción es de 5 bytes. (Las constantes que se pueden expresar en 8 o 16 bits usan diferentes códigos de operación, que no se muestran.) Si se usa ADD con dos registros como operandos, la instrucción ocupa 2 bytes y tiene el código de operación 0x01. La clase de instrucción (arbitraria) 19 se asigna a todas las combinaciones código de operación-operando que siguen las mismas reglas y deben procesarse de la misma manera que ADD con dos registros como operandos. La clase de instrucción designa en realidad un procedimiento dentro del ensamblador que se invoca para procesar todas las instrucciones de un tipo dado.

Algunos ensambladores permiten a los programadores escribir instrucciones empleando direccionamiento inmediato aunque no exista una instrucción correspondiente en el lenguaje objetivo. Tales instrucciones “seudoinmediatas” se manejan como sigue. El ensamblador asigna memoria para el operando inmediato al final del programa y genera una instrucción que hace referencia a ella. Por ejemplo, la mainframe IBM 3090 no tiene instrucciones inmediatas; no obstante, los programadores pueden escribir

L 14,=F'5'

para cargar el registro 14 con una constante de palabra completa (Full) con valor de 5. De este modo, el programador evita escribir explícitamente una seudoinstrucción para asignar una palabra que se inicializa con 5, darle una etiqueta y luego usarla en la instrucción L. Las constantes para las que el ensamblador automáticamente reserva memoria se llaman literales. Además de ahorrar al programador un poco de trabajo, las literales hacen más comprensibles los programas porque hacen evidente el valor de la constante en el enunciado fuente. La primera pasada del ensamblador debe construir una tabla de todas las literales empleadas en el programa. Nuestras tres computadoras de ejemplo tienen instrucciones inmediatas, por lo que sus ensambladores no manejan literales. Las instrucciones inmediatas son muy comunes hoy día, pero antes eran raras. Es probable que lo extendido del uso de literales hizo que los diseñadores de máquinas se dieran cuenta de que el direccionamiento

inmediato era una buena idea. Si se necesitan literales, se mantiene una tabla de literales durante el ensamblado, y se crea una nueva entrada cada vez que se encuentra una literal. Después de la primera pasada, esta tabla se ordena y se eliminan los duplicados.

La figura 7-10 muestra un procedimiento que podría servir como base para la primera pasada de un ensamblador. El estilo de programación es notable por derecho propio. Los nombres de procedimiento se escogieron de modo que den una idea clara de lo que los procedimientos hacen. Lo más importante es que la figura 7-10 representa un bosquejo de la primera pasada que, aunque no es completo, es un buen punto de partida. Es lo bastante corto como para entenderse fácilmente y deja bien claro cuál debe ser el siguiente paso: escribir los procedimientos que se usan en él.

Algunos de estos procedimientos serán relativamente cortos, como *verif\_simbolo*, que simplemente devuelve el símbolo como cadena de caracteres si lo hay y null si no lo hay. Otros procedimientos, como *obt\_long\_de\_tipo1* y *obt\_long\_de\_tipo2*, podrían ser más largos e invocar otros procedimientos. En general, el número de tipos no será dos, sino que dependerá del lenguaje que se ensambla y del número de tipos de instrucciones que tiene.

Estructurar los programas de este modo tiene otras ventajas además de la facilidad de programación. Si el ensamblador está siendo escrito por un grupo de personas, los distintos procedimientos pueden repartirse entre los programadores. Todos los detalles (desagradables) de la obtención de las entradas se ocultan en *leer\_sig\_linea*. Si llegaran a cambiar —por ejemplo, por un cambio de sistema operativo— sólo se verá afectado un procedimiento subsidiario; el procedimiento *primera\_pasada* en sí no requerirá modificaciones.

A medida que lee el programa, la primera pasada tiene que analizar sintácticamente cada línea para encontrar el código de operación (digamos **ADD**), buscar su tipo (básicamente, el patrón de operandos) y calcular la longitud de la instrucción. Esta información también se necesita en la segunda pasada, así que podríamos registrarla explícitamente para no tener que analizar la línea desde cero la próxima vez. Sin embargo, la reescritura del archivo de entrada causa más E/S. La ventaja relativa de hacer más E/S para eliminar el análisis sintáctico o hacer menos E/S y más análisis depende de las velocidades relativas de la CPU y el disco, la eficiencia del sistema de archivos y otros factores. En este ejemplo escribiremos un archivo temporal que contiene el tipo, código de operación, longitud y la línea de entrada real. Esta línea es la que se leerá en la segunda pasada en lugar del archivo de entrada original.

Cuando se lee la seudoinstrucción **END**, termina la primera pasada. En este momento se ordenan la tabla de símbolos y la de literales si es necesario. También se buscan y se eliminan entradas repetidas en la tabla de literales.

### 7.3.3 Segunda pasada

La función de la segunda pasada es generar el programa objeto y posiblemente imprimir el listado de ensamblado. Además, la segunda pasada debe producir cierta información que el enlazador necesita para enlazar procedimientos ensamblados en diferentes momentos para producir un solo archivo ejecutable. La figura 7-11 muestra un bosquejo de procedimiento para la segunda pasada.

```
public static void primera_pasada() {
 // Este procedimiento es un bosquejo de la primera pasada de un ensamblador sencillo.
 boolean mas_entradas = true; // bandera que detiene la primera pasada
 String linea, simbolo, literal, codop; // campos de la instrucción
 int contador_posic, longitud, valor, tipo; // variables diversas
 final int ENUNC_FINAL = -2; // indica fin de las entradas

 contador_posic = 0; // ensamblar la primera instrucción en 0
 inicializar_tablas(); // inicialización general

 while (mas_entradas) { // END asigna false a mas_entradas
 linea = leer_sig_linea(); // obtener una línea de entrada
 longitud = 0; // número de bytes en la instrucción
 tipo = 0; // tipo (formato) de la instrucción

 if (linea_no_es_comentario(linea)) {
 simbolo = verif_simbolo(linea); // ¿tiene rótulo esta línea?
 if (simbolo != null) // en tal caso, registrar símbolo y valor
 ingresar_simbolo_nuevo(simbolo, contador_posic);
 (literal = verif_literal(linea); // ¿la línea contiene una literal?
 if (literal != null) // en tal caso, ingresarla en la tabla
 ingresar_literal_nueva(literal);

 // Ahora determinar el tipo del cód. op. -1 indica código no válido.
 codop = extraer_codop(linea); // buscar mnemónico del código de operación
 tipo = buscar_tabla_cods(codop); // buscar formato, por ejemplo, OP REG1, REG2
 if (tipo < 0) // si no es cód. op., ¿es seudoinstrucción?
 tipo = buscar_tabla_seudo(codop);
 switch(tipo) { // determinar longitud de esta instrucción
 case 1: longitud = obt_long_de_tipo1(linea); break;
 case 2: longitud = obt_long_de_tipo2(linea); break;
 // otros casos aquí
 }
 }

 escrib_arch_temp(tipo, codop, longitud, linea); // inf. útil p/segunda pasada
 contador_posic = contador_posic + longitud; // actualizar cont. posic.
 if (tipo == ENUNC_FINAL) { // ¿terminamos con las entradas?
 mas_entradas = false; // entonces, realizar tareas de aseo
 rebob_temp_segunda_pasada(); // como rebobinar el archivo temporal
 ordenar_tabla_lit(); // y ordenar la tabla de literales
 quitar_literales_rep(); // y eliminar de ella los duplicados
 }
 }
}
```

Figura 7-10. Primera pasada de un ensamblador sencillo.

El funcionamiento de la segunda pasada es parecido al de la primera: se leen las líneas una por una y se procesan una por una. Dado que ya escribimos el tipo, código de operación y longitud al principio de cada línea (en el archivo temporal), se leen todos estos datos para

```

public static void segunda_pasada() {
 // Este procedimiento es un bosquejo de la segunda pasada de un ensamblador sencillo.
 boolean mas_entradas = true; // bandera que detiene la pasada
 String línea, codop; // campos de la instrucción
 int contador_posic, longitud, tipo; // variables diversas
 final int ENUNC_FINAL = -2; // indica fin de las entradas
 final int MAX_CODIGO = 16; // límite de bytes de código por instruc.
 byte código[] = new byte[MAX_CODIGO]; // contiene el código generado por instruc.

 contador_posic = 0; // ensamblar primera instrucción en 0

 while (mas_entradas) { // END asigna false a mas_entradas
 tipo = leer_tipo(); // obtener campo de tipo de la sig. línea
 codop = leer_codop(); // obtener campo de cód. op. de la sig. línea
 longitud = leer_long(); // obtener campo de longitud de la sig. línea
 línea = leer_línea(); // obtener la línea de entrada real

 if (tipo != 0) { // el tipo 0 es para comentarios
 switch(tipo) { // generar código de salida
 case 1: eval_tipo1(codop, longitud, línea, código); break;
 case 2: eval_tipo2(codop, longitud, línea, código); break;
 // otros casos aquí
 }
 }

 escribir_salida(código); // escribir el código binario
 escribir_listado(código, línea); // imprimir una línea del listado
 contador_posic = contador_posic + longitud; // actualizar cont. pos.
 if (tipo == ENUNC_FINAL) { // ¿terminamos con las entradas?
 mas_entradas = false; // entonces, realizar tareas de aseo
 finalizar(); // diversas cosas
 }
 }
}

```

**Figura 7-11.** Segunda pasada de un ensamblador sencillo.

ahorrar algo de análisis. El trabajo principal de la generación de código corre por cuenta de los procedimientos *eval\_tipo1*, *eval\_tipo2*, etc. Cada uno maneja un patrón específico, como el de un código de operación y dos operandos en registros, genera el código binario para la instrucción y lo devuelve en *código*. Luego se escribe el código. Lo más probable es que *escribir\_código* se limite a colocar en un buffer el código binario acumulado y escriba el archivo en disco en fragmentos grandes a fin de reducir el tráfico con el disco.

El enunciado fuente original y el código objeto generado a partir de él (en hexadecimal) pueden imprimirse o colocarse en un buffer para imprimirse posteriormente. Una vez que se ha ajustado el ILC, se adquiere el siguiente enunciado.

Hasta ahora hemos supuesto que el programa fuente no contiene errores. Cualquiera que haya escrito alguna vez un programa, en cualquier lenguaje, sabe qué tan realista es ese supuesto. Entre los errores más comunes están:

1. Se usó un símbolo pero no se definió.
2. Se definió un símbolo más de una vez.
3. El nombre que está en el campo de código de operación no corresponde a un código válido.
4. Un código de operación no tiene suficientes operandos.
5. Un código de operación tiene demasiados operandos.
6. Un número octal contiene un 8 o un 9.
7. Uso incorrecto de registros (por ejemplo, un salto a un registro).
8. Falta la instrucción **END**.

Los programadores son muy ingeniosos y siempre se les ocurren nuevos errores. Los errores de símbolo no definido a menudo se deben a errores de dedo, por lo que un ensamblador inteligente podría tratar de deducir cuál de los símbolos definidos se parece más al que no está definido y usarlo en vez de éste. Poco puede hacerse para corregir casi todos los demás errores. Lo mejor que puede hacer el ensamblador con un enunciado descarriado es imprimir un mensaje de error y tratar de continuar con el ensamblado.

### 7.3.4 La tabla de símbolos

Durante la primera pasada del proceso de ensamblado, el ensamblador acumula información acerca de los símbolos y sus valores, la cual debe guardarse en la tabla de símbolos para consultarla durante la segunda pasada. Hay varias formas de organizar la tabla de símbolos, y a continuación describiremos brevemente algunos de ellos. En todos los casos se intenta simular una **memoria asociativa**, que conceptualmente es un conjunto de pares (símbolo, valor). Dado el símbolo, la memoria asociativa debe producir el valor.

La técnica más sencilla es implementar la tabla de símbolos como un arreglo de pares cuyo primer elemento es el símbolo (o apunta a él) y cuyo segundo elemento es el valor (o apunta a él). Dado un símbolo qué buscar, la rutina de la tabla de símbolos simplemente recorre la tabla linealmente hasta que encuentra una entrada que coincide. Este método es fácil de programar pero es lento porque, en promedio, habrá que examinar media tabla en cada búsqueda.

Otra forma de organizar la tabla de símbolos consiste en ordenarla por símbolo y usar el algoritmo de **búsqueda binaria** para localizar un símbolo. Este algoritmo opera comparando la entrada que está a la mitad de la tabla con el símbolo. Si el símbolo es alfabéticamente anterior a dicha entrada, quiere decir que se encuentra en la primera mitad de la tabla; si no, deberá estar en la segunda mitad. Si el símbolo es igual a la entrada media, la búsqueda termina.

Suponiendo que la entrada media no es igual al símbolo buscado, al menos sabemos en qué mitad de la tabla debemos buscarlo. Ahora podemos aplicar la búsqueda binaria a la mitad correcta y, o bien encontraremos el símbolo o sabremos en qué cuarto de la tabla está.

Si aplicamos este algoritmo recursivamente, una tabla con  $n$  entradas puede examinarse en aproximadamente  $\log_2 n$  intentos. Obviamente, este método es mucho más rápido que una búsqueda lineal, pero requiere mantener ordenada la tabla.

Una forma totalmente distinta de simular una memoria asociativa es usar una técnica llamada **codificación por dispersión (hash coding)**. Este enfoque necesita una función "hash" que establece una correspondencia entre los símbolos y enteros dentro del intervalo 0 a  $k-1$ . Una posible función sería multiplicar entre sí los códigos ASCII de los caracteres de los símbolos, hacer caso omiso del desbordamiento, y obtener el residuo de la división del resultado entre  $k$ , o dividir el resultado entre un número primo. De hecho, se puede usar casi cualquier función que dé una distribución uniforme de los valores hash. Los símbolos pueden guardarse en una tabla que consiste en  $k$  **cuadros** numerados de 0 a  $k-1$ . Todos los pares (símbolo, valor) cuyo símbolo da  $i$  por hashing se almacenan en una lista enlazada a la que apunta la entrada  $i$  de la tabla hash. Con  $n$  símbolos y  $k$  entradas en la tabla hash, una lista promedio tendrá una longitud  $n/k$ . Si escogemos  $k$  aproximadamente igual a  $n$ , generalmente podremos localizar los símbolos con una sola consulta. Si ajustamos  $k$  podremos reducir el tamaño de la tabla a expensas de hacer más lentas las búsquedas. La codificación hash se ilustra en la figura 7.12.

## 7.4 ENLAZADO Y CARGA

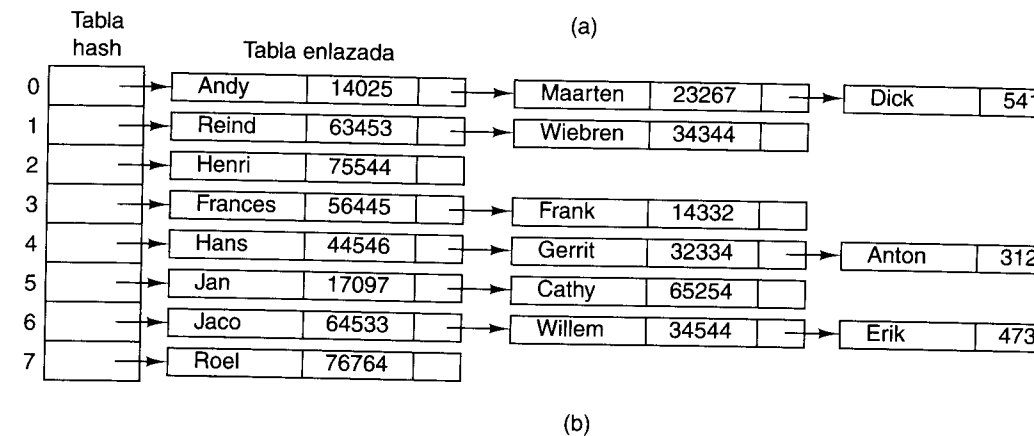
Casi todos los programas consisten en más de un procedimiento. Los compiladores y ensambladores generalmente traducen un procedimiento a la vez y colocan la salida traducida en disco. Antes de que el programa pueda ejecutarse, es preciso encontrar todos los procedimientos traducidos y enlazarlos en forma correcta. Si no se cuenta con memoria virtual, el programa enlazado también se deberá cargar explícitamente en la memoria principal. Los programas que realizan estas funciones reciben diversos nombres, como **enlazador (linker)**, **cargador de enlace (linking loader)** y **editor de enlace (linkage editor)**. La traducción completa de un programa fuente requiere dos pasos, como se muestra en la figura 7-13:

1. Compilación o ensamblado de los procedimientos fuente.
2. Enlazado de los módulos objeto.

El compilador o ensamblador se encarga del primer paso, y el segundo corre por cuenta del enlazador.

La traducción de procedimiento fuente a módulo objeto representa un cambio de nivel porque el lenguaje fuente y el lenguaje objetivo tienen diferentes instrucciones y notación. El proceso de enlazado, en cambio, no representa un cambio de nivel, ya que tanto las entradas del enlazador como sus salidas son programas para la misma máquina virtual. La función del enlazador es reunir procedimientos traducidos por separado y enlazarlos para que se ejecuten como una unidad llamada **programa binario ejecutable**. En MS-DOS Windows 95/98 y NT, los módulos objeto tienen la extensión *.obj* y los programas binarios ejecutables tienen la extensión *.exe*. En UNIX, los módulos objetos tienen la extensión *.o*; los programas binarios ejecutables no tienen extensión.

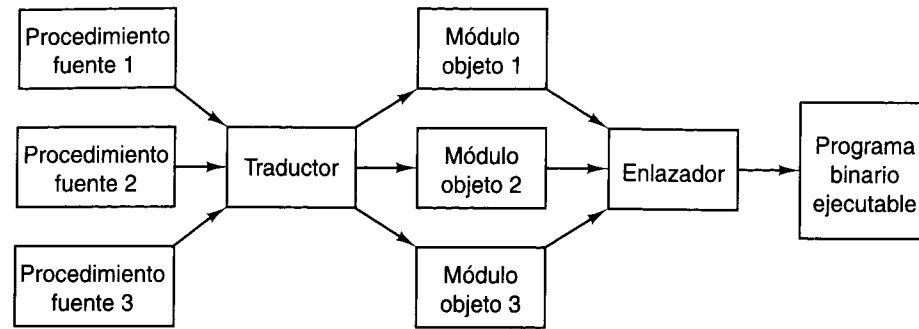
|         |       |   |
|---------|-------|---|
| Andy    | 14025 | 0 |
| Anton   | 31253 | 4 |
| Cathy   | 65254 | 5 |
| Dick    | 54185 | 0 |
| Erik    | 47357 | 6 |
| Frances | 56445 | 3 |
| Frank   | 14332 | 3 |
| Gerrit  | 32334 | 4 |
| Hans    | 44546 | 4 |
| Henri   | 75544 | 2 |
| Jan     | 17097 | 5 |
| Jaco    | 64533 | 6 |
| Maarten | 23267 | 0 |
| Reind   | 63453 | 1 |
| Roel    | 76764 | 7 |
| Willem  | 34544 | 6 |
| Wiebren | 34344 | 1 |



**Figura 7-12.** Codificación hash o por dispersión. (a) Símbolos, valores y códigos hash derivados de los símbolos. (b) Tabla hash de ocho entradas con listas enlazadas de símbolos y valores.

Los compiladores y ensambladores traducen cada procedimiento fuente como entidad individual por una buena razón. Si un compilador o ensamblador leyera una serie de procedimientos fuente y produjera directamente un programa en lenguaje de máquina listo para ejecutarse, la modificación de un enunciado de un procedimiento fuente requeriría volver a traducir todos los procedimientos fuente.

Si se usa la técnica de módulos objeto separados de la figura 7-13, sólo es necesario volver a traducir el procedimiento que se modificó y no los demás, aunque sí es necesario volver a enlazar todos los módulos objeto. Sin embargo, el enlazado suele ser mucho más rápido que la traducción; así, el proceso de dos pasos de traducir y enlazar puede ahorrar mucho tiempo durante el desarrollo de un programa. Esta ganancia es importante sobre todo si los programas tienen cientos o miles de módulos.



**Figura 7-13.** La generación de un programa binario ejecutable a partir de una colección de procedimientos fuente traducidos de forma independiente requiere un enlazador.

### 7.4.1 Tareas que realiza el enlazador

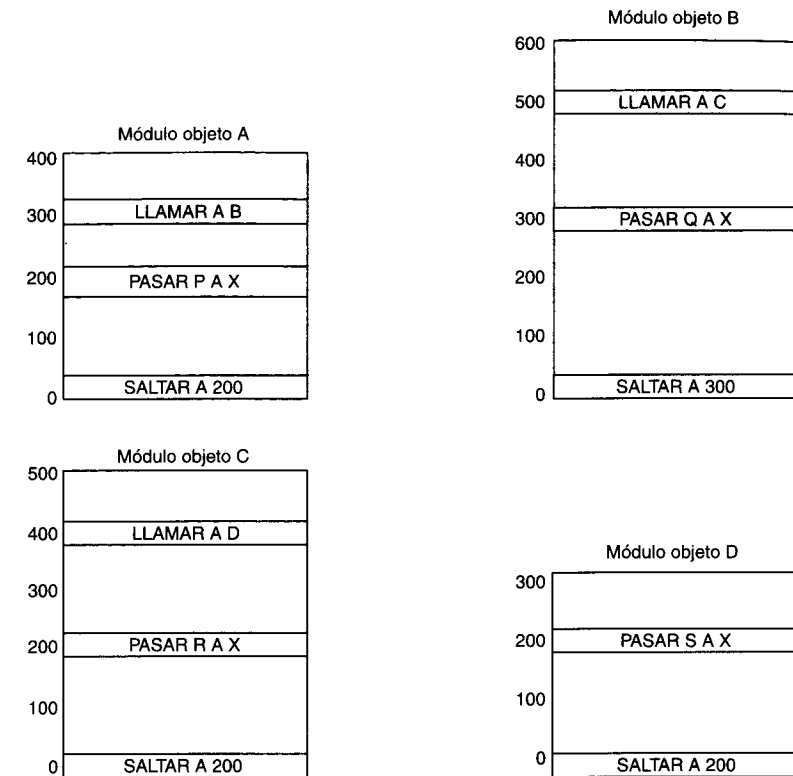
Al principio de la primera pasada del proceso de ensamblado, el contador de posiciones de instrucciones se pone en 0. Este paso equivale a suponer que el módulo objeto estará en la dirección (virtual) 0 durante su ejecución. En la figura 7-14 se muestran cuatro módulos objeto para una máquina genérica. En este ejemplo, cada módulo comienza con una instrucción de **SALTAR** a una instrucción de **PASAR** dentro del módulo.

Para ejecutar el programa, el enlazador obtiene los módulos objeto de la memoria principal a fin de formar la imagen de programa binario ejecutable, como se muestra en la figura 7-15(a). De lo que se trata es de crear una imagen exacta del espacio de direcciones virtual del programa ejecutable dentro del enlazador y colocar todos los módulos objeto en sus posiciones correctas. Si no hay suficiente memoria (virtual) para formar la imagen, puede usarse un archivo en disco. Por lo regular se usa una sección pequeña de la memoria que comienza en la dirección cero para vectores de interrupción, comunicación con el sistema operativo, atrapar apuntadores no inicializados u otras cosas, por lo que los programas casi siempre comienzan arriba de 0. En esta figura hemos iniciado (arbitrariamente) los programas en la dirección 100.

El programa de la figura 7-15(a), aunque ya se cargó en la imagen del archivo binario ejecutable, todavía no está listo para ejecutarse. Considere qué sucedería si la ejecución comenzara con la instrucción que está al principio del módulo A. El programa no saltaría a la instrucción **PASAR** como debería, porque esa instrucción ahora está en 300. De hecho, todas las instrucciones que hacen referencia a la memoria fallarían por la misma razón.

Este problema, llamado **problema de reubicación**, ocurre porque cada módulo objeto de la figura 7-14 representa un espacio de direcciones individual. En una máquina que tiene espacio de direcciones segmentado, como la Pentium II, cada módulo objeto podría en teoría tener su propio espacio de direcciones si se le coloca en su propio segmento. Sin embargo, os/2 es el único sistema operativo para el Pentium II que reconoce este concepto. Todas las versiones de Windows y UNIX sólo reconocen un espacio de direcciones lineal, así que todos los módulos objeto deben fusionarse en un solo espacio de direcciones.

Además, las instrucciones de llamada a procedimiento de la figura 7-15(a) tampoco funcionan. En la dirección 400, la intención del programador era llamar al módulo objeto B, pero



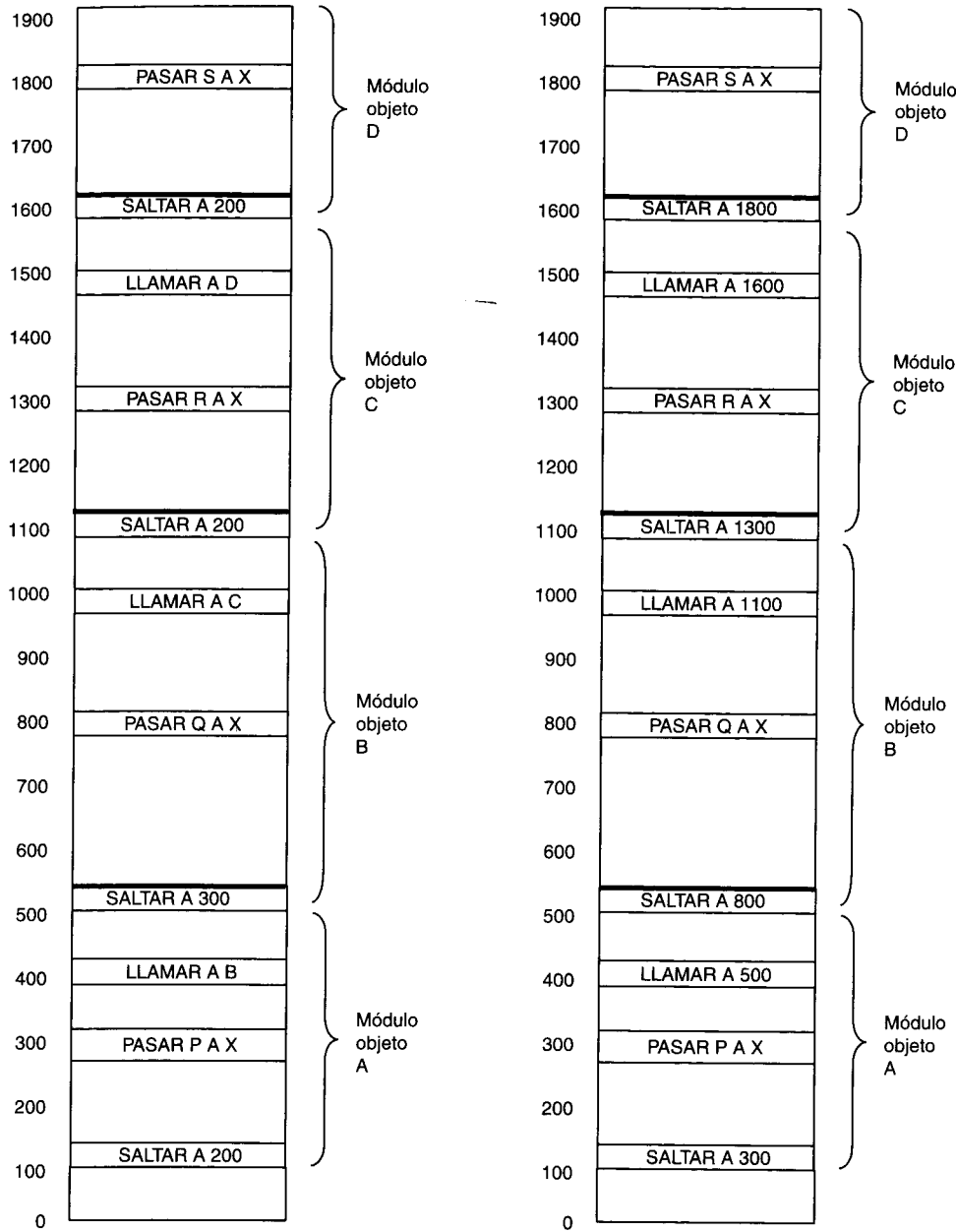
**Figura 7-14.** Cada módulo tiene su propio espacio de direcciones, que comienza en 0.

dado que cada procedimiento se traduce por su cuenta, el ensamblador no tiene forma de saber qué dirección debe insertar en la instrucción **LLAMAR A B**. La dirección del módulo objeto B no se conoce antes del enlazado. Este problema se denomina **problema de referencia externa**. El enlazador puede resolver ambos problemas.

El enlazador fusiona los espacios de direcciones de los módulos objeto en un solo espacio de direcciones lineal siguiendo estos pasos:

1. Construcción de una tabla con todos los módulos objeto y sus longitudes.
2. Con base en esta tabla, se asigna una dirección de inicio a cada módulo objeto.
3. Se buscan todas las instrucciones que hacen referencia a la memoria y se suma a cada una de ellas una **constante de reubicación** igual a la dirección de inicio de su módulo.
4. Se buscan todas las instrucciones que hacen referencia a otros procedimientos y se inserta la dirección del procedimiento en cuestión.





**Figura 7-15.** (a) Los módulos objeto de la figura 7-14 después de colocarse en la imagen binaria pero antes de reubicarse y enlazarse. (b) Los mismos módulos objeto una vez efectuados el enlazado y la reubicación. Juntos forman un programa binario ejecutable, listo para trabajar.

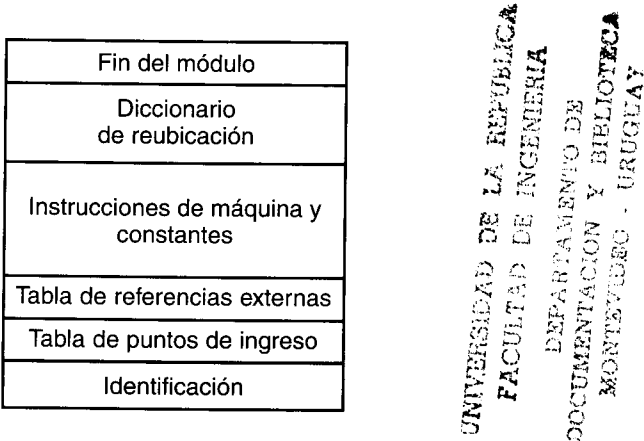
A continuación se muestra la tabla de módulos objeto que se construye en el paso 1 para los módulos de la figura 7-15. La tabla da el nombre, longitud y dirección de inicio de cada módulo.

| Módulo | Longitud | Dirección de inicio |
|--------|----------|---------------------|
| A      | 400      | 100                 |
| B      | 600      | 500                 |
| C      | 500      | 1100                |
| D      | 300      | 1600                |

La figura 7-15(b) muestra el aspecto que el espacio de direcciones de la figura 7-15(a) tiene después de que el enlazador ha llevado a cabo estos pasos.

7.4.2 Estructura de un módulo objeto

Los módulos objeto a menudo contienen seis partes, que se muestran en la figura 7-16. La primera parte contiene el nombre del módulo, cierta información que el enlazador necesita, como las longitudes de las distintas partes del módulo, y a veces la fecha de ensamblado.



**Figura 7-16.** Estructura interna de un módulo objeto producido por un traductor.

La segunda parte del módulo objeto es una lista de los símbolos definidos en el módulo a los que otros módulos podrían hacer referencia, junto con sus valores. Por ejemplo, si el módulo consiste en un procedimiento llamado *bigbug*, la tabla de puntos de ingreso contendría la cadena de caracteres “bigbug” seguida de la dirección a la que corresponde. El programador en lenguaje ensamblador indica qué símbolos deben declararse como **puntos de ingreso** empleando una seudoinstrucción como **PUBLIC** de la figura 7-3.

La tercera parte del módulo objeto consiste en una lista de los símbolos que se usan en el módulo pero se definen en otros módulos, junto con una lista que indica cuáles instrucciones

de máquina usan cuáles símbolos. El enlazador necesita la segunda lista para poder insertar las direcciones correctas en las instrucciones que usan símbolos externos. Un procedimiento puede llamar a otros procedimientos que se tradujeron de forma independiente declarando como externos los nombres de los procedimientos a invocar. El programador en lenguaje ensamblador indica cuáles símbolos se declararán como **símbolos externos** usando una seudoinstrucción como **EXTERN** de la figura 7-3. En algunas computadoras los puntos de ingreso y las referencias externas se combinan en una sola tabla.

La cuarta parte del módulo objeto es el código ensamblado y las constantes. Esta parte del módulo objeto es la única que se cargará en la memoria para ejecutarse. Las otras cinco partes serán utilizadas por el enlazador y se desecharán antes de que se inicie la ejecución.

La quinta parte del módulo objeto es el diccionario de reubicación. Como se muestra en la figura 7-15, es preciso sumar una constante de reubicación a las instrucciones que contienen direcciones de memoria. Puesto que el enlazador no tiene forma de saber por inspección cuáles de las palabras de datos de la parte 4 contienen instrucciones de máquina y cuáles contienen constantes, esta tabla proporciona información acerca de cuáles direcciones deben reubicarse. La información podría estar en forma de una tabla de bits, con un bit por cada dirección potencialmente reubicable, o una lista explícita de direcciones que deben reubicarse.

La sexta parte es una indicación de fin de módulo, a veces una suma de verificación para detectar errores ocurridos durante la lectura del módulo, y la dirección en la que debe iniciarse la ejecución.

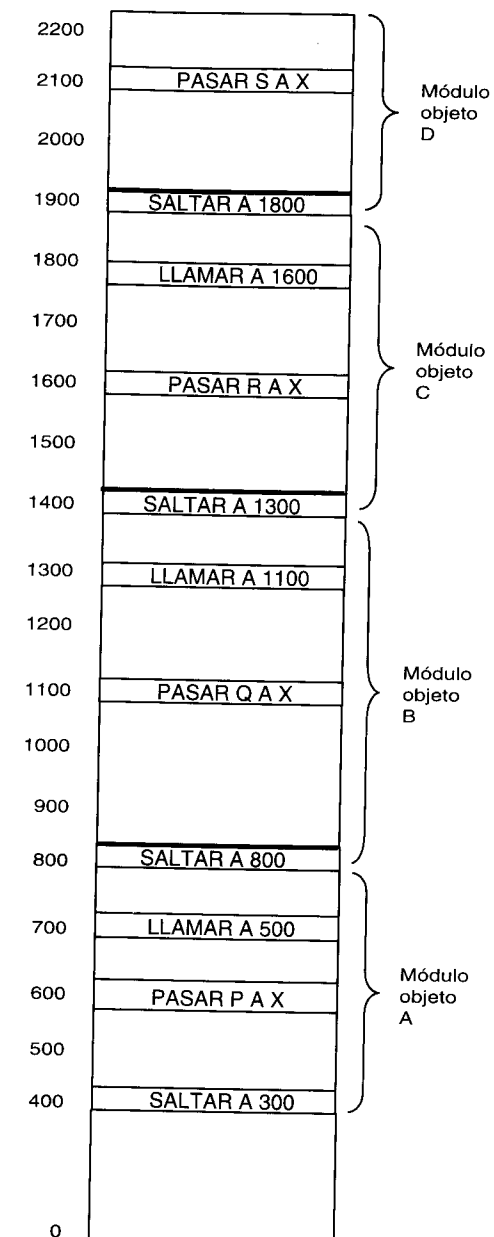
Casi todos los enlazadores requieren dos pasadas. Durante la primera pasada el enlazador lee todos los módulos objeto y construye una tabla de nombres y longitudes de módulos, y una tabla de símbolos global que consiste en todos los puntos de ingreso y referencias externas. Durante la segunda pasada se leen los módulos objeto, se reubican, y se enlazan uno por uno.

### 7.4.3 Tiempo de ligado y reubicación dinámica

En un sistema de multiprogramación un programa puede leerse de la memoria principal, ejecutarse durante un rato, escribirse en disco, y luego almacenarse nuevamente en la memoria principal para ejecutarse otra vez. En un sistema grande con muchos programas es difícil asegurar que un programa dado se colocará en las mismas localidades en cada ocasión.

La figura 7-17 muestra lo que sucedería si el programa ya reubicado de la figura 7-15(b) se reubicara en la dirección 400 en lugar de la dirección 100 donde el enlazador lo colocó originalmente. Todas las direcciones de memoria son incorrectas; además, la información de reubicación hace mucho que se desechó. Incluso si todavía estuviera disponible dicha información, el costo de reubicar todas las direcciones cada vez que el programa se trae a la memoria sería prohibitivo.

El problema de desplazar programas que ya se enlazaron y reubicaron está íntimamente relacionado con el momento en que se lleva a cabo el ligado final de los nombres simbólicos con direcciones de memoria física absolutas. Cuando se escribe un programa, contiene nombres simbólicos para las direcciones de memoria, por ejemplo, **BR L**. El momento en que se



**Figura 7-17.** El programa binario reubicado de la figura 7-15(b) desplazado hacia arriba 300 direcciones. Ahora muchas instrucciones hacen referencia a una dirección de memoria incorrecta.

determina la dirección real en la memoria principal que corresponde a *L* se denomina **tiempo de ligado**. Hay por lo menos seis posibilidades para el tiempo de ligado:

1. Cuando se escribe el programa.
2. Cuando se traduce el programa.
3. Cuando se enlaza el programa pero antes de cargarlo.
4. Cuando se carga el programa.
5. Cuando se carga un registro base que se usa para direccionamiento.
6. Cuando se ejecuta la instrucción que contiene la dirección.

Si una instrucción que contiene una dirección de memoria se desplaza después del ligado, será incorrecta (suponiendo que el objeto al que hace referencia también cambió de lugar). Si el traductor produce un binario ejecutable como salida, el ligado habrá ocurrido en el momento de la traducción, y el programa deberá ejecutarse en la dirección en la que el traductor esperaba que se ejecutara. El método de enlazado que se describió en la sección anterior liga los nombres simbólicos a direcciones absolutas durante el enlazado, y es por ello que el desplazamiento de los programas después de enlazados causa errores, como se muestra en la figura 7-17.

Aquí intervienen dos cuestiones relacionadas. La primera es la cuestión de cuándo los nombres simbólicos se ligan a direcciones virtuales. La segunda es cuándo las direcciones virtuales se ligan a direcciones físicas. Sólo si ya se llevaron a cabo ambas operaciones es completo el ligado. Cuando el enlazador fusiona los espacios de direcciones discretos de los módulos objeto en un solo espacio de direcciones lineal, está creando de hecho un espacio de direcciones virtual. La reubicación y el enlazado sirven para ligar nombres simbólicos a direcciones virtuales específicas. Esta observación es cierta sea que se use o no memoria virtual.

Supongamos por el momento que el espacio de direcciones de la figura 7-15(b) se pagina. Es evidente que las direcciones virtuales que corresponden a los nombres simbólicos *A*, *B*, *C* y *D* ya están determinados, aunque sus direcciones físicas en la memoria principal dependerán del contenido de la tabla de páginas en el momento en que se usen. Un programa binario ejecutable es en realidad un ligado de nombres simbólicos a direcciones virtuales.

Cualquier mecanismo que permita modificar fácilmente la correspondencia entre direcciones virtuales y direcciones físicas de la memoria principal facilitará el desplazamiento de los programas dentro de la memoria principal, aun después de haberse ligado a un espacio de direcciones virtual. Uno de esos mecanismos es la paginación. Una vez que un programa se ha cambiado de lugar en la memoria principal sólo hay que modificar su tabla de páginas, no el programa mismo.

Un segundo mecanismo es el uso de un registro de reubicación de tiempo de ejecución. El CDC 6600 y sus sucesores tenían un registro semejante. En las máquinas que usan esta técnica de reubicación, el registro siempre apunta a la dirección física en la memoria donde comienza el programa actual. El hardware suma a todas las direcciones de memoria el contenido del registro de ubicación antes de enviarlas a la memoria. Todo el proceso de reubicación es transparente para los programas de usuario; ni siquiera saben que está ocurriendo. Cuando un programa cambia de lugar, el sistema operativo debe actualizar el registro de reubicación.

Este mecanismo es menos general que la paginación porque es preciso mover todo el programa como una unidad (a menos que haya registros de reubicación distintos para el código y los datos, como en el Intel 8088, en cuyo caso tendrá que moverse como dos unidades).

En las máquinas que pueden hacer referencia a la memoria relativa al contador de programa se puede usar un tercer mecanismo. Cada vez que un programa cambia de lugar en la memoria principal sólo es preciso actualizar el contador de programa. Un programa en el que todas las referencias a la memoria son relativas al contador de programa o bien absolutas (por ejemplo, registros de dispositivos de E/S en direcciones absolutas) se dice que es **independiente de la posición**. Un procedimiento independiente de la posición se coloca en cualquier lugar del espacio de direcciones virtual sin necesidad de reubicación.

#### 7.4.4 Enlazado dinámico

La estrategia de enlazado que vimos en la sección 7.4.1 tiene la propiedad de que todos los procedimientos que un programa podría invocar se enlazan antes de que el programa pueda iniciar su ejecución. En una computadora con memoria virtual, si se lleva a cabo todo el enlazado antes de iniciar la ejecución no se aprovecharán todas las ventajas de la memoria virtual. Muchos programas tienen procedimientos que sólo se invocan en condiciones extraordinarias. Por ejemplo, los compiladores tienen procedimientos para compilar enunciados que raras veces se usan, además de procedimientos para manejar condiciones de error que casi nunca se presentan.

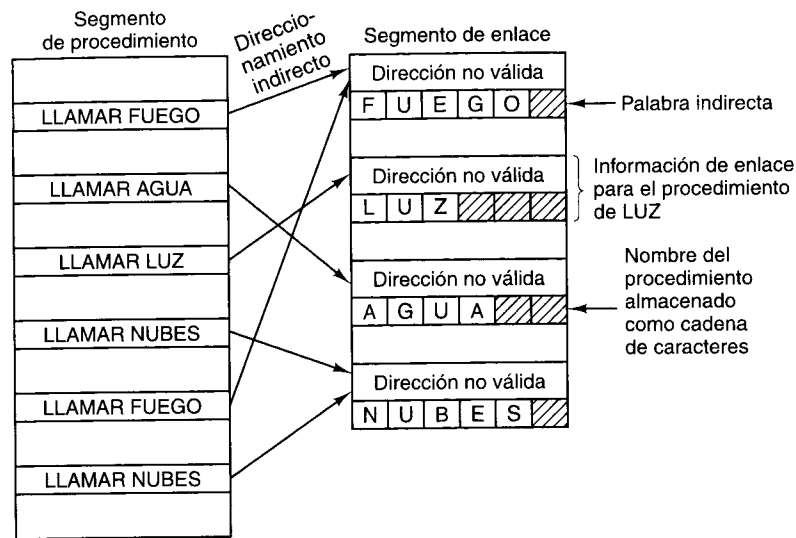
Una forma más flexible de enlazar procedimientos compilados por separado es enlazar cada procedimiento en el momento en que se invoca por primera vez. Este proceso se llama **enlazado dinámico** y se usó por primera vez en MULTICS, cuya implementación todavía no ha sido superada en algunos aspectos. En las siguientes secciones examinaremos el enlazado dinámico en varios sistemas.

#### Enlazado dinámico en MULTICS

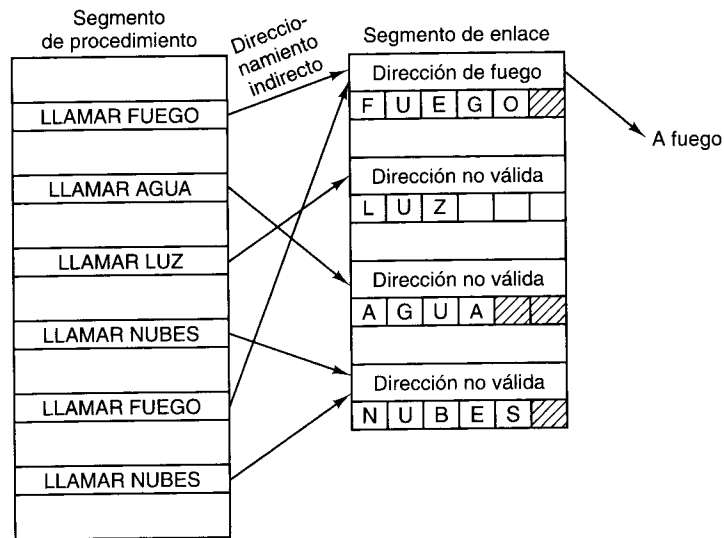
En la modalidad MULTICS del enlazado dinámico, cada programa está asociado a un segmento, llamado **segmento de enlace**, que contiene un bloque de información para cada procedimiento que podría invocarse. Este bloque de información inicia con una palabra reservada para la dirección virtual del procedimiento y va seguida del nombre del procedimiento, que se almacena como cadena de caracteres.

Cuando se está usando enlazado dinámico, las llamadas a procedimientos en el lenguaje fuente se traducen en instrucciones que direccionan indirectamente la primera palabra del bloque de enlace correspondiente, como se muestra en la figura 7-18(a). El compilador coloca en esta palabra una dirección no válida o bien un patrón de bits especial que hace que ocurra una trampa.

Cuando se invoca un procedimiento que está en un segmento distinto, el intento por direccionar la palabra no válida indirectamente hace que ocurra una trampa al enlazador dinámico. Entonces, el enlazador encuentra la cadena de caracteres en la palabra que sigue a la dirección no válida y busca un procedimiento compilado con este nombre en el directorio



(a)



(b)

**Figura 7-18.** Enlazado dinámico. (a) Antes de llamar a FUEGO. (b) Después de llamar a FUEGO y enlazarlo.

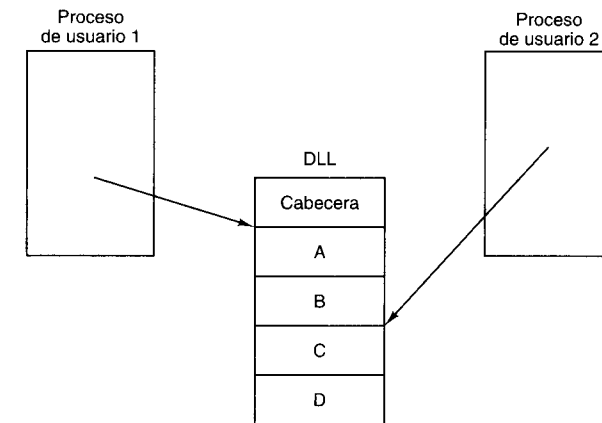
de archivos del usuario. A continuación, se asigna una dirección virtual a ese procedimiento, casi siempre en su propio segmento privado, y esta dirección virtual sobrescribe la dirección no válida del segmento de enlace, como se indica en la figura 7-18(b). Después se vuelve a ejecutar la instrucción que causó el fallo de enlace, y el programa puede continuar a partir del punto en el que estaba antes de la trampa.

Todas las referencias subsecuentes a ese procedimiento se ejecutarán sin causar un fallo de enlace, pues la palabra indirecta ahora contiene una dirección virtual válida. Por consiguiente, el enlazador dinámico sólo se invoca la primera vez que se llama a un procedimiento, y después ya no.

### Enlazado dinámico en Windows

Todas las versiones del sistema operativo Windows, incluido NT, manejan enlazado dinámico y se apoyan mucho en él. El enlazado dinámico emplea un formato de archivo especial llamado **DLL** (**biblioteca de enlace dinámico**, *Dynamic Link Library*). Las DLL pueden contener procedimientos, datos o ambas cosas, y es común usarlos para que dos o más procesos puedan compartir procedimientos o datos de biblioteca. Muchas DLL tienen la extensión *.dll*, pero también se usan otras extensiones como *.drv* (para bibliotecas de *drivers*) y *.fon* (para bibliotecas de tipos de letra).

La forma más común de DLL es una biblioteca que consiste en una colección de procedimientos que se pueden cargar en la memoria y a los que varios procesos pueden acceder al mismo tiempo. La figura 7-19 ilustra dos programas que comparten un archivo DLL que contiene cuatro procedimientos, A, B, C y D. El programa 1 usa el procedimiento A; el programa 2 usa el procedimiento C, aunque bien podrían haber usado el mismo procedimiento.



**Figura 7-19.** Uso de un archivo DLL por dos procesos.

El enlazador construye una DLL a partir de una colección de archivos de entrada. De hecho, la construcción de un archivo DLL se parece mucho a la construcción de un programa binario ejecutable, excepto que se proporciona una bandera especial al enlazador para indicarle que cree una DLL. Las DLL suelen construirse a partir de colecciones de procedimientos de biblioteca que es probable que muchos procedimientos necesiten. Los procedimientos de interfaz con la biblioteca de llamadas del sistema Windows y las grandes bibliotecas de gráficos son ejemplos comunes de DLL. La ventaja de usar las DLL es el ahorro de espacio en la memoria y en el disco. Si se ligara estáticamente una biblioteca común a cada programa

que la usa, aparecería en muchos binarios ejecutables en el disco y en la memoria, y se desperdiciaría espacio. Con las DLL, cada biblioteca aparece una sola vez en el disco y una vez en la memoria.

Además de ahorrar espacio, este enfoque facilita la actualización de los procedimientos de biblioteca aun después de que los programas que los usan han sido compilados y enlazados. En el caso de los paquetes de software comerciales, de los cuales los usuarios casi nunca tienen el código fuente, usar las DLL implica que el proveedor de software puede corregir los errores en las bibliotecas con sólo distribuir archivos DLL nuevos por Internet, sin tener que modificar los binarios del programa principal.

La principal diferencia entre una DLL y un binario ejecutable es que una DLL no puede iniciarse ni ejecutarse sola (porque no tiene programa principal). La información de su cabecera también es diferente. Además, la DLL incluye varios procedimientos extra que no están relacionados con los procedimientos de la biblioteca. Por ejemplo, hay un procedimiento que se invoca automáticamente cada vez que un proceso nuevo se liga a la DLL y otro que se llama automáticamente cada vez que un proceso se desliga de ella. Estos procedimientos pueden apartar y liberar memoria o administrar otros recursos que la DLL necesita.

Un programa se puede ligar a una DLL de dos maneras. En la primera, llamada **enlazado implícito**, el programa del usuario se enlaza estáticamente con un archivo especial llamado **biblioteca de importación** generado por un programa utilitario que extrae cierta información de la DLL. La biblioteca de importación es el mecanismo que permite al programa de usuario acceder a la DLL. Un programa de usuario se puede enlazar a varias bibliotecas de importación. Cuando un programa que usa enlazado implícito se carga en la memoria para ejecutarse, Windows lo examina para ver cuáles DLL usa y verifica si todas ellas ya están en la memoria. Las que no están en la memoria se cargan de inmediato (pero no necesariamente en su totalidad, ya que se pagan). Luego se hacen ciertos cambios a las estructuras de datos de las bibliotecas de importación para poder localizar los procedimientos invocados, algo análogo a los cambios que se muestran en la figura 7-18. Los procedimientos también tienen que mapearse en el espacio de direcciones virtual del programa. En este punto, el programa de usuario está listo para ejecutarse y puede llamar a los procedimientos de las DLL como si se hubieran enlazado estáticamente a él.

La alternativa al enlazado implícito es (¡oh sorpresa!) el **enlazado explícito**. Este enfoque no requiere bibliotecas de importación y no hace que las DLL se carguen al mismo tiempo que el programa de usuario. En vez de ello, el programa de usuario emite una llamada explícita en el momento de la ejecución para ligarse a una DLL, y luego efectúa llamadas adicionales para obtener las direcciones de los procedimientos que necesita. Una vez encontrados, pueden invocarse. Por último, el programa efectúa una llamada final para desligarse de la DLL. Cuando el último proceso se desliga de una DLL, ésta puede removerse de la memoria.

Es importante darse cuenta de que un procedimiento en una DLL no tiene identidad propia (como un hilo o proceso); se ejecuta en el hilo del invocador y usa la pila del invocador para sus variables locales. El procedimiento puede tener datos estáticos específicos para el proceso (así como datos compartidos), y por lo demás se comporta igual que un procedimiento enlazado estáticamente. La única diferencia básica está en la forma en que se efectúa el ligado.

## Enlazado dinámico en UNIX

El sistema UNIX tiene un mecanismo idéntico en lo esencial a las DLL de Windows; se llama **biblioteca compartida**. Al igual que un archivo DLL, una biblioteca compartida es un archivo permanente que contiene varios procedimientos o módulos de datos que están presentes en la memoria durante la ejecución y se pueden ligar a varios procesos al mismo tiempo. La biblioteca estándar de C y gran parte del código para trabajo con redes son bibliotecas compartidas.

UNIX sólo maneja enlazado implícito, por lo que una biblioteca compartida consta de dos partes: una **biblioteca anfitriona**, que se enlaza estáticamente con el archivo ejecutable, y una **biblioteca objetivo**, que se llama en el momento de la ejecución. Aunque los detalles difieren, los conceptos son prácticamente idénticos a los de las DLL.

## 7.5 RESUMEN

Aunque casi todos los programas pueden y deben escribirse en un lenguaje de alto nivel, hay situaciones ocasionales en las que se necesita lenguaje ensamblador, al menos en parte. Los programas para computadoras portátiles escasas de recursos, como las tarjetas inteligentes, procesadores incorporados a aparatos domésticos y asistentes digitales portátiles inalámbricos son posibles candidatos. Un programa en lenguaje ensamblador es una representación simbólica de un programa en lenguaje de máquina subyacente, y se traduce al lenguaje de máquina con un programa llamado ensamblador.

Cuando una ejecución extremadamente rápida es crucial para el éxito de alguna aplicación, una estrategia mejor que escribir todo en lenguaje ensamblador es escribir primero todo el programa en un lenguaje de alto nivel, luego determinar dónde pasa más tiempo el programa, y finalmente reescribir en lenguaje ensamblador sólo aquellas localidades del programa que se usen mucho. En la práctica, una fracción pequeña del código representa una fracción importante del tiempo de ejecución.

Muchos ensambladores tienen un recurso de macros que permite al programador asignar nombres simbólicos a secuencias de código de uso común para después incluirlas. Por lo regular estas macros pueden usar parámetros sin complicación. Las macros se implementan con una especie de algoritmo de procesamiento de cadenas literales.

Casi todos los ensambladores son de dos pasadas. La primera pasada se dedica a construir una tabla de símbolos para etiquetas, literales e identificadores declarados explícitamente. Los símbolos se pueden guardar sin ordenarse y luego buscarse linealmente, ordenarse primero y luego buscarse empleando búsqueda binaria, o procesarse con *hashing* (*codificación por dispersión*). Si no es necesario eliminar símbolos durante la primera pasada, el mejor método suele ser el *hashing*. La segunda pasada realiza la generación de código. Algunas seudoinstrucciones se ejecutan durante la primera pasada, y otras, durante la segunda.

Programas ensamblados de forma independiente se pueden enlazar para formar un programa binario ejecutable. El enlazador se encarga de este trabajo. Sus tareas primordiales son la reubicación y el ligado de nombres. El enlazado dinámico es una técnica en la que ciertos procedimientos no se enlazan sino hasta que se llaman realmente. Las DLL de Windows y las bibliotecas compartidas de UNIX usan enlazado dinámico.

## PROBLEMAS

- En cierto programa, el 1% del código ocupa el 50% del tiempo de ejecución. Compare las siguientes tres estrategias respecto al tiempo de programación y el tiempo de ejecución. Suponga que se requerirían 100 meses-programador para escribirlo en C, y que el código de ensamblador es 10 veces más difícil de escribir y cuatro veces más eficiente.
  - Todo el programa en C.
  - Todo el programa en lenguaje ensamblador.
  - Primero todo en C, y luego se reescribe el 1% crítico en ensamblador.
- ¿Las consideraciones que son válidas para los ensambladores de dos pasadas también son válidas para los compiladores?
  - Suponga que los compiladores producen módulos objeto, no código de ensamblador.
  - Suponga que los compiladores producen lenguaje ensamblador simbólico.
- Sugiera cómo los programadores en lenguaje ensamblador podrían definir sinónimos para códigos de operación. ¿Cómo podría implementarse esto?
- Todos los ensambladores para las CPU Intel tienen la dirección de destino como primer operando y la dirección de origen como segundo operando. ¿Qué problemas tendrían que resolverse para hacerlo al revés?
- ¿El siguiente programa puede ensamblarse en dos pasadas? EQU es una pseudoinstrucción que iguala el rótulo a la expresión que está en el campo de operandos.

```

A EQU B
B EQU C
C EQU D
D EQU 4

```

- La Compañía de Software Baratija está planeando producir un ensamblador para una computadora que maneja palabras de 48 bits. A fin de reducir los costos, el gerente del proyecto, el doctor Amarrete, decidió limitar la longitud de los símbolos permitidos de modo que cada símbolo se pueda almacenar en una sola palabra. Amarrete declaró que los símbolos sólo pueden consistir en letras, con excepción de la letra Q, que está prohibida (con objeto de hacer patente ante los clientes su preocupación por la eficiencia). ¿Qué longitud máxima puede tener un símbolo? Describa su esquema de codificación.
- ¿Qué diferencia hay entre una instrucción y una pseudoinstrucción?
- ¿Qué diferencia hay entre el contador de localidades de instrucciones y el contador de programa, si es que la hay? Después de todo, los dos siguen la pista a la siguiente instrucción de un programa.
- Muestre la tabla de símbolos después de encontrarse los siguientes enunciados para el Pentium II. El primer enunciado se asigna a la dirección 1000.

|           |            |           |
|-----------|------------|-----------|
| EVEREST:  | POP BX     | (1 BYTE)  |
| K2:       | PUSH BP    | (1 BYTE)  |
| WHITNEY:  | MOV BP,SP  | (2 BYTES) |
| MCKINLEY: | PUSH X     | (3 BYTES) |
| FUJI:     | PUSH SI    | (1 BYTE)  |
| KIBO:     | SUB SI,300 | (3 BYTES) |

- ¿Puede imaginar circunstancias en las que un lenguaje ensamblador permita que una etiqueta sea igual a un código de operación (por ejemplo, MOV como rótulo)? Comente.
- Muestre los pasos necesarios para encontrar Berkeley usando búsqueda binaria con la siguiente lista: Ann Arbor, Berkeley, Cambridge, Eugene, Madison, New Haven, Palo Alto, Pasadena, Santa Cruz, Stony Brook, Westwood y Yellow Springs. Al calcular el elemento de la mitad de una lista con número par de elementos, use el elemento que está inmediatamente después del índice de en medio.
- ¿Es posible usar búsqueda binaria con una tabla cuyo tamaño es primo?
- Calcule el código *hash* para cada uno de los siguientes símbolos sumando todas las letras ( $A = 1$ ,  $B = 2$ , etc.) y obteniendo el residuo de dividir el resultado entre el tamaño de la tabla *hash*. La tabla tiene 19 entradas, numeradas del 0 al 18.  
  
els, jan, jelle, maaie  
  
¿Cada símbolo genera un valor *hash* único? Si no es así, ¿cómo puede resolverse el problema de la colisión?
- El método de codificación por *hashing* descrito en el texto vincula todas las entradas que tienen el mismo código *hash* en una lista enlazada. Un método alternativo es tener una sola tabla con  $n$  entradas, cada una de las cuales tiene espacio para una clave y su valor (o apuntadores a ellos). Si el algoritmo de *hashing* genera una entrada que ya está llena, se usa un segundo algoritmo de *hashing* para intentarlo otra vez. Si esa entrada también está llena, se usa otro algoritmo, y así hasta encontrar una entrada vacía. Si la fracción de las entradas llenas es  $R$ , ¿cuántos intentos se necesitarán, en promedio, para introducir un símbolo nuevo?
- A medida que avanza la tecnología, podría llegar el día en que sea posible colocar miles de CPU idénticas en un chip, cada una con unas cuantas palabras de memoria local. Si todas las CPU pueden leer y escribir tres registros compartidos, ¿cómo podría implementarse una memoria asociativa?
- El Pentium II tiene una arquitectura segmentada, con varios segmentos independientes. Un ensamblador para esta máquina bien podría tener una pseudoinstrucción SEG N que ordenaría al ensamblador colocar el código y los datos subsecuentes en el segmento N. ¿Un esquema así afectaría al ILC?
- Los programas a menudo se enlazan con varias DLL. ¿No sería más eficiente poner todos los procedimientos en una DLL grande y luego enlazarse con ella?
- ¿Una DLL se puede mapear con los espacios de direcciones virtuales de dos procesos en diferentes direcciones virtuales? Si así se hiciera, ¿qué problemas podrían surgir? ¿Se pueden resolver? Si no, ¿qué puede hacerse para eliminarlos?
- Una forma de efectuar enlazado (estático) es la siguiente. Antes de explorar la biblioteca, el enlazador construye una lista de los procedimientos requeridos, es decir, de nombres definidos como EXTERN en los módulos que se están enlazando. Luego el enlazador recorre la biblioteca linealmente, extrayendo todos los procedimientos que están en la lista de nombres requeridos. ¿Funciona este plan? Si no, ¿por qué no y cómo puede remediarse?
- ¿Un registro puede usarse como parámetro real en una llamada a macro? ¿Y una constante? ¿Por qué sí o por qué no?

21. Usted debe implementar un ensamblador con macros. Por razones estéticas, su jefe ha decidido que las definiciones de las macros no tienen que preceder a sus llamadas. ¿Qué implicaciones tiene esta decisión para la implementación?
22. Piense en una forma de poner un ensamblador con macros en un ciclo infinito.
23. Un enlazador lee cinco módulos, cuyas longitudes son 200, 800, 600, 500 y 700 palabras, respectivamente. Si los módulos se cargan en ese orden, ¿cuáles son las constantes de reubicación?
24. Escriba un paquete de tabla de símbolos que conste de dos rutinas: *ingresar(simbolo, valor)* y *buscar(simbolo, valor)*. La primera ingresa símbolos nuevos en la tabla y la segunda los busca. Use alguna forma de codificación *hash*.
25. Escriba un ensamblador sencillo para la computadora Mic-1 del capítulo 4. Además de procesar las instrucciones de máquina, incluya un mecanismo para asignar constantes a símbolos durante el ensamblado, y una forma de ensamblar una constante en una palabra de máquina.
26. Añada un recurso de macros sencillo al ensamblador del problema anterior.

# 8

## ARQUITECTURAS DE COMPUTADORAS PARALELAS

Aunque las computadoras cada vez son más rápidas, lo que se les exige está creciendo por lo menos al mismo ritmo. Los astrónomos quieren simular toda la historia del Universo, desde el *big bang* hasta que acabe la función. A los ingenieros farmacéuticos les encantaría poder diseñar medicamentos a la medida de enfermedades específicas en sus computadoras en lugar de tener que sacrificar legiones de ratas. Los diseñadores de aviones podrían crear productos con mejor rendimiento de combustible si las computadoras pudieran efectuar todo el trabajo, sin necesidad de construir prototipos para túneles de viento. En pocas palabras, por más potencia de cómputo que tengamos al alcance, para muchos usuarios, sobre todo en las ciencias, la ingeniería y la industria, nunca es suficiente.

Aunque la frecuencia de reloj es cada vez más alta, la velocidad de los circuitos no se puede incrementar indefinidamente. La velocidad de la luz ya representa un problema importante para los diseñadores de computadoras del extremo superior, y las posibilidades de lograr que los electrones y fotones se muevan a mayor velocidad no son muy halagüeñas. Los problemas de disipación de calor están convirtiendo a las supercomputadoras en acondicionadores de aire de vanguardia. Por último, aunque los transistores son cada vez más pequeños, llegará el momento en que cada transistor tendrá tan pocos átomos que los efectos de la mecánica cuántica (por ejemplo, el principio de incertidumbre de Heisenberg) podrían convertirse en un problema importante.

Por ello, para poder atacar problemas cada vez más grandes, los arquitectos de computadoras están recurriendo a las computadoras paralelas. Aunque tal vez nunca sea posible construir una computadora con una sola CPU y un tiempo de ciclo de 0.001 ns, bien

podría ser factible construir una con 1000 CPU, cada una de ellas con un tiempo de ciclo de 1 ns. Si bien este último diseño usa CPU más lentas que el primero, en teoría su capacidad de cómputo total es la misma. Es en esto en lo que están fincadas las esperanzas.

Podemos introducir paralelismo en diversos niveles. En el nivel de instrucciones, por ejemplo, el uso de filas de procesamiento y los diseños superescalares pueden elevar el desempeño en un factor de 10 respecto a los diseños puramente secuenciales. Sin embargo, para lograr un mejoramiento por un factor de cien o mil o un millón es necesario repetir CPU enteras, o al menos porciones sustanciales de ellas, y lograr que todas colaboren de forma eficiente. Aquí es donde está el reto.

En este capítulo examinaremos los principios de diseño de computadoras paralelas y estudiaremos varios ejemplos. Todas estas máquinas consisten en elementos de procesamiento y elementos de memoria. Los diseños difieren en la cantidad de cada elemento que está presente, en su constitución y en su interconexión. Algunos diseños emplean un número relativamente bajo de componentes de gran potencia, mientras que otras usan cantidades enormes de componentes más débiles. También son comunes los diseños intermedios. Se ha trabajado mucho en el área de las arquitecturas paralelas; en este capítulo apenas podemos rascar la superficie. El lector puede encontrar información adicional en (Loshin, 1994; Pfister, 1998; Sima *et al.*, 1997; y Wilkinson, 1994).

## 8.1 ASPECTOS DEL DISEÑO DE COMPUTADORAS PARALELAS

Al analizar un nuevo sistema de cómputo paralelo, las tres preguntas fundamentales que debemos hacer son:

1. ¿Cuántos elementos de procesamiento hay, de qué tamaño y de qué tipo?
2. ¿Cuántos módulos de memoria hay, de qué tamaño y de qué tipo?
3. ¿Cómo se interconectan los elementos de procesamiento y de memoria?

Examinemos brevemente cada uno de estos puntos. Los elementos de procesamiento pueden variar desde las ALU mínimas hasta las CPU completas, con tamaños que van desde una pequeña fracción de un chip hasta un metro cúbico de circuitos electrónicos por elemento. Como cabría esperar, cuando el elemento de procesamiento es una fracción de un chip, es posible equipar una computadora con una gran cantidad de tales elementos, quizá hasta un millón de ellos. Si el elemento de procesamiento es una computadora completa, con su propia memoria y equipo de E/S, el número de elementos es naturalmente más reducido, aunque se han instalado sistemas con casi 10,000 CPU. Cada vez se están construyendo más computadoras paralelas a partir de piezas comerciales, sobre todo CPU. Las capacidades y limitaciones de estas piezas a menudo influyen considerablemente en el diseño.

Los sistemas de memoria a menudo se dividen en módulos que operan de forma independiente unos de otros, en paralelo para que muchas CPU puedan acceder a ellos al mismo tiempo. Dichos módulos pueden ser pequeños (kilobytes) o grandes (megabytes) y estar integrados a la CPU o situados en una tarjeta de circuitos distinta. Puesto que las memorias dinámicas (DRAM) grandes suelen ser mucho más lentas que las CPU, es común utilizar

complejos esquemas de caché para agilizar el acceso a la memoria. Con frecuencia se usan dos, tres y hasta cuatro niveles de caché.

Aunque existe cierta variación entre los diseños de CPU y de memoria, el área en la que más difieren los sistemas paralelos es en la forma en que se combinan las piezas. Los esquemas de interconexión se pueden dividir a grandes rasgos en dos categorías: estáticos y dinámicos. Los esquemas estáticos simplemente conectan todos los componentes en una configuración fija, como estrella, anillo o cuadrícula. En los esquemas de interconexión dinámica todas las piezas se conectan a una red de conmutación que puede encaminar dinámicamente mensajes entre los componentes. Los dos esquemas tienen puntos fuertes y débiles, como en breve veremos.

Ver las computadoras paralelas como una colección de chips conectados entre sí de una forma u otra es básicamente una perspectiva ascendente del mundo. En un enfoque descendente la pregunta que se haría sería: “¿Qué es lo que se va a ejecutar en paralelo?” Aquí también tenemos toda una gama de posibilidades. Algunas computadoras paralelas se diseñan de modo que puedan ejecutar varios trabajos independientes al mismo tiempo. Dichos trabajos nada tienen que ver unos con otros y no se comunican. Un ejemplo típico es una computadora con entre 8 y 64 CPU creada para un gran sistema UNIX de tiempo compartido que atiende a miles de usuarios remotos. Los sistemas de procesamiento de transacciones de los bancos (por ejemplo, cajeros automáticos), líneas aéreas (por ejemplo, sistemas de reservaciones) y grandes servidores de Web también pertenecen a esta categoría, lo mismo que las simulaciones independientes que se ejecutan de forma concurrente empleando diferentes conjuntos de parámetros.

En una zona diferente de este espectro se encuentran las computadoras paralelas que se usan para ejecutar un solo trabajo que consiste en muchos procesos paralelos. Por ejemplo, consideremos un programa de ajedrez que analiza un tablero dado generando una lista de movimientos válidos y luego genera procesos paralelos para analizar (recursivamente) cada nuevo tablero en paralelo. Lo que se busca con el paralelismo aquí no es atender a más usuarios, sino resolver con mayor rapidez un solo problema.

Pasando a otra zona del espectro, nos encontramos con máquinas en las que el paralelismo proviene del uso intensivo de filas de procesamiento o de muchas ALU que operan con el mismo flujo de instrucciones al mismo tiempo. Las supercomputadoras numéricas con hardware especial para procesar vectores pertenecen a esta categoría. Aquí no sólo se está resolviendo un solo problema principal, sino que todas las partes de la computadora están trabajando en muy estrecha colaboración en casi el mismo aspecto del problema (por ejemplo, diferentes elementos de los mismos dos vectores se están sumando en paralelo).

Aunque es difícil definirlo con exactitud, estos tres ejemplos difieren en lo que algunos llaman **tamaño de grano**. En los sistemas de tiempo compartido con múltiples CPU, la unidad de paralelismo es grande: todo un programa de usuario. La ejecución de programas grandes en paralelo con poca o ninguna comunicación entre los programas es **paralelismo de grano grueso**. El extremo opuesto, que ilustramos con el procesamiento de vectores, se llama **paralelismo de grano fino**.

El tamaño de grano se refiere a los algoritmos y al software, pero tiene un análogo directo en el hardware. Los sistemas con un número reducido de CPU grandes e independientes que tienen conexiones de baja velocidad entre sí se dice que están **débilmente acoplados**. Lo



opuesto son los sistemas **fuertemente acoplados**, en los que los componentes generalmente son más pequeños, más juntos e interactúan unos con otros con frecuencia a través de redes de comunicación con gran ancho de banda. En casi todos los casos, los problemas que tienen paralelismo de grano grueso se resuelven mejor en sistemas débilmente acoplados; así mismo, los problemas con paralelismo de grano fino se resuelven mejor en sistemas fuertemente acoplados. No obstante, es tanta la variedad de algoritmos, software y hardware, que en el mejor de los casos ésta es sólo una guía general. La amplia gama de tamaños de grano de los problemas y de posibilidades para el acoplamiento de los sistemas ha dado pie a la amplia variedad de arquitecturas que estudiaremos en este capítulo.

En las secciones que siguen examinaremos algunos de los aspectos del diseño de computadoras paralelas, comenzando con los modelos de comunicación y las redes de interconexión. Luego analizaremos cuestiones de desempeño y de software. Por último, concluiremos con una taxonomía de las arquitecturas de computadoras paralelas que determinará la organización del resto del capítulo.

### 8.1.1 Modelos de comunicación

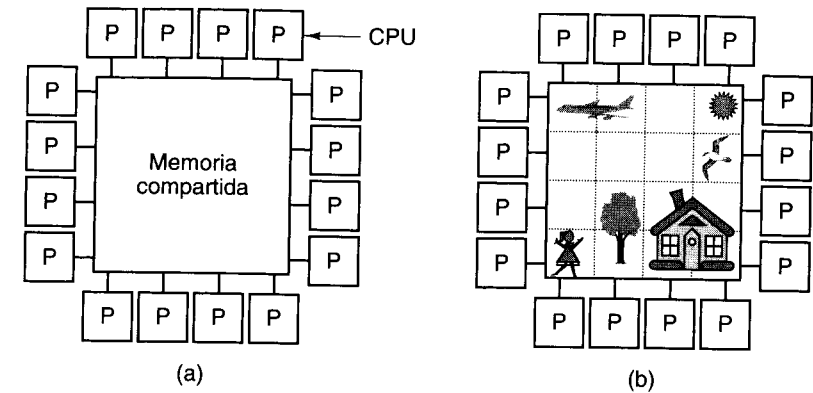
En cualquier sistema de cómputo paralelo, las CPU que trabajan en diferentes partes del mismo problema se deben comunicar entre sí para intercambiar información. La forma precisa en la que lo deben hacer es tema de muchos debates en la comunidad arquitectónica. Se han propuesto e implementado dos diseños distintos, multiprocesadores y multicomputadoras. En esta sección examinaremos ambos diseños.

#### Multiprocesadores

En el primer diseño, todas las CPU comparten una misma memoria física, como se ilustra en la figura 8-1(a). Un sistema basado en memoria compartida como éste se llama **multiprocesador** o a veces simplemente **sistema con memoria compartida**.

El modelo del multiprocesador se extiende al software. Todos los procesos que están ejecutándose en un multiprocesador pueden compartir un solo espacio de direcciones virtual mapeado en la memoria común. Cualquier proceso puede leer o escribir una palabra de memoria con sólo ejecutar una instrucción **LOAD** o **STORE**. No se necesita nada más. Dos procesos pueden comunicarse con sólo hacer que uno de ellos escriba datos en la memoria y que el otro los lea después.

La capacidad de dos (o más) procesos para comunicarse con sólo leer y escribir en la memoria es la razón por la que los multiprocesadores son tan populares. Se trata de un modelo fácil de entender para los programadores que se puede aplicar a una amplia gama de problemas. Consideremos, por ejemplo, un programa que examina una imagen de mapa de bits y enumera todos los objetos que contiene. Una copia de la imagen se guarda en la memoria, como se muestra en la figura 8-1(b). Cada una de las 16 CPU ejecuta un solo proceso, al cual se ha asignado una de las 16 secciones para que la analice. Sin embargo, cada proceso tiene acceso a toda la imagen, lo cual es indispensable porque algunos objetos ocupan varias secciones. Si un proceso descubre que uno de sus objetos rebasa la frontera de su sección, sim-



**Figura 8-1.** (a) Multiprocesador con 16 CPU que comparten una misma memoria. (b) Imagen dividida en 16 secciones, cada una de las cuales se analiza con una CPU distinta.

plemente sigue al objeto a la siguiente sección leyendo las palabras de esa sección. En este ejemplo, varios procesos descubrirán el mismo objeto, por lo que se requiere cierta coordinación al final para determinar cuántas, casas, árboles y aviones hay.

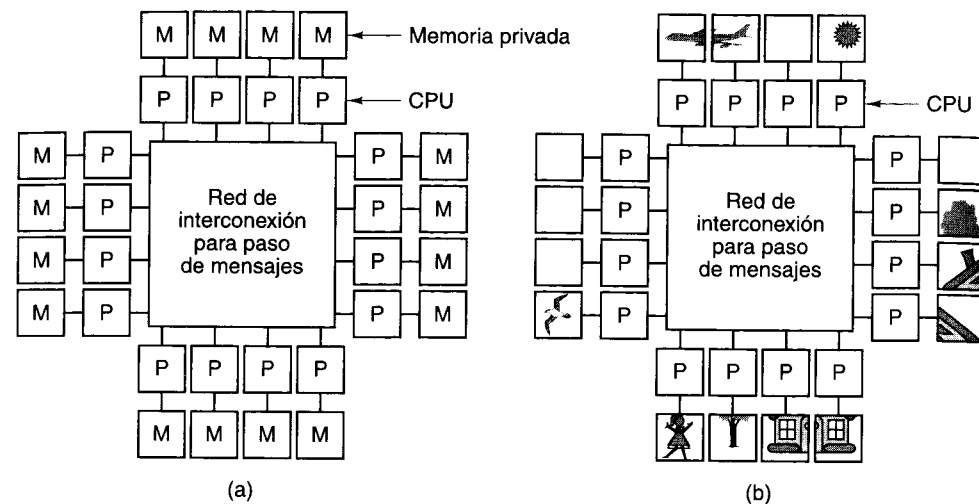
Muchos fabricantes de computadoras venden multiprocesadores. Como ejemplos podemos citar el Sun Enterprise 10000, el Sequent NUMA-Q, el SGI Origin 2000 y el HP/Convex Exemplar.

#### Multicomputadoras

El segundo diseño de arquitectura paralela es aquel en el que cada CPU tiene su propia memoria privada, accesible únicamente a ella y a ninguna otra CPU. Un diseño así se llama **multicomputadora** o **sistema de memoria distribuida**, y se ilustra en la figura 8-2(a). Las multicomputadoras suelen estar débilmente acopladas (aunque no siempre). El aspecto clave de una multicomputadora que la distingue de un multiprocesador es que cada CPU de una multicomputadora tiene su propia memoria local privada a la que puede acceder con sólo ejecutar instrucciones **LOAD** y **STORE**, pero a la que ninguna otra CPU puede acceder usando esas instrucciones. Así pues, los multiprocesadores tienen un solo espacio de direcciones físico compartido por todas las CPU, mientras que las multicomputadoras tienen un espacio de direcciones físico por CPU.

Puesto que las CPU de una multicomputadora no pueden comunicarse con sólo leer y escribir en la memoria común, necesitan un mecanismo de comunicación distinto. Lo que hacen es pasarse mensajes entre sí utilizando la red de interconexión. Como ejemplos de multicomputadoras podemos mencionar al IBM SP/2, la Intel/Sandia Option Red y la Wisconsin COW.

La ausencia de memoria compartida en hardware en una multicomputadora tiene importantes implicaciones para la estructura del software. Tener un solo espacio de direcciones virtual en el que todos los procesos pueden leer y escribir con sólo ejecutar instrucciones **LOAD**



**Figura 8-2.** (a) Multicomputadora con 16 CPU, cada una de las cuales tiene su propia memoria privada. (b) La imagen de mapa de bits de la figura 8-1 dividida entre las 16 memorias.

y **STORE** es imposible en una multicomputadora. Por ejemplo, si la CPU 0 (la de la esquina superior izquierda) de la figura 8-1(b) descubre que parte de su objeto se extiende hacia la sección asignada a la CPU 1, podrá seguir leyendo la memoria para acceder a la cola del avión. Por otra parte, si la CPU 0 de la figura 8-2(b) efectúa el mismo descubrimiento, no puede leer simplemente la memoria de la CPU 1; tiene que hacer algo muy distinto para obtener los datos que necesita.

En particular, la CPU 0 tiene que descubrir (de alguna manera) cuál CPU tiene los datos que necesita y enviar a esa CPU un mensaje en el que solicita una copia de los datos. Por lo regular, lo que haría entonces la CPU 0 sería bloquearse (es decir, esperar) hasta recibir una respuesta a su solicitud. Cuando el mensaje llega a la CPU 1, el software de esa CPU debe analizarlo y devolver los datos solicitados. Cuando el mensaje de respuesta llega a la CPU 0, el software se desbloquea y puede continuar su ejecución.

En una multicomputadora, la comunicación entre procesos a menudo utiliza primitivas en software como **send** (enviar) y **receive** (recibir). Esto hace que el software tenga una estructura diferente, mucho más complicada, que en un multiprocesador, y también implica que la división correcta de los datos y su colocación en los lugares óptimos es una cuestión importante en una multicomputadora. Esto es menos importante en un multiprocesador porque la colocación no afecta la corrección ni la programabilidad, aunque sí podría afectar el desempeño. En pocas palabras, programar una multicomputadora es mucho más difícil que programar un multiprocesador.

En estas condiciones, ¿para qué construir multicomputadoras, si los multiprocesadores son más fáciles de programar? La respuesta es sencilla: las multicomputadoras grandes son mucho más sencillas y económicas de construir que los multiprocesadores con el mismo

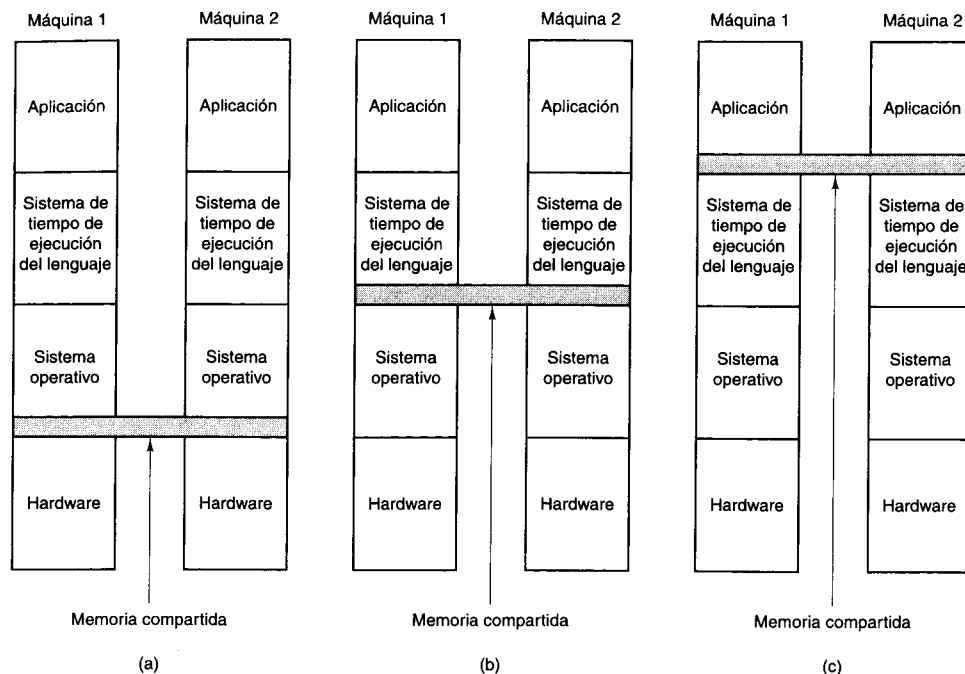
número de CPU. Implementar una memoria compartida por unos cuantos cientos de CPU es una empresa de gran magnitud, pero es relativamente fácil construir una multicomputadora con 10,000 CPU o más.

Así pues, tenemos un dilema: los multiprocesadores son difíciles de construir pero fáciles de programar, mientras que las multicomputadoras son fáciles de construir pero difíciles de programar. Esta observación ha dado pie a muchos intentos por construir sistemas híbridos que sean relativamente fáciles de construir y relativamente fáciles de programar. Tales trabajos han conducido a la revelación de que hay varias formas de implementar una memoria compartida, cada una con sus ventajas y desventajas. De hecho, gran parte de las investigaciones actuales sobre arquitecturas paralelas tiene que ver con la convergencia de las arquitecturas de multiprocesador y multicomputadora en formas híbridas que combinan las ventajas de cada una. El Santo Grial aquí es encontrar diseños **escalables**, es decir, que sigan funcionando bien a medida que se añaden más y más CPU.

Una estrategia para construir sistemas híbridos se basa en el hecho de que los sistemas de cómputo modernos no son monolíticos sino que se construyen en una serie de capas: el tema de este libro. Este concepto abre la posibilidad de implementar la memoria compartida en cualquiera de varios niveles, como se muestra en la figura 8-3. En la figura 8-3(a) vemos cómo el hardware implementa la memoria compartida como verdadero multiprocesador. En este diseño, hay una sola copia de sistema operativo con un solo conjunto de tablas, en particular la tabla de asignación de memoria. Cuando un proceso necesita más memoria, transfiere el control al sistema operativo a través de una trampa, éste busca en su tabla una página libre y hace corresponder la página con el espacio de direcciones del proceso. En lo que al sistema operativo concierne, hay una sola memoria y él lleva la contabilidad de qué proceso posee qué página en software. Hay muchas formas de implementar la memoria compartida en hardware, como veremos más adelante.

Una segunda posibilidad es usar hardware de multicomputadora y hacer que el sistema operativo simule la memoria compartida proporcionando un solo espacio de direcciones virtual paginado compartido para todo el sistema. En esta estrategia, llamada **memoria compartida distribuida** (DSM, *Distributed Shared Memory*) (Li, 1988; Li y Hudak, 1986, 1989), cada página se ubica en una de las memorias de la figura 8-2(a). Cada máquina tiene su propia memoria virtual y sus propias tablas de páginas. Cuando una CPU efectúa una **LOAD** o **STORE** con una página que no tiene, ocurre una trampa al sistema operativo. Éste localiza entonces la página y pide a la CPU que actualmente la tiene que anule su mapeo y envíe la página por la red de interconexión. Cuando la página llega, se mapea y la instrucción que causó el fallo se reinicia. En realidad, lo que el sistema operativo está haciendo es atender los fallos de página con la memoria remota en lugar de con el disco. Para el usuario, parece como si la máquina tuviera memoria compartida. Examinaremos DSM más adelante en este capítulo.

Una tercera posibilidad es hacer que un sistema de tiempo de ejecución en el nivel de usuario implemente una forma (tal vez específica para el lenguaje) de memoria compartida. Con esta estrategia, el lenguaje de programación proporciona algún tipo de abstracción de memoria compartida, que luego el compilador y el sistema de tiempo de ejecución implementan. Por ejemplo, el modelo Linda se basa en la abstracción de un espacio de tuplas (registros de datos que contienen una colección de campos) compartido. Los procesos de cualquier



**Figura 8-3.** Diversas capas en las que puede implementarse la memoria compartida. (a) El hardware. (b) El sistema operativo. (c) El sistema de tiempo de ejecución del lenguaje.

máquina pueden traer una tupla del espacio de tuplas compartido o enviar una tupla a ese espacio. Dado que el acceso al espacio de tuplas se controla totalmente en software (por el sistema de tiempo de ejecución Linda), no se requiere hardware especial ni apoyo del sistema operativo.

Otro ejemplo de memoria compartida específica para un lenguaje implementada por el sistema de tiempo de ejecución es el modelo Orca de objetos de datos compartidos. En Orca, los procesos comparten objetos genéricos, no sólo tuplas, y pueden ejecutar métodos específicos para cada objeto con ellos. Cuando un método modifica el estado interno de un objeto, corresponde al sistema de tiempo de ejecución asegurarse de que todas las copias del objeto en todas las máquinas se actualicen simultáneamente. Una vez más, puesto que los objetos son un concepto estrictamente de software, el sistema de tiempo de ejecución puede efectuar totalmente la implementación, sin ayuda del hardware ni del sistema operativo. Examinaremos tanto a Linda como a Orca más adelante en este capítulo.

### 8.1.2 Redes de interconexión

En la figura 8-2 vimos que las multicomputadoras deben su cohesión a las redes de interconexión. Ha llegado el momento de examinar con mayor detenimiento esas redes. Resulta interesante y sorprendente lo parecido que son los multiprocesadores y las multicomputadoras

en este sentido, porque los multiprocesadores a menudo tienen varios módulos de memoria que también deben conectarse entre sí y con las CPU. Por ello, el material de esta sección a menudo se aplica a ambos tipos de sistemas.

La razón fundamental por la que las redes de interconexión de los multiprocesadores y las multicomputadoras son similares es que en lo más fundamental ambas usan transferencia de mensajes. Incluso en una máquina con una sola CPU, si el procesador necesita leer o escribir una palabra, lo que normalmente hace es acertar ciertas líneas del bus y esperar una respuesta. Esta acción es fundamentalmente similar a la transferencia de mensajes: el iniciador envía una solicitud y espera una respuesta. En los multiprocesadores grandes, la comunicación entre las CPU y la memoria remota casi siempre consiste en que la CPU envía un mensaje explícito, llamado **paquete**, a la memoria solicitándole ciertos datos, y la memoria devuelve un paquete de respuesta.

Las redes de interconexión pueden tener hasta cinco componentes:

1. CPU.
2. Módulos de memoria.
3. Interfaces.
4. Enlaces.
5. Conmutadores.

Ya examinamos las CPU y las memorias con cierto detalle en otros capítulos y no lo haremos más aquí. Básicamente, se trata de los puntos finales de toda la comunicación. Las interfaces son los dispositivos que sacan los mensajes de las CPU y las memorias y los introducen en ellas. En muchos diseños, una interfaz es un chip o tarjeta que está conectada al bus local de cada CPU y puede hablar con ella y con la memoria local, si la hay. Es común que la interfaz incluya un procesador programable así como algo de RAM privada. Por lo regular, la interfaz puede leer y escribir en diversas memorias, a fin de trasladar bloques de datos de un lugar a otro.

Los enlaces son los canales físicos por los cuales se desplazan los bits; pueden ser eléctricos o de fibra óptica y seriales (anchura de un bit) o paralelos (anchura de más de un bit). Cada enlace tiene un ancho de banda máximo, que es el número de bits por segundo que puede transferir. Los enlaces pueden ser simplex (unidireccionales), semidúplex (en un sentido a la vez) o dúplex (en ambos sentidos a la vez).

Los conmutadores son dispositivos con varios puertos de entrada y varios puertos de salida. Cuando un paquete llega a un conmutador por un puerto de entrada, ciertos bits del paquete se usan para seleccionar el puerto de salida al que se enviará el paquete. Un paquete podría tener sólo 2 o 4 bytes, pero también podría ser mucho más largo (digamos, 8 KB).

Existe cierta analogía entre una red de interconexión y las calles de una ciudad. Las calles son como enlaces; cada una tiene una direccionalidad (de uno o dos sentidos), una tasa de datos máxima (límite de velocidad) y una anchura (número de carriles). Las intersecciones

son como conmutadores. En cada intersección, un paquete que llega (peatón o vehículo) puede escoger cuál puerto de salida (calle) usará, dependiendo de cuál es su destino final.

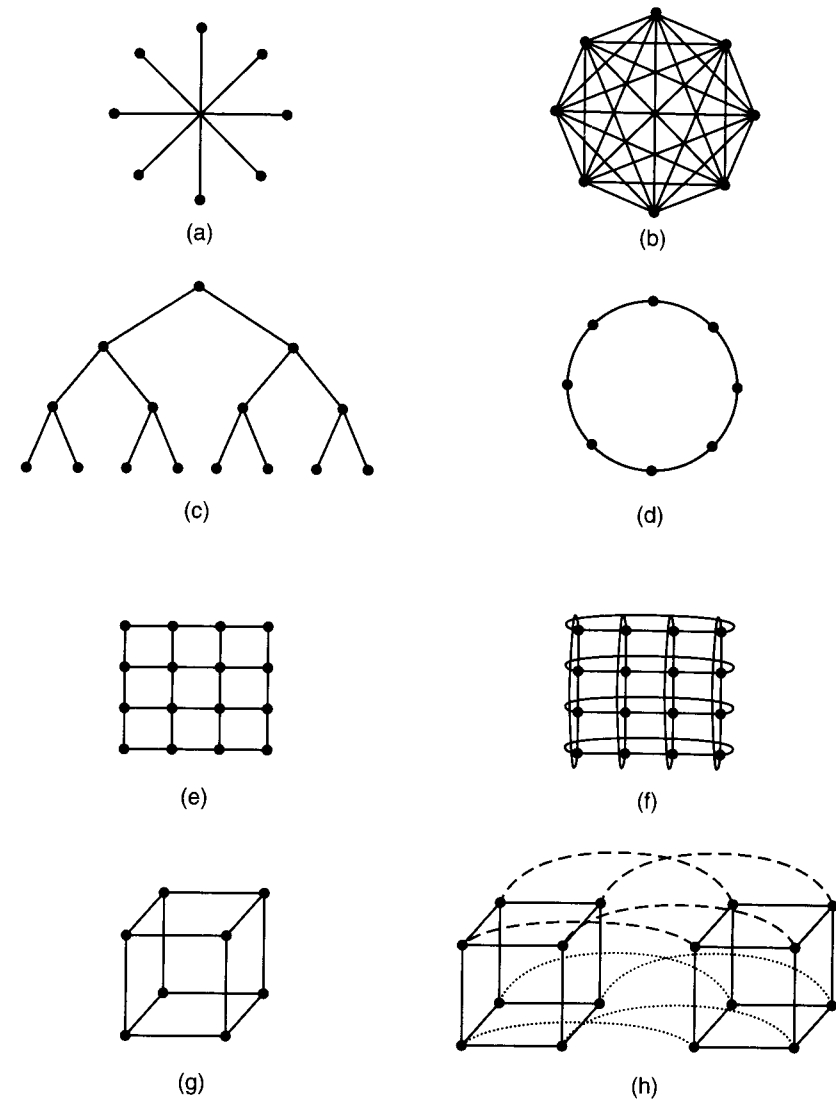
Al diseñar o analizar una red de interconexión, varias áreas destacan por su importancia. En primer lugar está la cuestión de topología, es decir, la forma en que están dispuestos los componentes. En segundo lugar está la forma en que el conmutador funciona y cómo maneja la lucha por recursos. En tercer lugar está el algoritmo de enrutamiento que se usa para que los mensajes lleguen a su destino de forma eficiente. A continuación examinaremos brevemente cada uno de estos temas.

### Topología

La topología de una red de interconexión describe la forma como están dispuestos los enlaces y los conmutadores; por ejemplo, en forma de anillo o de cuadrícula. Los diseños topológicos pueden modelarse como grafos, en los que los enlaces son aristas y los conmutadores son nodos, como se muestra en la figura 8-4. Cada nodo de una red de interconexión (o de su grafo) tiene cierto número de enlaces conectados a él. Los matemáticos llaman al número de enlaces **grado** del nodo; los ingenieros lo llaman **abanico de salida**. En general, cuanto mayor es el abanico de salida, más opciones de enrutamiento hay y mayor es la tolerancia de fallos, es decir, la capacidad para seguir funcionando aunque falle un enlace, desviando el tráfico por otros caminos. Si cada nodo tiene  $k$  aristas y el cableado es el debido, es posible diseñar la red de modo que permanezca plenamente conectada aunque fallen  $k - 1$  enlaces.

Otra propiedad de una red de interconexión (o su grafo) es su **diámetro**. Si medimos la distancia entre dos nodos por el número de aristas que es preciso recorrer para llegar de uno al otro, el título de una gráfica es la distancia entre los dos nodos que están más separados (es decir, entre los que la distancia es máxima). El diámetro de una red de interconexión se relaciona con el retraso de peor caso al enviar paquetes de una CPU a otra o de una CPU a la memoria porque cada brinco por un enlace toma una cantidad de tiempo finita. Cuanto más pequeño sea el diámetro, mejor será el desempeño de peor caso. Otra cosa importante es la distancia media entre dos nodos, la cual se relaciona con el tiempo de tránsito medio de los paquetes.

Otra propiedad importante de una red de interconexión es su capacidad de transmisión, es decir, cuántos datos puede transferir por segundo. Una medida útil de esta capacidad es el **ancho de banda bisectriz**. Para calcular esta cantidad, primero tenemos que dividir (conceptualmente) la red en dos partes iguales (en términos del número de nodos) pero desconexas eliminando un conjunto de aristas de su grafo. Luego calculamos el ancho de banda total de las aristas que se eliminaron. Puede haber muchas formas distintas de dividir la red en dos partes iguales. El ancho de banda bisectriz es el mínimo de todas las divisiones posibles. La importancia de este número radica en que si el ancho de banda bisectriz es, digamos, de 800 bits/s, entonces si hay mucha comunicación entre las dos mitades el rendimiento total podría estar limitado a sólo 800 bits/s, en el peor de los casos. Muchos diseñadores creen que el ancho de banda bisectriz es la métrica más importante de una red de interconexión. Muchas redes de interconexión se diseñan con el objetivo de maximizar el ancho de banda bisectriz.



**Figura 8-4.** Diversas topologías. Los puntos gruesos representan conmutadores. No se muestran las CPU ni las memorias. (a) Una estrella. (b) Una interconexión total. (c) Un árbol. (d) Un anillo. (e) Una cuadrícula. (f) Un toroide doble. (g) Un cubo. (h) Un hipercubo 4D.

Las redes de interconexión se pueden caracterizar por su **dimensionalidad**. Para nuestros fines, la dimensionalidad está determinada por el número de opciones que hay para llegar del origen al destino. Si nunca hay opciones (es decir, sólo hay un camino de cada origen a cada destino), la red es cero-dimensional. Si hay una dimensión en la que se puede tomar una decisión, por ejemplo, ir al oriente o ir al poniente, la red es unidimensional. Si hay dos ejes,

de modo que un paquete puede ir al oriente o al poniente, o bien ir al norte o al sur, la red es bidimensional, etcétera.

En la figura 8-4 se muestran varias topologías. Sólo se han incluido los enlaces (líneas) y los conmutadores (puntos). Las memorias y las CPU (que no se muestran) por lo regular estarían conectadas a los conmutadores mediante interfaces. En la figura 8-4(a) tenemos una configuración de **estrella** cero-dimensional, en la que las CPU y memorias estarían conectadas a los nodos exteriores, y el nodo central sólo realizaría conmutación. Aunque su diseño es sencillo, si el sistema es grande es muy probable que el conmutador central sea un importante cuello de botella. Además, desde el punto de vista de la tolerancia de fallos, este diseño es deficiente porque un solo fallo en el conmutador central destruye por completo el sistema.

En la figura 8-4(b) tenemos otro diseño cero-dimensional que está en el otro extremo del espectro, una **interconexión total**. Aquí cada nodo tiene una conexión directa con todos los demás nodos. Este diseño maximiza el ancho de banda bisectriz, minimiza el diámetro y es en extremo tolerante de fallos (puede perder cualquier enlace y aun así seguir plenamente conectado). Lo malo es que el número de enlaces requeridos para  $k$  nodos es  $k(k-1)/2$ , cifra que pronto se vuelve inmanejable cuando  $k$  es grande.

Una tercera topología cero-dimensional es el **árbol**, que se ilustra en la figura 8-4(c). Un problema con este diseño es que el ancho de banda bisectriz es igual a la capacidad de un enlace. Puesto que normalmente hay mucho tráfico cerca de la parte alta del árbol, los nodos de arriba se convertirán en cuellos de botella. Una forma de remediar este problema es incrementar el ancho de banda bisectriz dando a los enlaces superiores mayor ancho de banda. Por ejemplo, los enlaces del nivel más bajo podrían tener una capacidad de  $b$ , el siguiente nivel podría tener una capacidad de  $2b$ , y los enlaces del nivel superior podrían tener  $4b$  cada uno. Un diseño así se denomina **árbol grueso** y se ha usado en multicomputadoras comerciales como la (ahora difunta) CM-5 de Thinking Machines.

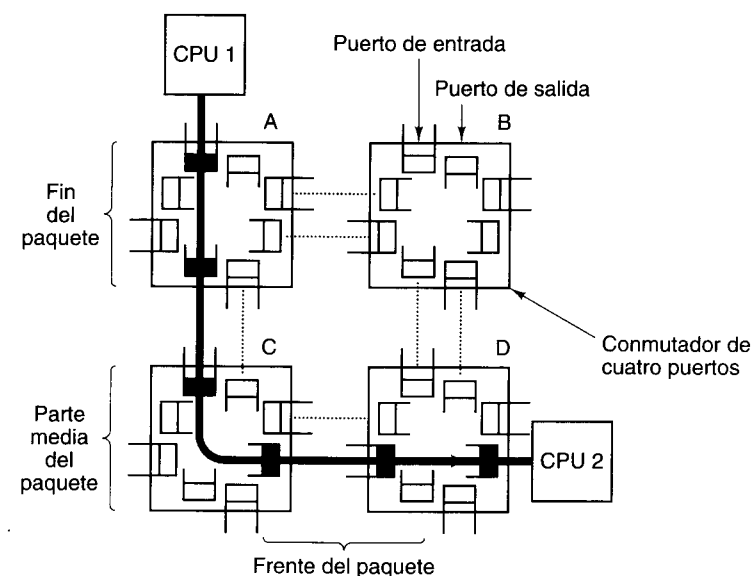
El **anillo** de la figura 8-4(d) es una topología unidimensional según nuestra definición porque cada paquete que se envía tiene la opción de ir a la derecha o ir a la izquierda. La **cuadrícula** o **mall** de la figura 8-4(e) es un diseño bidimensional que se ha usado en muchos sistemas comerciales. Es muy regular, fácil de cambiar de escala hasta tamaños muy grandes, y tiene un diámetro que aumenta según la raíz cuadrada del número de nodos. Una variante de la cuadrícula es el **doble toroide** de la figura 8-4(f), que es una cuadrícula cuyas aristas están conectadas. Esta topología no sólo es más tolerante de fallos que la cuadrícula, sino que su diámetro también es menor porque las esquinas opuestas ahora pueden comunicarse con sólo dos pasos.

El **cubo** de la figura 8-4(g) es una topología tridimensional regular. Hemos ilustrado un cubo de  $2 \times 2 \times 2$ , pero en el caso general podría ser un cubo de  $k \times k \times k$ . En la figura 8-4(h) tenemos un cubo tetradimensional construido a partir de dos cubos tridimensionales cuyas aristas correspondientes están conectadas. Podríamos crear un cubo pentadimensional clonando la estructura de la figura 8-4(h) y conectando los nodos correspondientes para formar un bloque de cuatro cubos. Para llegar a seis dimensiones podríamos repetir el bloque de cuatro cubos e interconectar los nodos correspondientes, y así. Un cubo  $n$ -dimensional formado de esta manera se denomina **hipercubo**. Muchas computadoras paralelas usan esta topología porque el diámetro crece linealmente con la dimensionalidad. En otras palabras, el diámetro

es el logaritmo base 2 del número de nodos, de modo que, por ejemplo, un hipercubo 10-dimensional tiene 1024 nodos pero un diámetro de sólo 10, lo que da excelentes propiedades de retraso. Observe que, en contraste, 1024 nodos dispuestos en una cuadrícula de  $32 \times 32$  tiene un diámetro de 62, más de seis veces más grande que el del hipercubo. El precio que se paga por el diámetro más pequeño es que el abanico de salida y por tanto el número de enlaces (y el costo) es mucho mayor para el hipercubo. No obstante, el hipercubo es una opción común en sistemas de alto desempeño.

## Conmutación

Una red de interconexión consiste en conmutadores y alambres que los conectan. En la figura 8-5 vemos una red pequeña con cuatro conmutadores. Cada conmutador de este ejemplo tiene cuatro puertos de entrada y cuatro puertos de salida. Además, cada conmutador tiene algunas CPU y circuitos de interconexión (que no se muestran en su totalidad). La tarea del conmutador es aceptar paquetes que llegan por cualquier puerto de entrada y enviarlos por el puerto de salida correcto.



**Figura 8-5.** Red de interconexión en forma de cuadrícula de cuatro conmutadores. Sólo se muestran dos de las CPU.

Cada puerto de salida está conectado a un puerto de entrada que pertenece a otro conmutador por medio de un enlace serial o paralelo, que se muestra como líneas punteadas en la figura 8-5. Los enlaces seriales transfieren un bit a la vez. Los enlaces paralelos transfieren varios bits a la vez y por tanto tienen también señales para controlar el enlace. Los enlaces paralelos son más rápidos que los enlaces seriales con la misma frecuencia de reloj, pero

tienen el problema del **sesgo** (asegurarse de que todos los bits lleguen al mismo tiempo) y son mucho más costosos.

Hay varias posibles estrategias de conmutación. En una de ellas, llamada **conmutación de circuitos**, antes de enviar un paquete se reserva por adelantado el camino completo desde el origen hasta el destino. Se aseguran todos los puertos y buffers, de modo que cuando la transmisión se inicie esté garantizado que todos los recursos están disponibles y los bits pueden viajar a toda velocidad desde el origen, a través de todos los conmutadores, hasta el destino. La figura 8-5 ilustra la conmutación de circuitos, con un circuito reservado de la CPU 1 a la CPU 2, que se indica con la flecha negra curva. Aquí se han reservado tres puertos de entrada y tres puertos de salida.

La conmutación de circuitos puede compararse con reservar la ruta de un desfile a través de una ciudad, pidiendo a la policía que bloquee todas las calles laterales con barricadas. Se requiere planificación por adelantado, pero una vez que se ha efectuado, el desfile puede avanzar a toda velocidad sin interferencia por parte de otro tráfico. La desventaja es que se requiere planificación anticipada y se prohíbe cualquier otro tráfico, aunque de momento no haya ningún integrante del desfile (paquete) a la vista.

Una segunda estrategia de conmutación es la **conmutación de paquetes con almacenamiento y reenvío**. Aquí no es necesario reservar nada con anticipación. Más bien, el origen envía un paquete completo al primer conmutador, donde se almacena en su totalidad. En la figura 8-6(a), la CPU 1 es el origen, y el paquete entero, destinado a la CPU 2, se coloca primero en un buffer dentro del conmutador A. Una vez que el paquete se ha acumulado cabalmente en el conmutador A, se transfiere al conmutador C, como se muestra en la figura 8-6(b). Ya que el paquete entero ha llegado a C, se transfiere al conmutador D, como se muestra en la figura 8-6(c). Por último, el paquete se envía al destino, la CPU 2. Observe que no se requiere preparación y no se reservan recursos por adelantado.

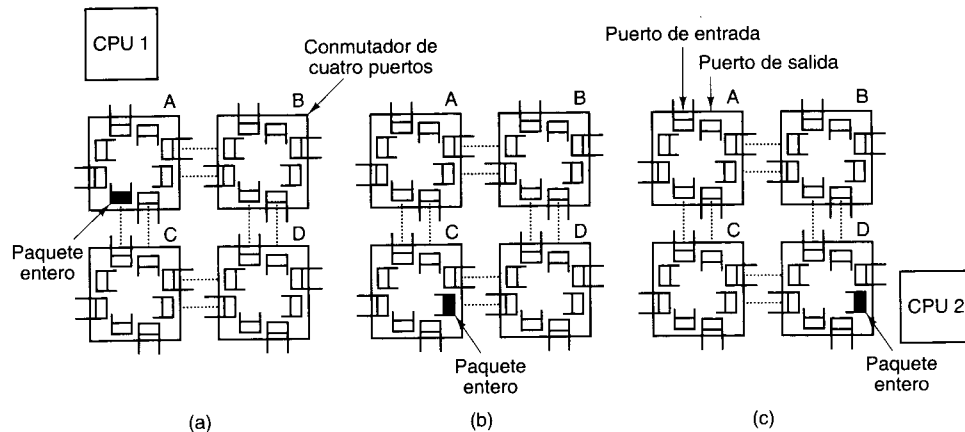


Figura 8-6. Conmutación de paquetes de almacenamiento y reenvío.

Los conmutadores de almacenamiento y reenvío deben colocar los paquetes en buffers porque cuando un origen (una CPU, memoria o conmutador) presenta un paquete, cabe la

posibilidad de que el puerto de salida requerido esté ocupado transmitiendo otro paquete. Si no se usaran buffers, los paquetes que llegaran y necesitaran un puerto ocupado tendrían que desecharse, y la red de interconexión sería muy poco confiable. Se usan tres estrategias de buffers. En la primera, **uso de buffers en la entrada**, cada puerto de entrada tiene asociados uno o más buffers en forma de cola de "primero que entra, primero que sale". Si el paquete que está a la cabeza de la cola no puede transmitirse porque el puerto de salida que necesita está ocupado, simplemente espera.

El problema con este diseño es que, si un paquete está esperando que se desocupe un puerto de salida, el paquete que viene atrás tiene que esperar, aunque esté destinado para un puerto desocupado. Esta situación se llama **bloqueo de cabeza de línea**, y se puede comparar a una sucesión de automóviles en una carretera de dos carriles que tiene que esperar porque el primer automóvil quiere dar vuelta a la izquierda pero no puede hacerlo a causa del tráfico en el otro sentido. Aunque el segundo automóvil y los que le siguen quieran seguir de frente, el automóvil que está a la cabeza los está bloqueando.

El bloqueo de cabeza de línea puede eliminarse con **uso de buffers en la salida**. En este diseño, los buffers están asociados a los puertos de salida, no a los de entrada. A medida que llegan los bits de un paquete, se almacenan en un buffer asociado al puerto de salida correcto. Así, los paquetes destinados al puerto  $m$  no pueden bloquear a los paquetes destinados al puerto  $n$ .

El número de buffers asociado a un puerto tanto de entrada como de salida es limitado. Si es preciso almacenar más paquetes de los que caben en los buffers, habrá que desechar paquetes. Una forma de mejorar esta situación es el uso de **buffers comunes**, en el que una sola reserva de buffers se reparte dinámicamente entre los puertos conforme se necesitan. Sin embargo, este esquema requiere una administración más compleja para llevar la contabilidad de los buffers, y también permite a una conexión con mucho tráfico acaparar todos los buffers y así bloquear otras conexiones. Además, cada conmutador necesita poder contener el paquete más grande, y probablemente varios paquetes de tamaño máximo, lo que tiende a incrementar las necesidades de memoria y reducir el tamaño máximo de paquete.

Aunque la conmutación de paquetes con almacenamiento y reenvío es flexible y eficiente, tiene el problema de aumentar la latencia (retraso) de la red de interconexión. Supongamos que el tiempo requerido para que un paquete avance un tramo en la figura 8-6 es de  $T$  ns. Puesto que el paquete debe copiarse cuatro veces para que llegue de la CPU 1 a la CPU 2 (en A, en C, en D y en la CPU de destino), y ningún copiado puede iniciarse antes de que termine el anterior, la latencia de la red de interconexión es de  $4T$ . Una solución es diseñar una red híbrida, con algunas de las propiedades de la conmutación de circuitos y algunas de la conmutación de paquetes. Por ejemplo, cada paquete podría dividirse lógicamente en unidades más pequeñas. Tan pronto como la primera unidad llega a un conmutador, se le puede transferir al siguiente conmutador, aun antes de que haya llegado la cola del paquete.

Semejante estrategia difiere de la conmutación de circuitos en cuanto a que no se reservan con anticipación recursos de extremo a extremo, y puede haber competencia por los recursos (puertos y buffers). En el **enrutamiento virtual de corte a través**, si la primera unidad de un paquete no puede avanzar, el resto del paquete sigue llegando. En el peor de los casos, este esquema se degrada hasta convertirse en conmutación de paquetes con almacenamiento y reenvío. En una estrategia alternativa, el **enrutamiento por túnel**, si la primera

unidad no puede avanzar, se le pide al origen que deje de transmitir, y así el paquete podría quedar tendido a lo largo de dos o tal vez aún más conmutadores. Una vez que están disponibles los recursos necesarios, el paquete puede continuar.

Vale la pena señalar que ambas estrategias de enrutamiento utilizan algo análogo a las instrucciones de filas de procesamiento de una CPU. En un instante dado, cada conmutador sólo está efectuando una fracción pequeña del trabajo, pero juntos logran un mejor desempeño que el que cada uno podría lograr individualmente.

### Algoritmos de enrutamiento

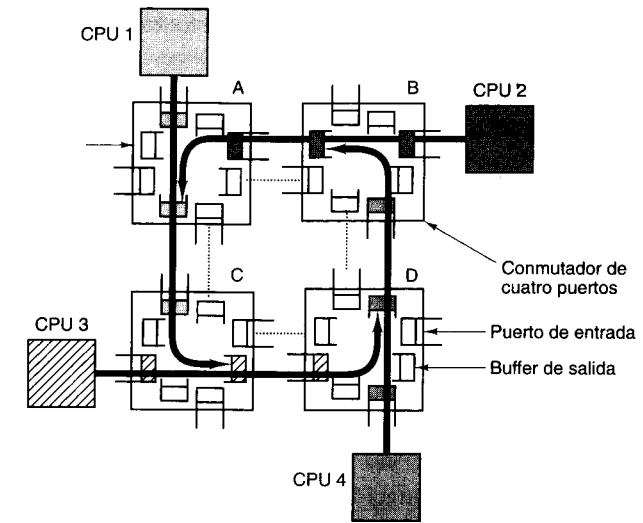
En cualquier red que no sea cero-dimensional hay que tomar decisiones acerca de la ruta que los paquetes han de seguir del origen al destino. En muchos casos existen varias rutas. La regla que determina por cuál sucesión de nodos debe viajar un paquete para ir del origen al destino se denomina **algoritmo de enrutamiento**.

Se requieren buenos algoritmos de enrutamiento porque es común que haya varios caminos disponibles. Un buen algoritmo de enrutamiento puede distribuir la carga entre varios enlaces a fin de aprovechar al máximo el ancho de banda disponible. Además, el algoritmo de enrutamiento debe evitar los bloqueos mutuos dentro de la red de interconexión. Ocurre un **bloqueo mutuo** (o bloqueo mortal) cuando varios paquetes que están en tránsito al mismo tiempo se han apoderado de recursos de tal manera que ninguno de ellos puede avanzar y todos quedan bloqueados indefinidamente.

En la figura 8-7 se da un ejemplo de bloqueo mutuo en una red de interconexión de conmutación de circuitos. (También puede haber bloqueos mutuos en las redes de conmutación de paquetes, pero es más fácil ilustrarlo en las de conmutación de circuitos.) Aquí cada CPU está tratando de enviar un paquete a la CPU que está en la posición diagonalmente opuesta. Cada una ha logrado reservar los puertos de entrada y de salida de su conmutador local, así como un puerto de entrada en el conmutador que sigue, pero no puede obtener el puerto de salida del segundo conmutador, por lo que tiene que esperar hasta que ese puerto se desocupe. Lo malo es que si las cuatro CPU inician este proceso simultáneamente, todas ellas se bloquearán y la red se suspenderá indefinidamente.

Los algoritmos de enrutamiento se pueden clasificar como de enrutamiento de origen o de enrutamiento distribuido. En el **enrutamiento de origen**, el origen determina con anticipación la ruta completa que se seguirá por la red de interconexión, expresada como una lista de números de puerto que se usarán en cada conmutador del camino. Por lo regular, si la ruta pasa por  $k$  conmutadores, los primeros  $k$  bytes de cada paquete contendrán los  $k$  números de puerto de salida requeridos, un byte por puerto. Cuando el paquete llega a un puerto, el primer byte se separa y se usa para determinar el puerto de salida que se usará. El resto del paquete se encamina entonces al puerto correcto. En cada paso del camino, el paquete se acorta en un byte, y expone un nuevo número de puerto que será el siguiente en seleccionarse.

En el enrutamiento distribuido, cada conmutador toma una decisión respecto al puerto al que enviará cada paquete que llegue. Si toma la misma decisión para cada paquete dirigido a un destino dado, se dice que el enrutamiento es **estático**. Si el conmutador toma en cuenta el gráfico actual, se dice que el enrutamiento es **adaptativo**.



**Figura 8-7.** Bloqueo mutuo en una red de interconexión con conmutación de circuitos.

Un algoritmo de enrutamiento muy utilizado en cuadrículas rectangulares con cualquier cantidad de dimensiones y que se sabe no causa bloqueos mutuos es el **enrutamiento dimensional**. En este algoritmo, el paquete primero se desplaza a lo largo del eje  $x$  hasta la coordenada correcta, y luego a lo largo del eje  $y$  hasta la coordenada correcta, y así sucesivamente si hay más dimensiones. Por ejemplo, para ir de  $(3, 7, 5)$  a  $(6, 9, 8)$  el paquete primero iría de  $x = 3$  a  $x = 6$  pasando por  $(4, 7, 5)$ ,  $(5, 7, 5)$  y  $(6, 7, 5)$ . Luego viajaría por el eje  $y$  viajando a  $(6, 8, 5)$  y  $(6, 9, 5)$ . Por último, el paquete viajaría por el eje  $z$  a  $(6, 9, 6)$ ,  $(6, 9, 7)$  y  $(6, 9, 8)$ . Este algoritmo evita los ciclos que causan un bloqueo mutuo.

### 8.1.3 Desempeño

Lo que se busca al construir una computadora paralela es que opere más rápidamente que una máquina uniprocador. Si no logra este sencillo objetivo, no tiene razón de existir. Además, la meta debe alcanzarse con eficacia de costos. Una máquina que es dos veces más rápida que un uniprocador pero cuesta 50 veces más probablemente no tendrá muchas ventas. En esta sección examinaremos algunos de los aspectos de desempeño asociados a las arquitecturas de computadoras paralelas.

### Métricas de hardware

Desde el punto de vista del hardware, las métricas de desempeño que interesan son la velocidad de la CPU y de E/S, y el desempeño de la red de interconexión. Las velocidades de la CPU y de E/S son las mismas que en el uniprocador, de modo que los parámetros clave de



interés en un sistema paralelo son los asociados a la interconexión. Dos son los elementos clave: la latencia y el ancho de banda, que examinaremos a continuación.

La latencia de viaje redondo es el tiempo que una CPU tarda en enviar un paquete y recibir una respuesta. Si el paquete se envía a una memoria, la latencia mide el tiempo que toma leer o escribir una palabra o un bloque de palabras. Si el paquete se envía a otra CPU, la latencia mide el tiempo de comunicación interprocesador para paquetes de ese tamaño. Por lo regular, la latencia que interesa es la correspondiente a paquetes mínimos, que a menudo son una palabra o una línea de caché pequeña.

La latencia comprende varios factores, y es diferente para las interconexiones de conmutación de circuitos, de almacenamiento y reenvío, de corte a través virtual y de enrutamiento por túnel. En el caso de la conmutación de circuitos, la latencia es la suma del tiempo de preparación y el tiempo de transmisión. Para preparar un circuito, es preciso enviar un paquete “de sondeo” que reserve los recursos e informe de los resultados de su gestión. Después, hay que armar el paquete de datos. Una vez que está listo, sus bits pueden fluir a toda velocidad, de modo que si el tiempo de preparación total es  $T_s$ , el tamaño del paquete es de  $p$  bits y el ancho de banda es de  $b$  bits/s, la latencia en un sentido será  $T_s + p/b$ . Si el circuito es dúplex, no habrá tiempo de preparación para la respuesta, de modo que la latencia mínima para enviar un paquete de  $p$  bits y obtener una respuesta de  $p$  bits es de  $T_s + 2p/b$  segundos.

En el caso de la conmutación de paquetes no es necesario enviar un paquete de sondeo al destino con anticipación, pero de todos modos hay cierto tiempo de preparación interno para armar el paquete,  $T_a$ . Aquí el tiempo de transmisión en un sentido es  $T_a + p/b$ , pero éste es sólo el tiempo requerido para que el paquete llegue al primer conmutador. Hay un retardo finito dentro del conmutador, digamos  $T_d$  y luego el proceso se repite con el siguiente conmutador, etc. El retraso  $T_d$  se compone de un tiempo de procesamiento y un retraso de la cola de espera, mientras se desocupa el puerto de salida. Si hay  $n$  conmutadores, la latencia total en un sentido será  $T_a + n(p/b + T_d) + p/b$ , donde el término final se debe al copiado del último conmutador al destino.

Las latencias en un sentido para el corte virtual a través y el enrutamiento por túnel en el mejor de los casos son cercanas a  $T_a + p/b$  porque no se requiere un paquete de sondeo para establecer un circuito, y tampoco hay retraso de almacenamiento y reenvío. Básicamente, la latencia consiste en el tiempo de preparación inicial mientras se arma el paquete, más el tiempo que toma “sacar los bits a la calle”. En todos los casos hay que sumar el retraso de propagación, pero éste suele ser pequeño.

La otra métrica de hardware es el ancho de banda. Muchos programas paralelos, sobre todo en las ciencias naturales, transfieren de aquí para allá una gran cantidad de datos, por lo que el número de bytes/s que el sistema puede transferir es crítico para el desempeño. Existen varias métricas para el ancho de banda. Ya vimos una de ellas, el ancho de banda bisectriz. Otra es el **ancho de banda agregado**, que se calcula por la simple suma de las capacidades de todos los enlaces. Este número proporciona la cantidad máxima de bits que pueden estar en tránsito en un momento dado. Otra métrica importante es el ancho de banda medio en la salida de cada CPU. Si cada CPU puede transmitir 1 MB/s, no sirve de mucho que la interconexión tenga un ancho de banda bisectriz de 100 GB/s. La comunicación estará limitada a la cantidad de datos que cada CPU puede enviar.

En la práctica es muy difícil acercarse siquiera al ancho de banda teórico. Muchas fuentes de gasto extra reducen la capacidad. Por ejemplo, siempre hay cierto gasto extra asociado a cada paquete: para armarlo, construir su cabecera y ponerlo en marcha. El envío de 1024 paquetes de 4 bytes nunca logrará el mismo ancho de banda que el envío de un paquete de 4096 bytes. Lamentablemente, el uso de paquetes pequeños es mejor para lograr latencias bajas, puesto que los paquetes grandes bloquean demasiado tiempo las líneas y los conmutadores. Así, hay un conflicto inherente entre lograr una latencia baja y aprovechar al máximo el ancho de banda. En algunas aplicaciones, una cosa es más importante que la otra, y en otras aplicaciones es al revés. No obstante, cabe señalar que siempre es posible comprar más ancho de banda (instalando más cables o cables más anchos), pero no es posible comprar latencias más bajas. Por ello, generalmente es mejor tratar de hacer que las latencias sean lo más bajas posibles, y preocuparse por el ancho de banda después.

### Métricas de software

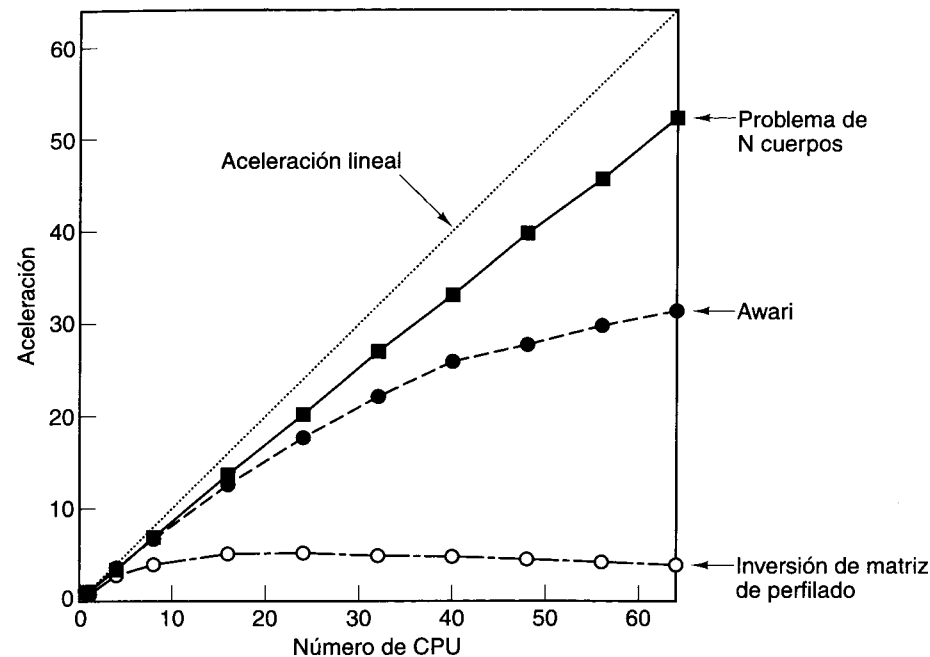
Las métricas de hardware como la latencia y el ancho de banda se ocupan de lo que el hardware hace. Sin embargo, los usuarios tienen un punto de vista distinto. Ellos quieren saber qué tanto va a aumentar la rapidez de ejecución de sus programas en una computadora paralela, en comparación con un uniprocador. Para ellos, la métrica clave es la aceleración: cuántas veces más rápidamente se ejecuta un programa en un sistema de  $n$  procesadores que en uno de un solo procesador. Por lo regular, estos resultados se presentan en gráficas como la de la figura 8-8. En ella vemos la ejecución de varios programas paralelos distintos en una multicomputadora formada por 64 CPU Pentium Pro. Cada curva muestra la aceleración de un programa con  $k$  CPU en función de  $k$ . La línea punteada indica la aceleración perfecta, en la que el uso de  $k$  CPU hace que el programa sea  $k$  veces más rápido, para cualquier  $k$ . Pocos programas logran una aceleración perfecta, pero algunos se acercan. El problema de *N* cuerpos se adapta muy bien a una ejecución paralela; *awari* (un juego de mesa africano) obtiene resultados razonables; pero la inversión de cierta matriz de perfilado no tarda menos de cinco veces menos por más CPU que estén disponibles. Los programas y los resultados se analizan en (Bal *et al.*, 1998).

Una parte de la razón por la que es casi imposible lograr una aceleración perfecta es que casi todos los programas tienen algún componente secuencial, como la fase de inicialización, la lectura de datos o la reunión de los resultados. Tener muchas CPU no ayuda mucho a estas actividades. Suponga que un programa se ejecuta durante  $T$  segundos en un uniprocador, y que una fracción  $f$  de este tiempo corresponde a código secuencial y una fracción  $(1 - f)$  es potencialmente paralelizable, como se muestra en la figura 8-9(a). Si este último código se puede ejecutar en  $n$  CPU sin gasto extra, su tiempo de ejecución podrá reducirse de  $(1 - f)T$  a  $(1 - f)T/n$  en el mejor de los casos, como se muestra en la figura 8-9(b). Esto da un tiempo de ejecución total para las partes secuencial y paralela de  $fT + (1 - f)T/n$ . La aceleración no es más que el tiempo de ejecución del programa original,  $T$ , dividido entre este nuevo tiempo de ejecución:

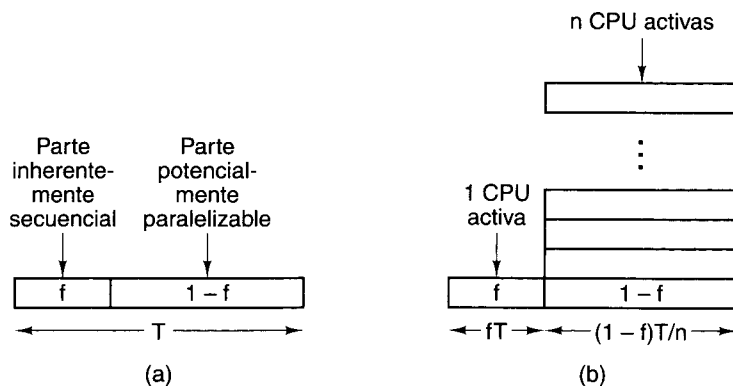
$$\text{Aceleración} = \frac{n}{1 + (n - 1)f}$$

Con  $f = 0$  podemos obtener una aceleración lineal, pero con  $f > 0$  no es posible lograr la aceleración perfecta a causa del componente secuencial. Este resultado se conoce como **ley de Amdahl**.





**Figura 8-8.** Los programas reales logran una aceleración menor que la perfecta indicada por la línea punteada.



**Figura 8-9.** (a) Programa que tiene una parte secuencial y una parte paralelizable. (b) Efecto de ejecutar una parte del programa en paralelo.

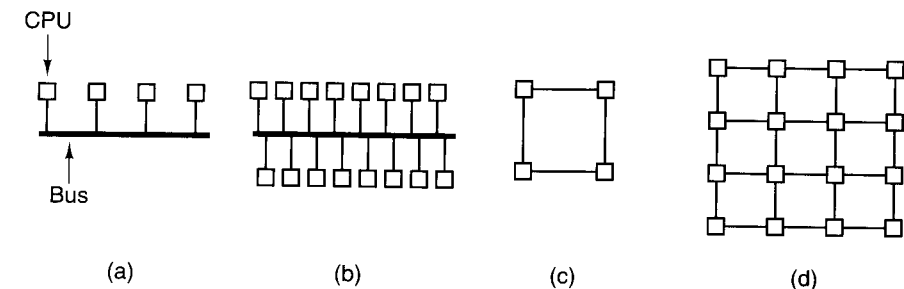
La ley de Amdahl no es la única razón por la que es casi imposible lograr una aceleración perfecta. Las latencias de comunicación distintas de cero, los anchos de banda de comunicación finitos y las ineficiencias algorítmicas también son factores. Además, aunque se contara

con 1000 CPU, no todos los programas pueden escribirse de modo que aprovechen tantas CPU, y el gasto extra de ponerlas todas en marcha puede ser considerable. Por añadidura, es común que el algoritmo mejor conocido no se pueda paralelizar bien, y sea necesario usar un algoritmo subóptimo en el caso paralelo. A pesar de todo esto, para muchas aplicaciones es altamente deseable lograr que el programa funcione con una rapidez  $n$  veces mayor, aunque se necesiten  $2n$  CPU para ello. Después de todo, las CPU no son tan caras, y muchas compañías se conforman con una eficiencia muy por debajo del 100% en otras facetas de su operación.

### Cómo lograr un buen desempeño

La forma más sencilla de mejorar el desempeño es agregar más CPU al sistema. Sin embargo, esta adición debe efectuarse de tal manera que se evite la creación de cuellos de botella. Un sistema en el que es posible añadir más CPU e incrementar de manera acorde la potencia de cómputo es un sistema **escalable**.

Para ver algunas de las implicaciones de la escalabilidad, considere cuatro CPU conectadas por un bus, como se ilustra en la figura 8-10(a). Ahora imagine aumentar la escala del sistema a 16 CPU añadiendo 12 más, como se muestra en la figura 8-10(b). Si el ancho de banda del bus es de  $b$  MB/s, entonces si cuadruplicamos el número de CPU también habremos reducido el ancho de banda disponible por CPU de  $b/4$  MB/s a  $b/16$  MB/s. Un sistema así no es escalable.



**Figura 8-10.** (a) Sistema con cuatro CPU basado en bus. (b) Sistema de 16 CPU basado en bus. (c) Sistema de cuatro CPU basado en cuadrícula. (d) Sistema de 16 CPU basado en cuadrícula.

Hagamos ahora lo mismo con un sistema basado en una cuadrícula, como el que se muestra en la figura 8-10(c) y 8-10(d). Con esta topología, la adición de nuevas CPU también implica la adición de nuevos enlaces, por lo que el aumento de escala del sistema no hace que el ancho de banda por CPU se reduzca, como en el caso de un bus. De hecho, la proporción enlaces/CPU aumenta de 1.0 con 4 CPU (4 CPU, 4 enlaces) a 1.5 con 16 CPU (16 CPU, 24 enlaces), así que la adición de CPU mejora el ancho de banda colectivo por CPU.

Desde luego, el ancho de banda no es la única consideración. La adición de CPU al bus no aumenta el diámetro de la red de interconexión ni la latencia en ausencia de tráfico, pero la adición de CPU a la cuadrícula sí lo hace. En el caso de una cuadrícula de  $n \times n$ , el diámetro es de  $2(n-1)$ , de modo que la latencia de peor caso (y la latencia media) aumenta aproxima-

damente con la raíz cuadrada del número de CPU. Si hay 400 CPU, el diámetro es de 38, mientras que si hay 1600 CPU el diámetro es de 78. Cuadruplicar el número de CPU duplica aproximadamente el diámetro y por tanto la latencia media.

Idealmente, un sistema escalable debe mantener el mismo ancho de banda medio por CPU y una latencia media constante a medida que se añaden más CPU. En la práctica, es posible mantener suficiente ancho de banda por CPU, pero en todos los diseños prácticos la latencia crece al aumentar el tamaño. Lo mejor que puede hacerse es que el crecimiento sea logarítmico, como en un hipercubo.

El aumento de la latencia al aumentar la escala del sistema es un problema porque la latencia a menudo es fatal para el desempeño en las aplicaciones de grano mediano y fino. Si un programa necesita datos que no están en la memoria local, a menudo su obtención implica un retraso sustancial, y cuanto mayor es el sistema, más largo es el retraso, como acabamos de ver. Este problema se presenta tanto en multiprocesadores como en multicomputadoras, porque en ambos casos la memoria física se divide en módulos dispersos.

Una consecuencia de esta observación es que los diseñadores de sistemas a menudo hacen hasta lo imposible por reducir, o al menos ocultar, la latencia, empleando varias técnicas que mencionaremos a continuación. La primera técnica para ocultar la latencia es la repetición de datos. Si es posible mantener copias de un bloque de datos en varios lugares, será posible acelerar los accesos desde esos lugares. Una de las técnicas de repetición es el uso de cachés, en el que una o más copias de algunos bloques de datos se mantienen cerca de donde se están usando, y también cerca de donde “deben estar”. Otra estrategia consiste en mantener varias copias pares —copias que tienen la misma categoría— en contraposición a la relación asimétrica primario/secundario que se usa en las cachés. Cuando se mantienen varias copias, en cualquier forma, las cuestiones fundamentales son dónde se colocan los bloques de datos, cuándo y quién los coloca ahí. Las respuestas varían desde la colocación dinámica por demanda efectuada por hardware hasta la colocación intencional en el momento de la carga siguiendo directrices del compilador. En todos los casos, la coherencia de gestión es importante.

Una segunda técnica para ocultar la latencia es la **prebúsqueda**. Si se puede traer un dato antes de que se necesite, el proceso de búsqueda podrá traslaparse con la ejecución normal para que cuando el dato se necesite, ya esté ahí. La prebúsqueda puede ser automática o estar bajo el control del programa. Cuando una caché carga no sólo la palabra a la que se hizo referencia, sino toda una línea de caché que contiene la palabra, está apostando a que las palabras que siguen también vayan a necesitarse pronto.

La prebúsqueda también puede controlarse explícitamente. Cuando un compilador se da cuenta de que va a necesitar ciertos datos, puede incluir una instrucción explícita para obtenerlos, y colocar esa instrucción en una posición suficientemente adelantada como para que los datos lleguen a tiempo. Esta estrategia requiere que el compilador tenga un conocimiento total de la máquina subyacente y de su temporización, así como control sobre dónde se colocan todos los datos. Tales instrucciones **LOAD** especulativas funcionan óptimamente cuando se sabe con seguridad que los datos se van a necesitar. Causar un fallo de página por un **LOAD** que está en una rama que después no se toma resulta muy costoso.

Una tercera técnica que puede ocultar la latencia es el **multienlace** (*multithreading*). Casi todos los sistemas modernos manejan el concepto de multiprogramación, en el que varios

procesos pueden ejecutarse simultáneamente (o en seudoparalelo por tiempo compartido). Si puede lograrse que la conmutación entre procesos sea lo bastante rápida (por ejemplo, si se proporciona a cada proceso su propio mapa de memoria y registros en hardware), entonces cuando un proceso se bloquea mientras espera la llegada de datos remotos el hardware puede conmutar rápidamente a otro proceso que sí pueda continuar. En el caso limitante, la CPU ejecuta la primera instrucción del enlace 1, la segunda instrucción del enlace 2, etc. De este modo, la CPU puede mantenerse ocupada, aunque los enlaces individuales tengan latencias de memoria largas.

De hecho, algunas máquinas conmutan automáticamente entre los procesos de forma cíclica, después de cada instrucción, a fin de ocultar las latencias largas. Una de las primeras supercomputadoras, la CDC 6600, llevó esta idea a tal extremo que en sus anuncios se aseguraba que tenía 10 unidades de procesamiento periféricas que podían operar en paralelo. En realidad sólo había un procesador periférico que simulaba 10 procesadores ejecutando instrucciones de cada uno por turno circular, primero una instrucción del procesador periférico 1, luego una instrucción del procesador periférico 2, y así.

Una cuarta técnica para ocultar la latencia es el uso de escrituras que no se bloquean. Normalmente, cuando se ejecuta una instrucción **STORE**, la CPU espera hasta que se finaliza la instrucción antes de continuar. Con escrituras que no se bloquean, se inicia la operación de memoria, pero el programa de todos modos continúa. Continuar después de un **LOAD** es más difícil, pero con ejecución en desorden hasta eso es posible.

### 8.1.4 Software

Aunque este capítulo trata primordialmente las arquitecturas de computadora paralelas, es apropiado hablar un poco acerca del software también. Después de todo, sin software paralelo el hardware paralelo no sirve de mucho, y es por ello que los buenos diseñadores de hardware toman en cuenta las necesidades del software al diseñar el hardware. Si desea un tratamiento del software para computadoras paralelas, vea (Wilkinson y Allen, 1999).

Existen cuatro estrategias generales para producir software para computadoras paralelas. En un extremo está la adición de bibliotecas numéricas especiales a lenguajes secuenciales por lo demás normales. Por ejemplo, se podría invocar desde un programa secuencial un procedimiento de biblioteca que invierta una matriz grande o resuelva un conjunto de ecuaciones diferenciales parciales en un procesador paralelo sin que el procesador siquiera se dé cuenta de la existencia de paralelismo. El problema con este enfoque es que el paralelismo sólo puede usarse en unos cuantos procedimientos y el grueso del código seguirá siendo secuencial.

Una segunda estrategia es la adición de bibliotecas especiales que contienen primitivas de comunicación y control. Aquí el programador tiene la responsabilidad de crear y controlar el paralelismo dentro de un lenguaje de programación convencional, empleando estas primitivas adicionales.

El siguiente paso es añadir unas cuantas construcciones especiales a lenguajes de programación existentes, como la capacidad de bifurcar fácilmente nuevos procesos paralelos, ejecutar las iteraciones de un ciclo en paralelo, o efectuar operaciones aritméticas con

todos los elementos de un vector al mismo tiempo. Este enfoque se ha adoptado ampliamente y se han modificado muchos lenguajes de programación para que incluyan algo de paralelismo.

La estrategia más extrema sería inventar un lenguaje totalmente nuevo especialmente para el procesamiento en paralelo. La ventaja obvia de inventar un nuevo lenguaje es que éste sería idóneo para la programación en paralelo, pero la desventaja igualmente obvia es que los programadores tendrían que aprender un nuevo lenguaje. Muchos de ellos tienen otras cosas que hacer con su tiempo. Casi todos los lenguajes paralelos nuevos son imperativos (con instrucciones para modificar variables de estado), pero unos cuantos son funcionales, basados en lógica u orientados a objetos.

Ha habido tantas bibliotecas, extensiones y lenguajes inventados para la programación en paralelo, y abarcan una gama tan amplia de posibilidades, que es imposible clasificarlos todos de alguna forma razonable. En lugar de intentarlo, nos concentraremos en cinco cuestiones básicas que constituyen el corazón de todo el software para computadoras paralelas.

1. Modelos de control.
2. Granularidad del paralelismo.
3. Paradigmas computacionales.
4. Métodos de comunicación.
5. Primitivas de sincronización.

Cada una de estas cuestiones se estudiará en su oportunidad.

### Modelos de control

Tal vez la decisión fundamental que el software debe tomar es si habrá un solo enlace de control o varios. En el primer modelo hay un programa y un contador de programa, pero varios conjuntos de datos. Cuando se emite una instrucción, se ejecuta con todos los conjuntos de datos simultáneamente, con diferentes elementos de procesamiento.

Por ejemplo, imagine un programa meteorológico que cada hora recibe mediciones de temperatura de miles de sensores remotos que tienen que promediar para cada sensor. Cuando el programa emite la instrucción

**CARGAR LA TEMPERATURA PARA LA 1 A.M. EN EL REGISTRO R1**

cada procesador la ejecuta utilizando sus propios datos y su propio R1. Después, cuando el programa emite la instrucción

**SUMAR LA TEMPERATURA PARA LAS 2 A.M. AL REGISTRO R1**

una vez más, cada procesador lo hace utilizando sus propios datos. Al final del cálculo, cada procesador habrá calculado la temperatura media para un sensor distinto.

Las implicaciones de este modelo de programación para el hardware son enormes. De hecho, lo que dice es que cada elemento de procesamiento es básicamente una ALU y una memoria, sin lógica propia para decodificar instrucciones. En vez de ello, una sola unidad centralizada trae instrucciones y le dice a las ALU lo que tienen que hacer a continuación.

En el modelo alternativo hay varios enlaces de control, cada uno con su propio contador de programa, registros y variables locales. Cada enlace de control ejecuta su propio programa con sus propios datos, posiblemente comunicándose o sincronizándose con otros enlaces de control cada cierto tiempo. Existen muchas variaciones de esta idea básica, y juntas constituyen el modelo dominante del procesamiento en paralelo. Por esta razón, ahora nos concentraremos primordialmente en la computación en paralelo con múltiples enlaces de control.

### Granularidad del paralelismo

Se puede introducir paralelismo de control en diversos niveles. En el nivel más bajo, las instrucciones de máquina individuales pueden contener paralelismo (por ejemplo, el IA-64). En este nivel, los programadores normalmente no tienen conocimiento del paralelismo; está bajo el control del compilador o del hardware.

Un nivel más arriba está el **paralelismo en el nivel de bloques**, que permite a los programadores controlar explícitamente cuáles enunciados deben ejecutarse secuencialmente y cuáles deben ejecutarse en paralelo. Una de las formas más elegantes de expresar este tipo de paralelismo se debe a Algol 68, en el cual se usaba

**begin** Enunciado-1; Enunciado-2; Enunciado-3 **end**

para crear un bloque con (por ejemplo) tres enunciados arbitrarios que se debían ejecutar en sucesión. En contraste,

**begin** Enunciado-1, Enunciado-2, Enunciado-3 **end**

servía para ejecutar los mismos tres enunciados en paralelo. Mediante una colocación cuidadosa de signos de punto y coma, comas, paréntesis y delimitadores **begin/end**, se podían expresar combinaciones arbitrarias de ejecución secuencial y paralela.

Se presenta un paralelismo con grano un poco más grueso cuando es posible llamar a un procedimiento y no obligar al invocador a esperar a que termine antes de continuar. No esperar implica que el invocador y el invocado se ejecutarán en paralelo. Si el invocador está en un ciclo que llama a un procedimiento en cada iteración y no espera a que termine ninguno de ellos, es posible iniciar un gran número de procedimientos en paralelo a la vez.

Otra forma de paralelismo es tener un método para que un proceso cree o bifurque múltiples **enlaces** o **procesos ligeros**, todos los cuales se ejecutan dentro del espacio de direcciones del proceso. Cada enlace tiene su propio contador de programa, registros y pila, pero fuera de eso comparte el resto del espacio de direcciones (y todas las variables globales) con todos los demás enlaces. (En contraste con los enlaces, diferentes procesos no comparten un mismo espacio de direcciones.) Los enlaces se ejecutan con independencia unos de otros, posiblemente en diferentes CPU. En algunos sistemas, el sistema operativo tiene conoci-

miento de todos los enlaces y se encarga de planificarlos; en otros, cada proceso de usuario se encarga de su propia planificación y gestión de enlaces, sin que el sistema operativo siquiera sepa que existen enlaces.

Por último, la forma más burda de paralelismo es hacer que varios procesos independientes colaboren para resolver un problema. A diferencia de los enlaces, que comparten un solo espacio de direcciones, cada proceso independiente tiene su espacio propio, por lo que los procesos deben cooperar a cierta distancia. Esto implica que el problema tiene que dividirse en fragmentos relativamente grandes, uno para cada proceso. Sin embargo, los procesos independientes ofrecen la mayor oportunidad para aprovechar el paralelismo a gran escala, sobre todo en multicomputadoras.

### Paradigmas computacionales

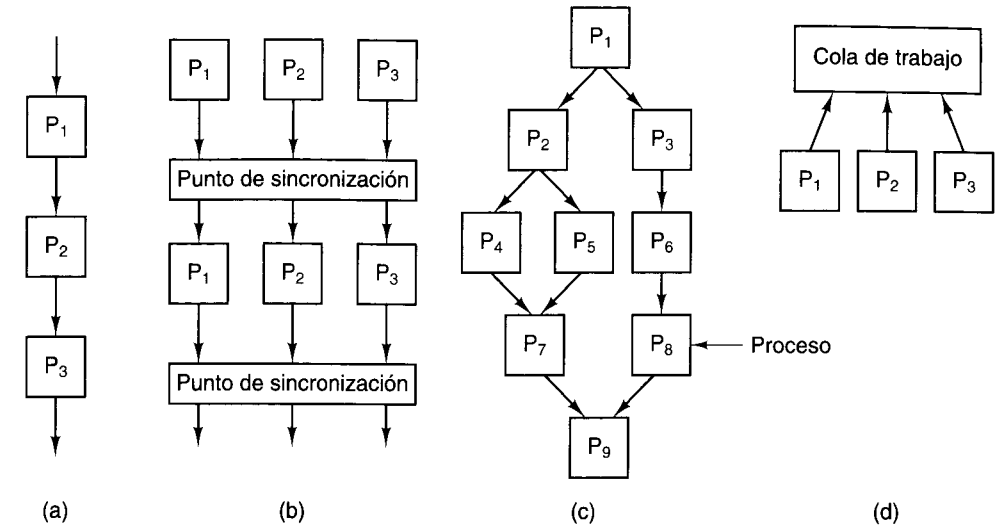
Casi todos los programas paralelos, sobre todo los que implican grandes números de enlaces o procesos independientes, usan algún paradigma subyacente para estructurar su trabajo. Existen muchos de esos paradigmas. En esta sección sólo mencionaremos algunos de los más utilizados.

Una generalización de la idea de tener un programa paralelo que consta de un enlace de control y múltiples unidades de ejecución es el **paradigma de programa único, múltiples datos** (SPMD, *Single Program Multiple Data*). La idea aquí es que aunque el sistema consiste en varios procesos independientes, todos ejecutan el mismo programa pero con diferentes conjuntos de datos. Sólo que ahora, a diferencia del ejemplo de las temperaturas, no están en coordinación, sincronizados hasta la última instrucción. En vez de ello, cada uno realiza el mismo cálculo, pero a su propio ritmo.

Un segundo paradigma es la **fila de procesamiento** (*pipeline*), que se ilustra en la figura 8-11(a) con tres procesos. Los datos se alimentan al primer proceso, que los transforma y los alimenta al segundo proceso, etc. Si el flujo de datos es largo (por ejemplo, una presentación en video), todos los procesadores podrían estar ocupados al mismo tiempo. Las filas de procesamiento UNIX funcionan así y se pueden ejecutar como procesos individuales en paralelo en una multicomputadora o un multiprocesador.

Otro paradigma es el cálculo de la figura 8-11(b), en el que el trabajo se divide en fases, por ejemplo, iteraciones de un ciclo. Durante cada fase, varios procesos trabajan en paralelo, pero cuando cada uno termina espera hasta que los demás han terminado también antes de iniciar la siguiente fase. Un cuarto paradigma es el de **divide y vencerás**, que se ilustra en la figura 8-11(c). En este paradigma un proceso inicia y luego se bifurca en otros procesos a los que puede delegar parte del trabajo. Una analogía en el mundo de la construcción es un contratista general que recibe un pedido, y luego subcontrata gran parte del trabajo a subcontratistas de albañilería, electricidad, plomería, pintura y otras especialidades. Éstos, a su vez, podrían subcontratar parte de su trabajo a otros subcontratistas especializados.

Nuestro último modelo es el **paradigma del trabajador repetido**, a veces llamado **granja de tareas**, el cual se ilustra en la figura 8-11(d). Aquí se tiene una cola de trabajo centralizada y los trabajadores sacan tareas de la cola y las llevan a cabo. Si una tarea genera nuevas tareas, éstas se añaden a la cola central. Cada vez que un trabajador termina su tarea actual, acude a



**Figura 8-11.** Paradigmas computacionales. (a) Fila de procesamiento. (b) Cómputo en fases. (c) Divide y vencerás. (d) Trabajador repetido.

la cola de tareas para obtener una nueva. Este paradigma también puede implementarse con un administrador activo en la parte superior que reparte el trabajo, en lugar de obligar a los trabajadores que lo consigan por su cuenta.

### Métodos de comunicación

Cuando un programa se divide en fragmentos (digamos, procesos) que se ejecutan en paralelo, los fragmentos (procesos) a menudo necesitan comunicarse entre sí. Esta comunicación puede efectuarse de una de dos maneras: variables compartidas o transferencia explícita de mensajes. En el primer método, todos los procesos tienen acceso a la memoria lógica común y pueden comunicarse leyéndola y escribiendo en ella. Por ejemplo, un proceso puede ajustar una variable y otro proceso puede leerla.

En un multiprocesador, las variables pueden compartirse entre múltiples procesos mapeando la misma página al espacio de direcciones de cada proceso. Entonces se pueden leer y escribir variables compartidas empleando instrucciones de máquina **LOAD** y **STORE**. En una multicomputadora, aunque no tiene memoria física compartida, también es posible compartir variables lógicamente, aunque es un poco más complicado. Como vimos antes, Linda maneja el concepto de espacio de tuplas compartido, aun en una multicomputadora, y Orca maneja el concepto de objetos compartidos cruzando fronteras de máquinas. Otra posibilidad es compartir un solo espacio de direcciones en una multicomputadora y paginar a través de la red de interconexión. En síntesis, es posible permitir que los procesos se comuniquen a través de variables compartidas tanto en multiprocesadores como en multicomputadoras.

La alternativa a la comunicación a través de memoria compartida es la comunicación por transferencia explícita de mensajes. En este modelo, los procesos usan primitivas como **send** y **receive** para comunicarse. Un proceso emite un **send**, nombrando a otro proceso como destino. Tan pronto como el segundo emite un **receive**, el mensaje se copia en el espacio de direcciones del receptor.

La transferencia de mensajes tiene muchas variaciones, pero todas se reducen a tener dos primitivas **send** y **receive** que sirven para transmitir mensajes. Éstas generalmente se implementan como llamadas al sistema. Veremos algunos de los detalles más adelante en este capítulo.

Otro aspecto importante de la transferencia de mensajes es el número de receptores. El caso más simple es el de un transmisor y un receptor, llamado **transferencia de mensajes punto a punto**. Sin embargo, a veces es útil entregar un mensaje a todos los procesos, en lo que se conoce como **difusión** (*broadcasting*), o a un subconjunto específico de los procesos, en lo que se conoce como **multidifusión** (*multicasting*).

Cabe señalar que la transferencia de mensajes es fácil de implementar en un multiprocesador con sólo copiar del transmisor al receptor. Por tanto, las cuestiones de memoria física compartida (multiprocesador *versus* multicomputadora) y memoria lógica compartida (comunicación a través de variables compartidas *versus* transferencia explícita de mensajes) son independientes. Las cuatro combinaciones tienen sentido y pueden implementarse; se enumeran en la figura 8-12.

| Físico<br>(hardware) | Lógico<br>(software)  | Ejemplos                                               |
|----------------------|-----------------------|--------------------------------------------------------|
| Multiprocesador      | Variables compartidas | Procesamiento de imágenes como en la figura 8-1        |
| Multiprocesador      | Transf. de mensajes   | Transf. de mensajes simulada con buffers en la memoria |
| Multicomputadora     | Variables compartidas | DSM, Linda, Orca, etc. en un SP/2 o una red de PC      |
| Multicomputadora     | Transf. de mensajes   | PVM o MPI en un SP/2 o una red de PC                   |

Figura 8-12. Combinaciones de compartimiento físico y lógico.

Primitivas de sincronización

Los procesos paralelos no sólo necesitan comunicarse; con frecuencia también necesitan sincronizar sus acciones. Un ejemplo de sincronización es cuando los procesos comparten variables lógicamente y tienen que asegurarse de que mientras un proceso está escribiendo en una estructura de datos compartida ningún otro proceso esté tratando de leerla. En otras palabras, se requiere alguna forma de **exclusión mutua** para evitar que varios procesos usen los mismos datos al mismo tiempo.

Existen diversas primitivas de software que pueden servir para asegurar la exclusión mutua. Éstas incluyen semáforos, candados, mutexes y secciones críticas. La característica común de todas estas primitivas es que permiten a un proceso solicitar el uso exclusivo de algún recurso (variable compartida, dispositivo de E/S, etc.). Si se otorga el permiso, el pro-

ceso puede usar el recurso. Si un segundo proceso pide permiso mientras el primero todavía está usando el recurso, se le negará el permiso o se bloqueará hasta que el primer proceso haya liberado el recurso.

Un segundo tipo de primitiva de sincronización que se necesita en muchos programas paralelos es alguna forma de permitir que todos los procesos se bloqueen hasta que finalice cierta fase del trabajo, como se muestra en la figura 8-11(b). Una primitiva común aquí es la **barrera**. Cuando un proceso llega a una barrera, se bloquea hasta que todos los procesos llegan a ella. Cuando llega el último proceso, todos los procesos se liberan simultáneamente y pueden continuar.

8.1.5 Taxonomía de computadoras paralelas

Aunque podríamos hablar mucho más acerca del software para computadoras paralelas, las limitaciones de espacio nos obligan a regresar al tema principal de este capítulo, la arquitectura de las computadoras paralelas. Se han propuesto y construido muchos tipos de computadoras paralelas, por lo que es natural preguntarse si hay alguna forma de clasificarlas dentro de alguna taxonomía. Muchos investigadores lo han intentado, con regular éxito (Flynn, 1972; Gajski y Pier, 1985; Treleaven, 1985). Lamentablemente, todavía no ha surgido el Carlos Linneo† de la computación en paralelo. El único esquema que se usa mucho es el de Flynn, pero hasta el de él es, en el mejor de los casos, una aproximación muy burda (figura 8-13).

| Flujos de instrucciones | Flujos de datos | Nombre | Ejemplos                                           |
|-------------------------|-----------------|--------|----------------------------------------------------|
| 1                       | 1               | SISD   | Máquina clásica de von Neumann                     |
| 1                       | Varios          | SIMD   | Supercomputadora vectorial, procesador de arreglos |
| Varios                  | 1               | MISD   | Tal vez ninguno                                    |
| Varios                  | Varios          | MIMD   | Multiprocesador, multicomputadora                  |

Figura 8-13. Taxonomía de Flynn para computadoras paralelas.

La clasificación de Flynn se basa en dos conceptos: flujos de instrucciones y flujos de datos. Un flujo de instrucciones corresponde a un contador de programa. Un sistema que tiene *n* CPU tiene *n* contadores de programa, y por tanto *n* flujos de instrucciones.

Un flujo de datos consiste en un conjunto de operandos. El ejemplo de cálculo de temperaturas que se dio antes tiene varios flujos de datos, uno para cada sensor.

Los flujos de instrucciones y de datos son, hasta cierto punto, independientes, por lo que existen cuatro combinaciones, las cuales se enumeran en la figura 8-13. SISD es la computadora secuencial clásica de von Neumann; tiene un flujo de instrucciones, un flujo de datos y

† Carlos Linneo (1707-1778) fue el biólogo sueco que ideó el sistema que ahora se usa para clasificar todas las plantas y animales según su reino, phylum, clase, orden, familia, género y especie.

hace una cosa a la vez. Las máquinas SIMD tienen una sola unidad de control que emite una instrucción a la vez, pero tienen múltiples ALU para ejecutarla con varios conjuntos de datos simultáneamente. La ILLIAC IV (figura 2-8) es el prototipo de las máquinas SIMD. Existen máquinas SIMD modernas y se usan para cálculos científicos.

Las máquinas MISD son una categoría un tanto extraña, en la que varias instrucciones operan con un mismo dato. No se sabe a ciencia cierta si existen máquinas de este tipo, aunque algunas personas clasifican a las máquinas con filas de procesamiento como MISD.

Por último, tenemos las MIMD, que no son más que múltiples CPU independientes que operan como parte de un sistema mayor. Casi todos los procesadores paralelos pertenecen a esta categoría. Tanto los multiprocesadores como las multicomputadoras son máquinas MIMD.

La taxonomía de Flynn termina aquí, pero la hemos extendido en la figura 8-14. Hemos dividido a SIMD en dos subgrupos. El primero es el de las supercomputadoras numéricas y otras máquinas que operan con vectores, realizando la misma operación con cada elemento del vector. El segundo es para las máquinas tipo paralelo, como la ILLIAC IV, en las que una unidad de control maestra, difunde instrucciones a muchas ALU independientes.

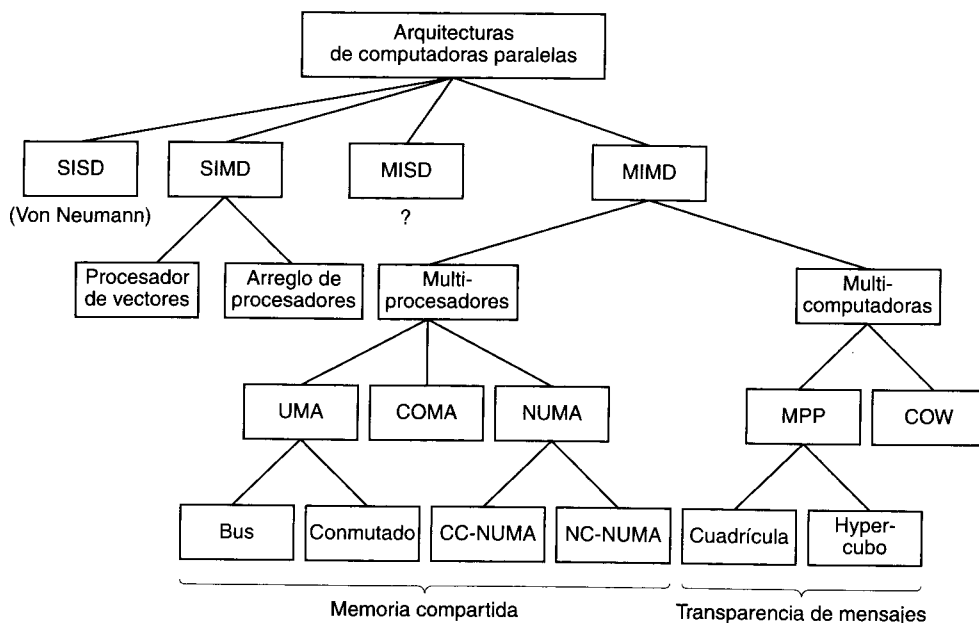


Figura 8-14. Una taxonomía de computadoras paralelas.

En nuestra taxonomía, la categoría MIMD se ha dividido en multiprocesadores (máquinas con memoria compartida) y multicomputadoras (máquinas que transfieren mensajes). Existen tres clases de multiprocesadores, que se distinguen por la forma en que se implementa la memoria compartida: **acceso uniforme a la memoria** (UMA, *Uniform Memory Access*),

**acceso no uniforme a la memoria** (NUMA, *NonUniform Memory Access*) y **sólo acceso a memoria caché** (COMA, *Cache Only Memory Access*). Estas categorías existen porque en los multiprocesadores grandes la memoria normalmente se divide en varios módulos. Las máquinas UMA tienen la propiedad de que cada CPU tiene el mismo tiempo de acceso a todos los módulos de memoria. En otras palabras, cada palabra de memoria se puede leer con la misma rapidez que cualquier otra palabra de memoria. Si esto es técnicamente imposible, las referencias más rápidas se frenan de modo que tarden lo mismo que las más lentas y los programadores no noten las diferencias. Éste es el significado de “uniforme” aquí. Esta uniformidad hace que el desempeño sea predecible, lo cual es un factor importante para escribir código eficiente.

En contraste, un multiprocesador NUMA no posee esta propiedad. Es común que haya un módulo de memoria cercano a cada CPU y que el acceso a ese módulo sea mucho más rápido que el acceso a módulos distantes. El resultado es que, por razones de desempeño, es importante dónde se coloquen el código y los datos. Las máquinas COMA tampoco son uniformes, pero de diferente manera. Estudiaremos cada uno de estos tipos y sus subcategorías con detalle más adelante.

La otra categoría principal de máquinas MIMD comprende las multicomputadoras que, a diferencia de los multiprocesadores, no tienen una memoria primaria compartida en el nivel arquitectónico. En otras palabras, el sistema operativo de una CPU de multicomputadora no puede acceder a memoria conectada a una CPU distinta con sólo ejecutar una instrucción **LOAD**; tiene que enviar un mensaje explícito y esperar una respuesta. La capacidad del sistema operativo para leer una palabra distante con sólo emitir una **LOAD** es lo que distingue a los multiprocesadores de las multicomputadoras. Como ya dijimos, aun en una multicomputadora los programas de usuario podrían tener la capacidad para acceder a memoria remota usando instrucciones **LOAD** y **STORE**, pero esta ilusión es mantenida por el sistema operativo, no por el hardware. La diferencia es sutil, pero muy importante. Dado que las multicomputadoras no tienen acceso directo a memoria remota, suele describirseles como máquinas **sin acceso a memoria remota**, (NORMA, *NO Remote Memory Access*).

Las multicomputadoras se pueden dividir a grandes rasgos en dos categorías. La primera categoría contiene los **procesadores masivamente paralelos** (MPP, *Massively Parallel Processors*), que son supercomputadoras caras que consisten en muchas CPU acopladas estrechamente por una red de interconexión patentada de alta velocidad. La Cray T3E y la SP/2 de IBM son ejemplos muy conocidos.

La otra categoría consiste en PC o estaciones de trabajo normales, tal vez montadas en anaqueles, y conectadas mediante tecnología comercial ordinaria. En el aspecto lógico no hay mucha diferencia, pero las enormes supercomputadoras que cuestan muchos millones de dólares se usan de diferente manera que las redes de PC armadas por los usuarios a una fracción del precio de un MPP. Estas máquinas “hechas en casa” reciben diversos nombres, como **redes de estaciones de trabajo** (NOW, *Network of Workstations*) y **cúmulos de estaciones de trabajo** (COW, *Cluster of Workstations*).

En las secciones que siguen examinaremos con cierto detalle las principales categorías: SIMD, MIMD/multiprocesadores y MIMD/multicomputadoras. La meta es presentar un panorama general de cada tipo, sus subcategorías y sus principios de diseño fundamentales. Se usarán varios ejemplos como ilustración.

## 8.2 COMPUTADORAS SIMD

Las computadoras SIMD (un solo flujo de instrucciones, múltiples flujos de datos; *Single Instruction stream Multiple Data stream*) se usan para resolver problemas científicos y de ingeniería que son computacionalmente intensivos e implican estructuras de datos regulares como vectores y arreglos. Estas máquinas se caracterizan por tener una sola unidad de control que ejecuta instrucciones una por una, pero cada instrucción opera con varios datos. Las dos especies principales de computadoras SIMD son los arreglos de procesadores y los procesadores de vectores, que examinaremos a continuación.

### 8.2.1 Arreglos de procesadores

La idea de un arreglo de procesadores surgió hace más de 40 años (Unger, 1958), pero tuvieron que pasar 10 años para que se construyera el primero, ILLIAC IV, y entrara en servicio (para la NASA). Desde entonces, varias compañías han construido arreglos de procesadores comerciales, como el CM-2 de Thinking Machines y el Maspar MP-2, pero ninguno de ellos ha tenido un éxito comercial espectacular.

La idea en que se basa el **arreglo de procesadores** es que una sola unidad de control proporciona las señales que controlan muchos elementos de procesamiento, como se muestra en la figura 2-7. Cada elemento de procesamiento consiste en una CPU o ALU ampliada y por lo regular un poco de memoria local. Puesto que una sola unidad de control está alimentando a todos los elementos de procesamiento, éstos operan de forma sincronizada.

Aunque todos los arreglos de procesadores siguen este modelo general, hay varios aspectos de diseño en los que pueden diferir. El primero es la estructura del elemento de procesamiento. Dichos elementos pueden ir desde los extraordinariamente simples hasta otros más complejos. En el extremo inferior, los elementos de procesamiento pueden ser poco más que una ALU de un bit, como en el caso del CM-2. En esa máquina, cada ALU tomaba dos operandos de un bit de su memoria local, más un bit de su palabra de situación del programa (digamos, el bit de acarreo). El resultado de la operación era un bit de datos y algunos bits indicadores. Para realizar una suma entera de 32 bits, la unidad de control tenía que difundir una instrucción de suma de un bit 32 veces. A 600 ns por instrucción, se requerían 19.2  $\mu$ s para efectuar una suma entera, más tiempo que en la IBM PC original. Sin embargo, si trabajaban todos los 65,536 elementos de procesamiento, era posible realizar 3000 millones de sumas por segundo, lo que daba un tiempo de suma efectivo de 300 picosegundos.

Otras opciones para el elemento de procesamiento incluyen una ALU de 8 bits, una ALU de 32 bits o una unidad más potente con capacidad de punto flotante. Hasta cierto punto, la decisión de qué incluir en el elemento de procesamiento depende de los objetivos de la máquina. Para triturar números, la capacidad de punto flotante es razonable (aunque su costo seguramente reducirá mucho el número de elementos de procesamiento), pero para recuperación de información tal vez no se necesite.

Un segundo aspecto de diseño es la forma de interconectar los elementos de procesamiento. En principio, casi todas las topologías que se ilustran en la figura 8-4 son posibles candidatos. Las cuadrículas rectangulares son una opción común porque se ajustan bien a

muchos problemas bidimensionales que implican matrices e imágenes y su escala se puede aumentar con facilidad ya que la adición de más procesadores automáticamente aumenta el ancho de banda.

Una tercera cuestión de diseño es el grado de autonomía local que los elementos de procesamiento tienen. En el tipo de diseño que estamos describiendo, la unidad de control especifica la instrucción que debe ejecutarse, pero en muchos procesadores de arreglos cada elemento de procesamiento puede optar por ejecutar o no ejecutar una instrucción, con base en datos locales como bits de código de condición. Esta característica añade mucha flexibilidad. Puesto que los procesadores de arreglos tienen un futuro incierto, no profundizaremos más en ellos.

### 8.2.2 Procesadores vectoriales

El otro tipo de máquina SIMD es el **procesador vectorial**, que ha tenido un éxito comercial mucho mayor. Un linaje de máquinas diseñadas por Seymour Cray para Cray Research (ahora parte de Silicon Graphics), comenzando con la Cray-1 en 1976 y continuando hasta la C90 y la T90, ha dominado la computación científica durante décadas. En esta sección examinaremos los principios fundamentales que se aplican en estas computadoras de alto desempeño.

Una aplicación de trituración de números típica contiene muchos enunciados como

```
for (i = 0; i < n; i++) a[i] = b[i] + c[i];
```

donde  $a$ ,  $b$  y  $c$  son **vectores**, es decir, arreglos de números, casi siempre en punto flotante. El ciclo ordena a la computadora sumar los  $i$ -ésimos elementos de  $b$  y  $c$  y almacenar el resultado en el  $i$ -ésimo elemento de  $a$ . Técnicamente, el programa especifica que los elementos deben sumarse en sucesión, pero normalmente el orden no importa.

En la figura 8-15 se muestra una posible arquitectura SIMD apropiada para este tipo de procesamiento vectorial. La máquina recibe dos vectores de  $n$  elementos como entradas y opera con los elementos correspondientes en paralelo utilizando una ALU vectorial que puede operar con los  $n$  elementos simultáneamente y produce como resultado un vector. Los vectores de entrada y de salida se pueden almacenar en la memoria o en registros vectoriales especiales.

Las computadoras vectoriales también necesitan efectuar operaciones escalares (no vectoriales) y operaciones mixtas escalares-vectoriales. Los tipos básicos de operaciones vectoriales se enumeran en la figura 8-16. La primera,  $f_1$ , efectúa alguna operación, como coseno o raíz cuadrada, con cada elemento de un solo vector. La segunda,  $f_2$ , toma un vector como entrada y produce un escalar como salida. Un ejemplo típico es sumar todos los elementos. La tercera,  $f_3$ , realiza una operación diádica con dos vectores, como sumar los elementos correspondientes. Por último,  $f_4$  combina un operando escalar con uno vectorial. Un ejemplo típico es multiplicar cada elemento de un vector por una constante. A veces es más rápido convertir el escalar en un vector, cuyos elementos son todos iguales al escalar, y luego realizar la operación con dos vectores.

Todas las operaciones comunes con vectores pueden construirse utilizando estas formas. Por ejemplo, el producto interior (producto punto) de dos vectores consiste en multiplicar primero los elementos correspondientes ( $f_3$ ) y obtener después la sumatoria del resultado ( $f_2$ ).

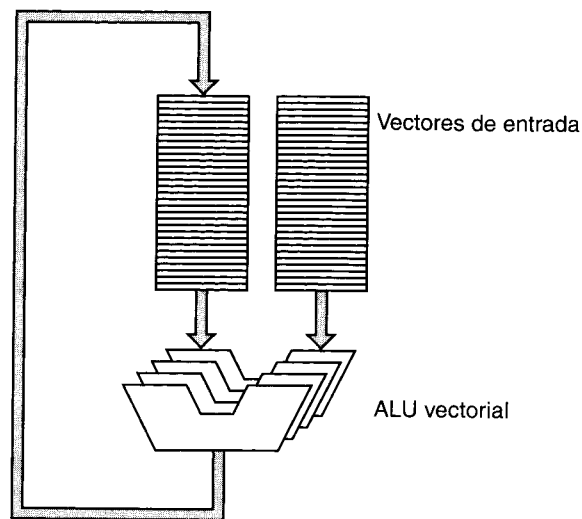


Figura 8-15. ALU vectorial.

| Operación                        | Ejemplos                                    |
|----------------------------------|---------------------------------------------|
| $A_i = f_1(B_i)$                 | $f_1$ = coseno, raíz cuadrada               |
| Escalar = $f_2(A)$               | $f_2$ = sumatoria, mínimo                   |
| $A_i = f_3(B_i, C_i)$            | $f_3$ = sumar, restar                       |
| $A_i = f_4(\text{escalar}, B_i)$ | $f_4$ = multiplicar $B_i$ por una constante |

Figura 8-16. Diversas combinaciones de operaciones vectoriales y escalares.

En la práctica, pocas supercomputadoras se construyen en realidad como lo que se muestra en la figura 8-15. La razón no es técnica —ciertamente es factible construir tales máquinas— sino económica. Un diseño con, digamos, 64 ALUs de muy alta velocidad sería demasiado costoso, incluso para una supercomputadora.

El método que se emplea normalmente es combinar el procesamiento de vectores con el uso de filas de procesamiento. Las operaciones de punto flotante son muy complejas, requieren varios pasos, y cualquier operación de múltiples pasos es candidata para una implementación con filas de procesamiento. Si no está familiarizado con la aritmética de punto flotante, lea el Apéndice B.

Como ejemplo de uso de filas de procesamiento para operaciones de punto flotante, considere la figura 8-17. En este ejemplo, un número normalizado tiene una mantisa mayor o igual que 1, pero menor que 10. De lo que se trata aquí es de restar  $9.212 \times 10^{11}$  a  $1.082 \times 10^{12}$ .

Para restar dos números de punto flotante primero es necesario ajustarlos de modo que sus exponentes tengan el mismo valor. En este ejemplo podemos convertir el sustraendo (el

| Paso | Nombre               | Valores                                        |
|------|----------------------|------------------------------------------------|
| 1    | Traer operandos      | $1.082 \times 10^{12} - 9.212 \times 10^{11}$  |
| 2    | Ajustar exponente    | $1.082 \times 10^{12} - 0.9212 \times 10^{12}$ |
| 3    | Ejecutar resta       | $0.1608 \times 10^{12}$                        |
| 4    | Normalizar resultado | $1.608 \times 10^{11}$                         |

Figura 8-17. Pasos de una resta de punto flotante.

número que se resta) a  $0.9212 \times 10^{12}$  o bien convertir el minuendo (el número del cual se resta) a  $10.82 \times 10^{11}$ . En general, ambas conversiones implican riesgos. Elevar un exponente podría causar un subdesbordamiento de la mantisa, y bajar un exponente podría causar un desbordamiento de la mantisa. El subdesbordamiento es el menos grave de los dos, porque el número se puede aproximar con 0, así que seguiremos la primera ruta. Después de ajustar ambos exponentes a 12, tenemos los valores que se muestran en el paso 2 de la figura 8-17. A continuación realizamos la resta, seguida de la normalización del resultado.

El uso de filas de procesamiento puede aplicarse al ciclo for loop que se presentó al principio de esta sección. En la figura 8-18 vemos un sumador de punto flotante con filas de procesamiento de cuatro etapas. En cada ciclo, la primera etapa obtiene un par de operandos. Luego, la segunda etapa ajusta el exponente del operando más pequeño de modo que coincida con el del más grande. La tercera etapa efectúa la operación, y la cuarta normaliza el resultado. De este modo, un resultado sale de la fila de procesamiento en cada ciclo.

| Paso                 | Ciclo      |            |            |            |            |            |            |
|----------------------|------------|------------|------------|------------|------------|------------|------------|
|                      | 1          | 2          | 3          | 4          | 5          | 6          | 7          |
| Buscar operandos     | $B_1, C_1$ | $B_2, C_2$ | $B_3, C_3$ | $B_4, C_4$ | $B_5, C_5$ | $B_6, C_6$ | $B_7, C_7$ |
| Ajustar exponente    |            | $B_1, C_1$ | $B_2, C_2$ | $B_3, C_3$ | $B_4, C_4$ | $B_5, C_5$ | $B_6, C_6$ |
| Ejecutar operación   |            |            | $B_1, C_1$ | $B_2, C_2$ | $B_3, C_3$ | $B_4, C_4$ | $B_5, C_5$ |
| Normalizar resultado |            |            |            | $B_1, C_1$ | $B_2, C_2$ | $B_3, C_3$ | $B_4, C_4$ |

Figura 8-18. Sumador de punto flotante con filas de procesamiento.

Una diferencia importante entre usar filas de procesamiento para operaciones con vectores y usarla para la ejecución de instrucciones de propósito general es la ausencia de saltos al operar con vectores. Cada ciclo se aprovecha al máximo, y no se desperdician ranuras.

### La supercomputadora vectorial Cray-1

Las supercomputadoras por lo regular tienen múltiples ALUs, cada una especializada en alguna operación específica, y todas capaces de operar en paralelo. Por ejemplo, consideremos la Cray-1, una de las primeras supercomputadoras. Aunque ya no se usa, tiene una arquitectura simple tipo RISC que la convierte en un buen objeto de estudio, y su arquitectura perdura en muchas supercomputadoras vectoriales modernas.



Como la generalidad de las máquinas RISC, la Cray-1 está orientada hacia los registros, con instrucciones en su mayoría de 16 bits que consisten en un código de operación de siete bits y tres números de registro de tres bits para los tres operandos. Hay cinco clases de registros, como se muestra en la figura 8-19. Los ocho registros A de 24 bits sirven para direccionar la memoria. Los 64 registros B de 24 bits sirven para guardar registros A cuando no se necesitan, en lugar de escribirlos de vuelta en la memoria. Los ocho registros S de 64 bits sirven para contener cantidades escalares (enteras y de punto flotante). Los valores de estos registros pueden utilizarse como operandos para operaciones tanto enteras como de punto flotante. Los 64 registros T de 64 bits son almacenamiento adicional para los registros S, otra vez con objeto de reducir el número de instrucciones **LOAD** y **STORE**.

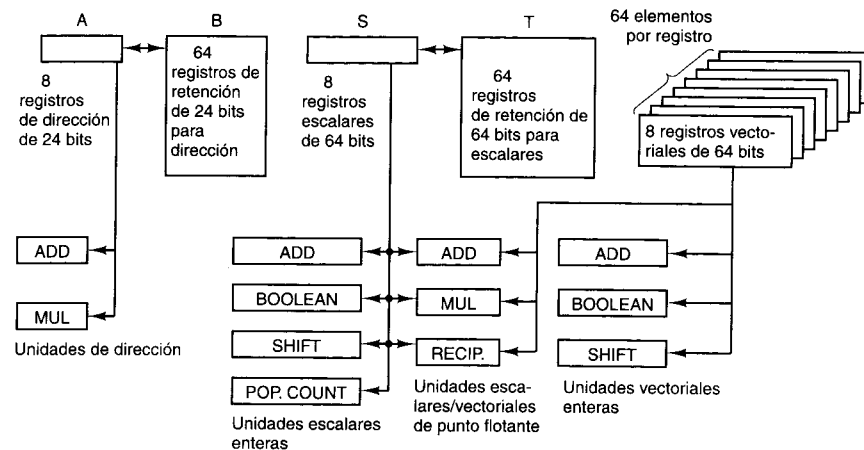


Figura 8-19. Registros y unidades funcionales de la Cray-1.

La parte más interesante del conjunto de registros de la Cray-1 es el grupo de ocho registros vectoriales. Cada registro puede contener un vector de punto flotante de 64 elementos. Es posible sumar, restar o multiplicar dos vectores con una sola instrucción de 16 bits. No es posible dividir, pero sí calcular recíprocos. Los registros vectoriales se pueden cargar de la memoria y guardarse en ella, pero tales transferencias son costosas, y lo mejor es minimizarlas. Todas las operaciones con vectores usan operandos en registros.

No todas las supercomputadoras tienen la propiedad estilo RISC de requerir que todos los operandos estén en registros. La CDC Cyber 205, por ejemplo, realizaba operaciones con vectores en la memoria. Este enfoque permitía manejar vectores de longitud arbitraria pero hacía mucho más lenta la máquina porque la memoria era un cuello de botella importante.

La Cray-1 tiene 12 unidades funcionales distintas, como se ilustra en la figura 8-19. Dos son para aritmética con direcciones de 24 bits y cuatro son para operaciones con escalares enteros de 64 bits. Al igual que su antepasada, la CDC 6600, la Cray-1 carece de una unidad para efectuar multiplicación entera (aunque sí tiene una para multiplicación de punto flotante). Las seis unidades restantes trabajan con vectores, y su estructura interna utiliza muchas filas de procesamiento. Las unidades de suma, multiplicación y recíproco también operan con números escalares de punto flotante, además de con vectores.

Al igual que muchas otras computadoras vectoriales, la Cray-1 permite **encadenar operaciones**. Por ejemplo, una forma de calcular

$$R1 = R1 * R2 + R3$$

donde **R1**, **R2** y **R3** son registros vectoriales, sería efectuar la multiplicación vectorial, elemento por elemento, guardar el resultado en algún lugar, y luego efectuar la suma vectorial. Con el encadenamiento, tan pronto como se han multiplicado los primeros elementos, el producto puede ir directamente al sumador, junto con el primer elemento de **R3**. No se requiere un almacenamiento intermedio. Esta técnica mejora considerablemente el desempeño.

Como comentario final acerca de la Cray-1, resulta interesante examinar su desempeño absoluto. El reloj opera a 80 MHz y tiene una memoria principal de 8 MB. En su época (de mediados a fines de los años setenta) era la computadora más potente del mundo. Hoy día ya no es posible comprar una PC con un reloj tan lento ni una memoria tan pequeña; ya nadie las fabrica. Esta observación es un testimonio a la rapidez con que avanza la industria de las computadoras.

### 8.3 MULTIPROCESADORES CON MEMORIA COMPARTIDA

Como vimos en la figura 8-14, los sistemas MIMD se pueden dividir en multiprocesadores y multicomputadoras. En esta sección examinaremos los multiprocesadores; la siguiente estará dedicada a las multicomputadoras. Un multiprocesador es un sistema de cómputo que tiene varias CPU y un solo espacio de direcciones visible para todas las CPU. La máquina ejecuta una copia del sistema operativo, con un conjunto de tablas que incluyen las tablas en las que se lleva la contabilidad de cuáles páginas de memoria están ocupadas y cuáles están libres. Cuando un proceso se bloquea, su CPU guarda su estado en las tablas del sistema operativo y busca en esas tablas otro proceso que pueda ejecutar. Es esta imagen de sistema único lo que distingue un multiprocesador de una multicomputadora.

Un multiprocesador, como todas las computadoras, debe tener dispositivos de E/S, como discos, adaptadores de red y otros equipos. En algunos sistemas de multiprocesador sólo ciertas CPU tienen acceso a los dispositivos de E/S, y por tanto tienen una función de E/S especial. En otros, cada CPU tiene el mismo acceso a todos los dispositivos de E/S. Si toda CPU tiene igual acceso a todos los módulos de memoria y todos los dispositivos de E/S, y el sistema operativo la trata como intercambiable con las demás, el sistema se describe como **multiprocesador simétrico** (SMP, *Symmetric MultiProcessor*). La explicación que sigue está orientada hacia este tipo de sistemas.

#### 8.3.1 Semántica de la memoria

Aunque todos los multiprocesadores presentan a las CPU la imagen de un solo espacio de direcciones compartido, a menudo hay muchos módulos de memoria presentes, y cada uno contiene alguna porción de la memoria física. Las CPU y las memorias a menudo están conectadas por una red de interconexión compleja, como se explicó en la sección 8.1.2. Va-

rias CPU pueden estar intentando leer una palabra de memoria al mismo tiempo que varias otras CPU están tratando de escribir la misma palabra, y algunos de los mensajes de solicitud podrían rebasar a otros en el camino y entregarse en un orden distinto de aquel en que fueron emitidos. Si a este problema sumamos la existencia de múltiples copias de algunos bloques de memoria (por ejemplo, en cachés), el resultado puede convertirse fácilmente en un caos si no se toman medidas estrictas para evitarlo. En esta sección veremos lo que significa realmente la memoria compartida y cómo pueden responder razonablemente las memorias en estas condiciones.

Una perspectiva de la semántica de memoria es verla como un contrato entre el software y el hardware de memoria (Adve y Hill, 1990). Si el software acepta cumplir con ciertas reglas, la memoria conviene en entregar ciertos resultados. La discusión se centra entonces en la naturaleza de las reglas. Estas reglas se denominan **modelos de consistencia** y se han propuesto e implementado varios de ellos.

Para tener una idea de la naturaleza del problema, supongamos que la CPU 0 escribe el valor 1 en alguna palabra de memoria y poco después la CPU 1 escribe el valor 2 en la misma palabra. Ahora la CPU 2 lee la palabra y obtiene el valor 1. ¿El dueño de la computadora debe llevar la máquina al taller para que la reparen? Todo depende de lo que la memoria prometió (su contrato).

### Consistencia estricta

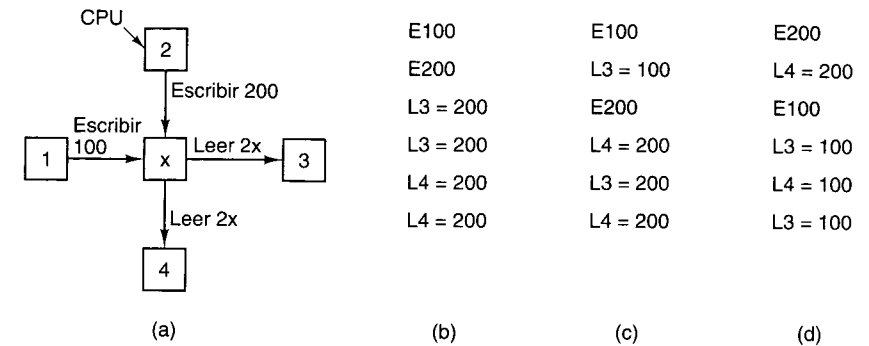
El modelo más sencillo es el de **consistencia estricta**. Con este modelo, cualquier lectura de una posición  $x$  siempre devuelve el valor de la escritura más reciente en  $x$ . Los programadores aman a este modelo, pero en la práctica es imposible de implementar de otra manera que no sea tener un solo módulo de memoria que atiende todas las solicitudes bajo el régimen de primero que llega, primero que se atiende, sin uso de cachés ni repetición de datos. Una implementación así convertiría a la memoria en un enorme cuello de botella y por ello no es un candidato serio, lo cual es lamentable.

### Consistencia secuencial

El siguiente mejor modelo se llama **consistencia secuencial** (Lamport, 1979). La idea aquí es que, en presencia de múltiples solicitudes de lectura y escritura, el hardware escoge (de forma no determinista) cierta intercalación de todas las solicitudes, pero todas las CPU perciben el mismo orden.

Para entender qué significa esto, consideremos un ejemplo. Suponga que la CPU 1 escribe el valor 100 en la palabra  $x$ , y 1 ns después la CPU 2 escribe el valor 200 en la palabra  $x$ . Ahora suponga que 1 ns después de emitirse la segunda instrucción de escritura (que no necesariamente ha terminado de ejecutarse) otras dos CPU, la 3 y la 4, leen la palabra  $x$  dos veces en rápida sucesión, como se muestra en la figura 8-20(a). En la figura 8-20(b)-(d) se muestran tres posibles ordenamientos de los seis sucesos (dos escrituras y cuatro lecturas). En la figura 8-20(b), la CPU 3 obtiene (200, 200) y la CPU 4 obtiene (200, 200). En la figura 8-20(c), las CPU obtienen (100, 200) y (200, 200), respectivamente. En la figura 8-20(d) las

CPU obtienen (100, 100) y (200, 200), respectivamente. Todas estas posibilidades son válidas, además de otras que no se muestran.



**Figura 8-20.** (a) Dos CPU escriben y dos CPU leen una palabra de memoria en común. (b) – (d) Tres posibles intercalados en el tiempo de las dos escrituras y cuatro lecturas.

Sin embargo —y ésta es la esencia de la consistencia secuencial— pase lo que pase, una memoria secuencialmente consistente nunca permitirá que la CPU 3 obtenga (100, 200) mientras la CPU 4 obtiene (200, 100). Si ocurriera esto, implicaría que, según la CPU 4, la escritura de 100 por la CPU 1 terminó antes que la escritura de 200 por la CPU 2. No hay problema con eso, pero también implicaría que, según la CPU 3, la escritura de 200 por la CPU 2 terminó antes que la escritura de 100 por la CPU 1. Por sí solo, este resultado también es posible. El problema es que la consistencia secuencial garantiza que hay un solo ordenamiento global de todas las escrituras que todas las CPU ven. Si la CPU 3 observa que 100 se escribió primero, entonces la CPU 4 también deberá ver este orden.

Aunque la consistencia secuencial no es una regla tan potente como la consistencia estricta, de todos modos es muy útil. En realidad, lo que está diciendo es que, cuando suceden varias cosas al mismo tiempo, existe un orden verdadero en el que ocurren, tal vez determinado por los tiempos y la casualidad, pero todos los procesadores observan este mismo orden. Aunque esta afirmación puede parecer obvia, a continuación analizaremos modelos de consistencia que no garantizan ni siquiera esto.

### Consistencia del procesador

Un modelo de consistencia menos rígido, pero que es más fácil de implementar en multiprocesadores grandes, es la **consistencia del procesador** (Goodman, 1989). Este modelo tiene dos propiedades:

1. Todas las CPU ven las escrituras de cualquier CPU en el orden en que se emitieron.
2. Para cada palabra de la memoria, todas las CPU ven todas las escrituras en ella en el mismo orden.

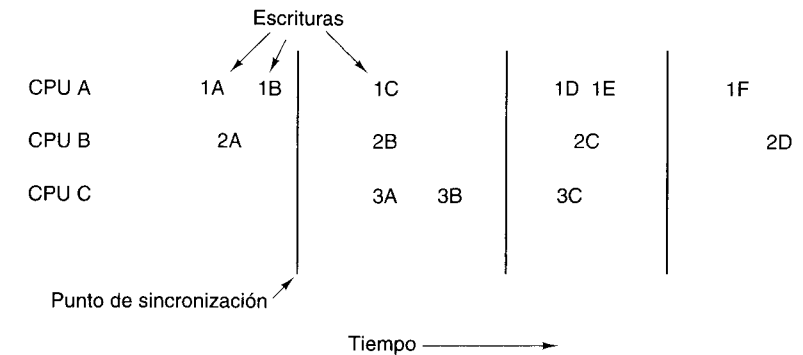
Ambos puntos son importantes. El primero dice que si la CPU 1 emite escrituras con los valores 1A, 1B y 1C a alguna localidad de memoria en ese orden, entonces todos los demás procesadores las verán en ese orden también. En otras palabras, cualquier otro procesador que esté en un ciclo corto observando 1A, 1B y 1C leyendo las palabras escritas nunca verá el valor escrito por 1B y luego el valor escrito por 1A, etc. El segundo punto es necesario para asegurar que cada localidad de memoria tendrá un valor inequívoco después de que varias CPU escriben en ella y después se detienen. Todo mundo debe estar de acuerdo en quién fue el último.

Incluso con estas restricciones, el diseñador tiene mucha flexibilidad. Considere lo que sucede si la CPU 2 emite las escrituras 2A, 2B y 2C al mismo tiempo que la CPU 1 emite sus tres escrituras. Otras CPU que estén leyendo activamente la memoria observarán cierta intercalación de las seis escrituras, digamos 1A, 1B, 2A, 2B, 1C, 2C o 2A, 1A, 2B, 2C, 1B, 1C o muchas otras. La consistencia de procesador *no* garantiza que toda CPU verá el mismo ordenamiento (a diferencia de la consistencia secuencial, que sí ofrece esta garantía). Por tanto, está permitido que el hardware se comporte de tal manera que algunas CPU vean el primero de los dos ordenamientos que pusimos de ejemplo, que algunas vean el segundo y que algunas vean otros ordenamientos. Lo que *sí* se garantiza es que ninguna CPU verá una secuencia en la que 1B ocurre antes que 1A, etc. El orden en que cada CPU realiza sus escrituras se observa en todos lados.

### Consistencia débil

Nuestro siguiente modelo, **consistencia débil**, ni siquiera garantiza que las escrituras de una sola CPU se verán en orden (Dubois *et al.*, 1986). En una memoria con consistencia débil, una CPU podría ver 1A antes que 1B y otra CPU podría ver 1A después de 1B. Sin embargo, a fin de poner cierto orden en el caos, las memorias con consistencia débil tienen variables de sincronización o una operación de sincronización. Cuando se ejecuta una sincronización, todas las escrituras pendientes se terminan y no se inicia ninguna nueva hasta que todas las viejas se llevan a cabo y la sincronización misma termina. Lo que una sincronización hace realmente es “vaciar la fila de procesamiento” y poner la memoria en un estado estable sin operaciones pendientes. Las operaciones de sincronización son en sí secuencialmente consistentes; es decir, cuando varias CPU las emiten, se escoge algún orden, pero todas las CPU ven el mismo orden.

En la consistencia débil, el tiempo se divide en épocas bien definidas delimitadas por las sincronizaciones (secuencialmente consistentes), como se ilustra en la figura 8-21. No se garantiza un orden relativo para 1A y 1B, y diferentes CPU podrían ver las dos escrituras en diferente orden; es decir, una CPU podría ver 1A y luego 1B, mientras que otra CPU podría ver 1B y después 1A. Esta situación se permite. Sin embargo, todas las CPU ven 1B antes que 1C porque la primera operación de sincronización obliga a que 1A, 1B y 2A terminen antes que se permita el inicio de 1C, 2B, 3A o 3B. Así pues, al realizar operaciones de sincronización, el software puede imponer cierto orden sobre la secuencia de sucesos, aunque no sin costo porque el vaciado de la fila de procesamiento de memoria toma cierto tiempo.



**Figura 8-21.** Una memoria con consistencia débil usa operaciones de sincronización para dividir el tiempo en épocas secuenciales.

### Consistencia de liberación

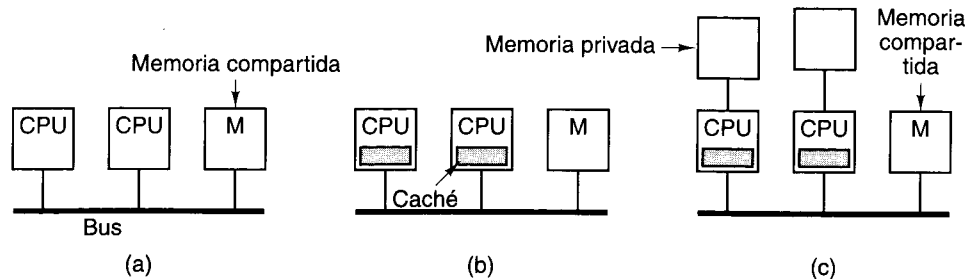
La consistencia débil tiene el problema de que es muy ineficiente porque debe terminar todas las operaciones de memoria pendiente y detener las nuevas hasta que terminen las actuales. La **consistencia de liberación** mejora la situación al adoptar un modelo parecido al de secciones críticas (Gharachorloo *et al.*, 1990). La idea en que se basa este modelo es que, cuando un proceso sale de una sección crítica no es necesario hacer que todas las escrituras se lleven a cabo inmediatamente; sólo es necesario asegurarse de que terminen antes de que cualquier proceso ingrese otra vez en esa sección crítica.

En este modelo, la operación de sincronización que ofrece la consistencia débil se divide en dos operaciones distintas. Si quiere leer o escribir una variable de datos compartida, una CPU (es decir, su software) debe efectuar primero una operación **acquire** (adquirir) sobre la variable de sincronización para obtener acceso exclusivo a los datos compartidos. Luego la CPU puede usarlos como desee, leyéndolos y escribiéndolos a voluntad. Cuando termina, la CPU efectúa una operación **release** (liberar) sobre la variable de sincronización para indicar que ya terminó. La **release** no hace que las escrituras pendientes finalicen, pero ella misma no termina sino hasta que todas las escrituras emitidas previamente se llevan a cabo. Además, no se impide que nuevas operaciones de memoria se inicien de inmediato.

Cuando se emite la siguiente **acquire**, se verifica si ya se finalizaron todas las operaciones **release** previas. Si no, la **acquire** se detiene hasta que todas terminan (y por tanto terminan también todas las escrituras realizadas antes). De este modo, si la siguiente **acquire** ocurre suficiente tiempo después de la **release** más reciente, no tiene que esperar antes de iniciar y se puede ingresar en la sección crítica sin retraso. Si ocurre demasiado pronto después de una **release**, la **acquire** (y todas las instrucciones que le siguen) se retrasarán hasta que todas las **release** pendientes terminen, lo que garantiza que las variables dentro de la sección crítica se han actualizado. Este esquema es un poco más complicado que la consistencia débil, pero tiene la importante ventaja de que no retrasa instrucciones con tanta frecuencia a fin de mantener la consistencia.

### 8.3.2 Arquitecturas SMP basadas en el bus UMA

Los procesadores más sencillos se basan en un solo bus, como se ilustra en la figura 8-22(a). Dos o más CPU y uno o más módulos de memoria usan el mismo bus para comunicarse. Cuando una CPU quiere leer una palabra de memoria, primero verifica si el bus está ocupado. Si el bus está ocioso, la CPU coloca en él la dirección de la palabra que desea, habilita unas cuantas señales de control, y espera hasta que la memoria coloca la palabra deseada en el bus.



**Figura 8-22.** Tres multiprocesadores basados en bus. (a) Sin cachés. (b) Con cachés. (c) Con cachés y memorias privadas.

Si el bus está ocupado cuando una CPU quiere leer o escribir en la memoria, la CPU esperará hasta que el bus se desocupe. Aquí radica el problema de este diseño. Con dos o tres CPU, la contención por el bus será manejable; con 32 o 64 será intolerable. El sistema estará limitado totalmente por el ancho de banda del bus, y casi todas las CPU estarán ociosas la mayor parte del tiempo.

La solución a este problema es añadir una caché a cada CPU, como se muestra en la figura 8-22(b). La caché puede estar dentro del chip de la CPU, junto al chip, en la tarjeta del procesador, o en alguna combinación de las tres posibilidades. Puesto que ahora muchas lecturas se pueden atender con la caché local, habrá mucho menos tráfico de bus, y el sistema podrá manejar más CPU.

Otra posibilidad más es el diseño de la figura 8-22(c), en el que cada CPU tiene no sólo una caché, sino también una memoria privada local a la que accede a través de un bus dedicado (privado). Para usar esta configuración de manera óptima, el compilador debe colocar todo el texto del programa, cadenas, constantes y demás datos sólo de lectura, pilas y variables locales en las memorias privadas. Entonces, la memoria compartida se usará sólo para variables compartidas escribibles. En casi todos los casos, esta colocación cuidadosa reduce considerablemente el tráfico de bus, pero requiere la cooperación activa del compilador.

#### Caché espía

Aunque los argumentos de desempeño antes mencionados son ciertamente válidos, hemos pasado por alto un problema fundamental. Supongamos que la memoria es secuencialmente consistente. ¿Qué sucede si la CPU 1 tiene una línea en su caché, y luego la CPU 2 trata de

leer una palabra de la misma línea de caché? Si no se fijan reglas especiales, ella también traerá una copia a su caché. En principio, tener la misma línea en dos cachés es aceptable. Supongamos ahora que la CPU 1 modifica la línea e inmediatamente después la CPU 2 lee su copia de la línea de su caché. La CPU 2 obtendrá **datos obsoletos**, y esto viola el contrato entre el software y la memoria. Al programa que se ejecuta en la CPU 2 no le hará gracia.

Este problema, llamado en la literatura problema de **coherencia de caché** o **consistencia de caché**, es muy grave. Si no se soluciona, no es posible usar cachés, y los multiprocesadores orientados a bus estarán limitados a dos o tres CPU. En vista de la importancia de este problema, se han propuesto muchas soluciones (por ejemplo, Goodman, 1983; Papamarcos y Patel, 1984). Aunque todos estos algoritmos de uso de cachés, llamados **protocolos de coherencia de cachés**, difieren en los detalles, todos ellos evitan que aparezcan diferentes versiones de la misma línea de caché simultáneamente en dos o más cachés.

En todas las soluciones, el controlador de caché se diseña especialmente de modo que pueda vigilar el bus, observando todas las solicitudes de bus de otras CPU y cachés y emprendiendo acciones en algunos casos. Estos dispositivos se llaman **cachés espía** (*snooping caches* o *snoopy caches*) porque "espían" el bus. El conjunto de reglas implementadas por los cachés, las CPU y la memoria para evitar que diferentes versiones de los datos aparezcan en varias cachés constituyen el protocolo de coherencia de cachés. La unidad de transferencia y almacenamiento de una caché se denomina **línea de caché** y suele ser de 32 o 64 bytes.

El protocolo de coherencia de caché más sencillo es el de **escritura a través**. La mejor manera de explicarlo es distinguir los cuatro casos que se muestran en la figura 8-23. Cuando una CPU trata de leer una palabra que no está en su caché (es decir, cuando hay un fallo de lectura), su controlador de caché carga en la caché la línea que contiene la palabra. La memoria proporciona la línea, y en este protocolo siempre está actualizada. Las lecturas subsiguientes (es decir, aciertos de caché) se pueden satisfacer con la caché.

| Acción               | Solicitud local             | Solicitud remota          |
|----------------------|-----------------------------|---------------------------|
| Fallo de lectura     | Buscar datos en memoria     |                           |
| Acierto de lectura   | Usar datos de caché local   |                           |
| Fallo de escritura   | Actualizar datos en memoria |                           |
| Acierto de escritura | Actualizar caché y memoria  | Invaldar entrada de caché |

**Figura 8-23.** Protocolo de coherencia de caché de escritura a través. Las celdas vacías indican que no se hace nada.

Cuando hay un fallo de escritura, la palabra que se modificó se escribe en la memoria principal. La línea que contiene la palabra a la que se hizo referencia *no* se carga en la caché. Cuando hay un acierto de escritura, la caché se actualiza y la palabra se escribe también a través en la memoria principal. La esencia de este protocolo es que el resultado de todas las operaciones de escritura sea que la palabra escrita se escribe siempre en la memoria a fin de mantener a ésta actualizada en todo momento.

Examinemos otra vez todas estas acciones, pero esta vez desde el punto de vista del espía, que se muestra en la columna de la derecha en la figura 8-23. Llamemos a la caché que realiza las acciones caché 1, y al espía, caché 2. Cuando la caché 1 tiene un fallo de lectura, hace una solicitud de bus para traer una línea de la memoria. La caché 2 ve esto pero no hace nada. Cuando la caché 1 tiene un acierto de lectura, la solicitud se satisface localmente, y no ocurre ninguna solicitud de bus, por lo que la caché 2 no tiene conocimiento de los aciertos de lectura de la caché 1.

Las escrituras son más interesantes. Si la CPU 1 efectúa una escritura, la caché 1 emitirá una solicitud de escritura al bus, tanto en los fallos como en los aciertos. En todas las escrituras, la caché 2 verifica si tiene la palabra que se está escribiendo. Si no la tiene, desde su punto de vista se trata de una solicitud remota/fallo de escritura y no hace nada. (A fin de aclarar un punto sutil, observe que en la figura 8-23 un fallo remoto implica que la palabra no está presente en la caché del espía; no importa si estaba en la caché del originador o no. Así, una misma solicitud puede ser un acierto localmente y un fallo en el espía, o viceversa.)

Suponga ahora que la caché 1 escribe una palabra que *sí está* presente en la caché 2 (solicitud remota/acierto de escritura). Si la caché 2 no hace nada, tendrá datos obsoletos, por lo que marca como no válida la entrada de caché que contiene la palabra recién modificada. De hecho, lo que hace es sacar los datos de la caché. Puesto que todos las cachés espían todas las solicitudes de bus, cada vez que se escribe una palabra el efecto neto es actualizarla en la caché del originador, actualizarla en la memoria, y sacarla de todos los demás cachés. De este modo, se evita que haya versiones inconsistentes.

Desde luego, la CPU de la caché 2 está en libertad de leer la misma palabra ya en el siguiente ciclo. En ese caso, la caché 2 leerá la palabra de la memoria, que está actualizada. En ese punto, la caché 1, la caché 2 y la memoria tendrán copias idénticas de la palabra. Si cualquiera de las CPU efectúa una escritura entonces, se depurará la caché de la otra, y la memoria se actualizará.

Puede haber muchas variaciones de este protocolo básico. Por ejemplo, cuando hay un acierto de escritura, la caché espía normalmente invalida su entrada que contiene la palabra que se está escribiendo. Como alternativa, esa caché podría aceptar el valor nuevo y actualizar su caché en lugar de marcarlo como no válido. En lo conceptual, actualizar la caché es lo mismo que marcarla como no válido e inmediatamente leer la palabra de la memoria. En todos los protocolos de caché se debe escoger entre una **estrategia de actualizar** y una **estrategia de invalidar**. Estos protocolos tienen diferente desempeño bajo diferentes cargas. Los mensajes de actualización llevan una carga útil y por tanto son más grandes que los de invalidación, pero podrían evitar fallos de caché futuros.

Otra variante es cargar la caché espía cuando hay fallos de escritura. La corrección del algoritmo no cambia por esta carga, sólo su desempeño. La cuestión es: "¿Qué probabilidad hay de que una palabra recién escrita se escriba otra vez pronto?" Si tal probabilidad es alta, puede ser mejor cargar la caché cuando hay fallos de escritura, lo que se denomina **política de escribir y asignar**. Si la probabilidad es baja, es mejor no actualizar cuando hay fallos de escritura. Si la palabra se lee pronto, se cargará por el fallo de lectura de todos modos; poco se ganará cargándola después del fallo de escritura.

Al igual que con muchas soluciones sencillas, ésta es ineficiente. Cada operación de escritura accede a la memoria por el bus, de modo que aunque el número de CPU no sea muy

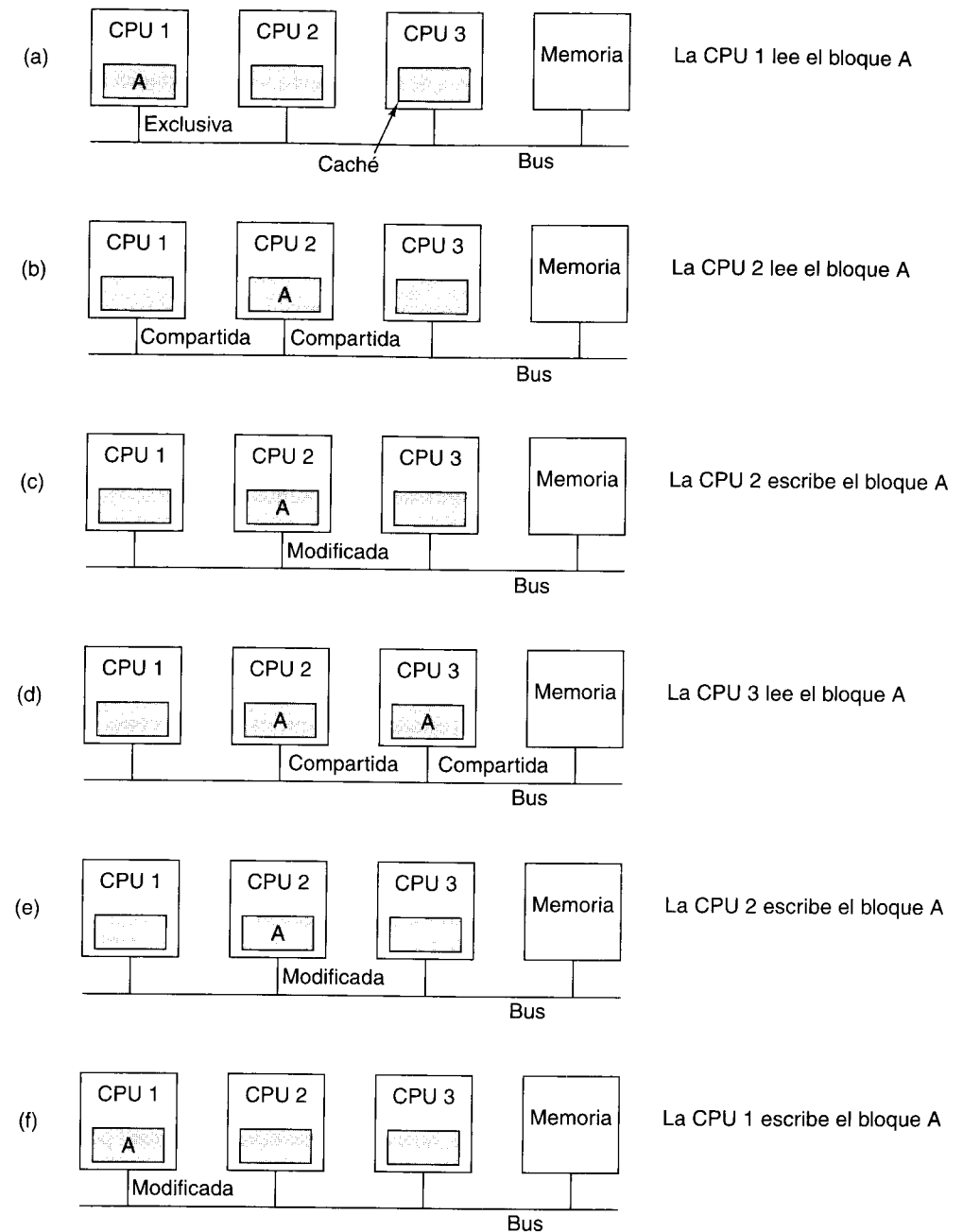


Figura 8-24. Protocolo de coherencia de caché MESI.

grande el bus de todos modos será un cuello de botella. A fin de restringir el tráfico de bus, se han inventado otros protocolos de caché. Todos ellos tienen la propiedad de que no todas las escrituras pasan directamente a la memoria. En vez de ello, cuando se modifica una línea de caché, se enciende un bit dentro de la caché que indica que la línea de caché es correcta pero la memoria no. Tarde o temprano, será preciso escribir esa línea sucia en la memoria, pero posiblemente después de que se haya escrito en ella muchas veces. Este tipo de protocolo se conoce como **protocolo de reescritura**.

### El protocolo de coherencia de caché MESI

Un protocolo de coherencia de caché de escritura de vuelta muy utilizado es **MESI**, cuyo nombre proviene de las iniciales en inglés de los cuatro estados que usa (Papamarcos y Patel, 1984). Se basa en el **protocolo de escritura única** (Goodman, 1983) anterior. El protocolo MESI se usa en el Pentium II y muchas otras CPU para espiar el bus. Cada entrada de caché puede estar en uno de los cuatro estados siguientes:

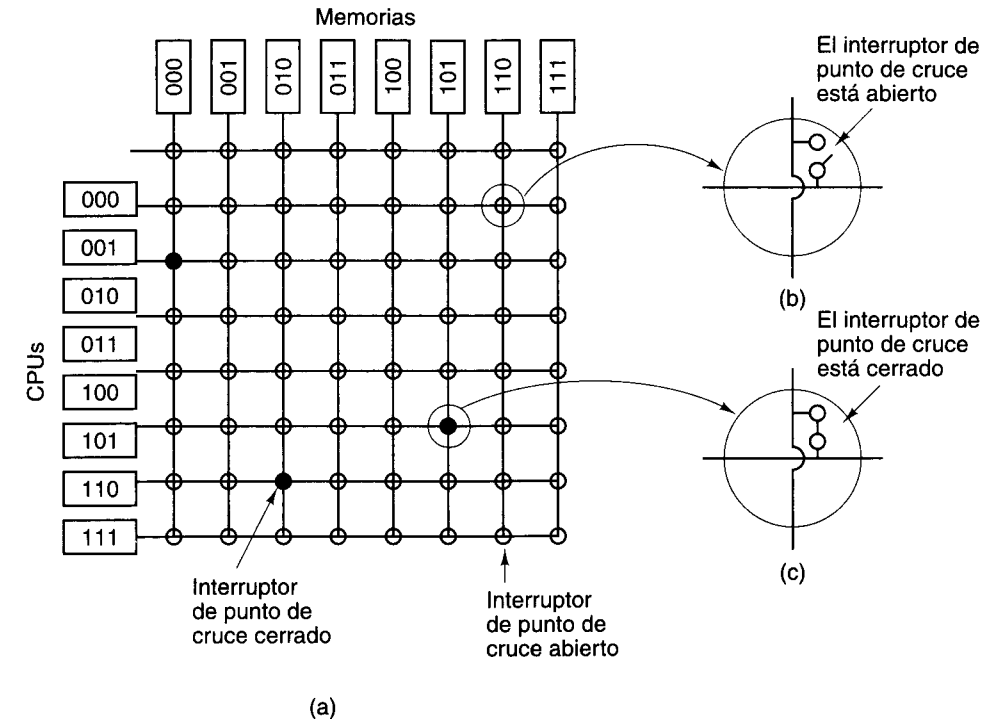
1. No válida – La entrada de caché no contiene datos válidos.
2. Compartida – Varios cachés podrían contener la línea; la memoria está actualizada.
3. Exclusiva – Ninguna otra caché contiene la línea; la memoria está actualizada.
4. Modificada – La entrada es válida; la memoria no es válida; no existen copias.

Cuando la CPU arranca, todas las entradas de caché se marcan como no válidas. La primera vez que se lee la memoria, la línea a la que se hace referencia se trae al caché de la CPU que leyó la memoria y se marca como exclusiva (estado E), ya que es la única copia que hay en una caché, como se ilustra en la figura 8-24(a) para el caso en que la CPU 1 lee la línea A. Lecturas subsecuentes efectuadas por esa CPU usan la entrada en la caché y no pasan por el bus. Otra CPU podría traer la misma línea y colocarla en su caché, pero como la caché que originalmente la tenía (CPU 1) espía, la CPU ve que ya no está sola y anuncia por el bus que también tiene una copia. Ambas copias se marcan como compartidas (estado S, *shared*), como se muestra en la figura 8-24(b). Así, el estado S implica que la línea está en uno o más cachés para lectura y la memoria está actualizada. Lecturas subsecuentes de una CPU de una línea que tiene en caché en el estado S no usan el bus y no hacen que el estado cambie.

Considere ahora qué sucede si la CPU 2 escribe en la línea de caché que tiene en el estado S. Esa CPU coloca una señal de invalidar en el bus para indicar a todas las demás CPU que desechen sus copias. La copia que queda en caché pasa al estado M (modificado), como se muestra en la figura 8-24(c). La línea no se escribe en la memoria. Vale la pena señalar que si una línea está en el estado E cuando se escribe, no es necesario enviar una señal por el bus para invalidar otros cachés porque se sabe que no existen otras copias.

Consideremos ahora lo que sucede si la CPU 3 lee la línea. La CPU 2, que ahora posee la línea, sabe que la copia que está en la memoria no es válida, por lo que aserta una señal en el bus para pedir a la CPU 3 que espere mientras escribe su línea de vuelta en la memoria. Cuando termina, la CPU 3 trae una copia, y la línea se marca como compartida en ambos cachés, como se muestra en la figura 8-24(d). Más adelante, la CPU 2 escribe la línea otra vez, lo que invalida la copia que está en la caché de la CPU 3, como se muestra en la figura 8-24(e).

Por último, la CPU 1 escribe una palabra en la línea. La CPU 2 ve que se ha intentado una escritura y aserta una señal de bus para pedir a la CPU 1 que espere mientras escribe su línea de vuelta en la memoria. Una vez que termina, la CPU 2 marca su propia copia como no válida, ya que sabe que otra CPU está a punto de modificarla. En este punto tenemos la situación en la que una CPU está escribiendo una línea que no está en ninguna caché. Si se está usando la política de escribir y asignar, la línea se cargará en la caché y se marcará como modificada, como se muestra en la figura 8-24(f). Si no se está usando la política de escribir y asignar, la escritura se efectuará directamente en la memoria y la línea no estará en ninguna caché.



**Figura 8-25.** (a) Conmutador de barra cruzada de 8 × 8. (b) Punto de cruce abierto. (c) Punto de cruce cerrado.

### 8.3.3 Multiprocesadores UMA que usan conmutadores de barras cruzadas

Aun con todas las posibles optimaciones, el uso de un solo bus limita el tamaño de un multiprocesador UMA a unas 16 o 32 CPU. Si queremos rebasar ese límite, necesitamos un tipo de red de interconexión distinto. El circuito más sencillo para conectar  $n$  CPU a  $k$  memorias es el **conmutador de barras cruzadas** que se muestra en la figura 8-25. Este tipo de conmutadores se ha usado desde hace décadas en las centrales de conmutación telefónica para conectar un grupo de líneas entrantes a un conjunto de líneas salientes de forma arbitraria.

En cada intersección de una línea horizontal (entrante) y una vertical (saliente) hay un **punto de cruce**. Un punto de cruce es un conmutador pequeño que se puede abrir o cerrar eléctricamente, dependiendo de si se quiere conectar o no las líneas horizontal y vertical. En la figura 8-25(a) vemos tres puntos de cruce cerrados simultáneamente, lo que permite conexiones entre los pares (CPU, memoria) (001, 000), (101, 101) y (110, 010) al mismo tiempo. Hay muchas otras combinaciones posibles. De hecho, el número de combinaciones es igual al número de formas distintas en que cuatro torres se pueden colocar sin peligro en un tablero de ajedrez.

Una de las propiedades más útiles del conmutador de barras cruzadas es que es una **red no bloqueadora**, lo que significa que a ninguna CPU se le niega la conexión que necesita porque algún punto de cruce o línea está ocupado (suponiendo que el módulo de memoria en sí está disponible). Además, no es necesario planear con anticipación. Aunque ya se hayan establecido siete conexiones arbitrarias, siempre será posible conectar la última CPU con la última memoria. Más adelante veremos esquemas de interconexión que no poseen estas propiedades.

Una de las peores propiedades del conmutador de barras cruzadas es el hecho de que el número de puntos de cruce aumenta en proporción a  $n^2$ . Con 1000 CPU y 1000 módulos de memoria necesitamos un millón de puntos de cruce. Un conmutador de barras cruzadas de ese tamaño no es factible. No obstante, para sistemas de tamaño mediano, un diseño de barras cruzadas resulta práctico.

### El Sun Enterprise 10000

Como ejemplo de multiprocesador UMA basado en un conmutador de barras cruzadas, consideremos el Sun Enterprise 10000 (Charlesworth, 1998; Charlesworth *et al.*, 1998). Este sistema consiste en un solo mueble que tiene hasta 64 CPU. El conmutador de barras cruzadas, llamado **Gigaplane-XB**, está empacado en una tarjeta de circuitos que tiene ocho ranuras de inserción en cada lado. Cada ranura puede aceptar una tarjeta de procesadores enorme ( $40 \times 50$  cm) que contiene cuatro CPU UltraSPARC de 333 MHz y 4 GB de RAM. Debido a los requisitos de temporización tan estrictos y la baja latencia de las barras cruzadas, acceder a memoria en otra tarjeta no es más lento que acceder a memoria en la misma tarjeta.

Tener un solo bus para todo el tráfico entre procesadores y la memoria no sería factible, por lo que en el Enterprise 10000 se usa otra estrategia. Primero, hay un conmutador de barras cruzadas  $16 \times 16$  para transferir datos entre la memoria y los cachés. Las líneas de caché son de 64 bytes y el conmutador tiene 16 bytes de anchura, por lo que se requieren cuatro ciclos para transferir una línea de caché. El conmutador opera punto a punto, por lo que no puede usarse para mantener la consistencia de las cachés.

Por ello, además del conmutador de barras cruzadas, hay cuatro buses de direcciones que sirven para espiar, como se muestra en la figura 8-26. Cada bus se usa para una cuarta parte del espacio de direcciones físico, de modo que se usan dos bits de dirección para seleccionar el bus que se espiará. Cada vez que una CPU tiene un fallo de lectura y necesita leer la memoria, la dirección se coloca en el bus de dirección apropiado para ver si tal vez está en alguna caché remota. Las 16 tarjetas espían por los cuatro buses de direcciones simultáneamente, de modo que si no hay respuesta quiere decir que la línea no está en caché y se debe traer de la memoria.

La búsqueda de información en la memoria se efectúa de punto a punto por el conmutador de barras cruzadas, de 16 en 16 bytes. El ciclo de bus es de 12 ns (83.3 MHz) y cada bus de direcciones se puede espiar en un ciclo sí y otro no, para un total de 167 millones de espionajes/s. Cada espionaje podría requerir la transferencia de una línea de caché de 64 bytes, de modo que el conmutador podría tener que transferir 9.93 GB/s (recuerde que 1 GB =  $1.0737 \times 10^9$  bytes/s, no  $10^9$  bytes/s). De hecho, se puede hacer pasar una línea de caché de 64 bytes a través del conmutador en cuatro ciclos de bus (48 ns), lo que da un ancho de banda de 1.24 GB/s por transferencia. Puesto que el conmutador puede manejar 16 transferencias simultáneas, su ancho de banda instantáneo máximo es de 19.87 GB/s, que es suficiente para mantenerse a la par de la tasa de espionaje, aun tomando en cuenta la contención por el conmutador, que reduce el ancho de banda práctico a cerca del 60% del valor teórico.

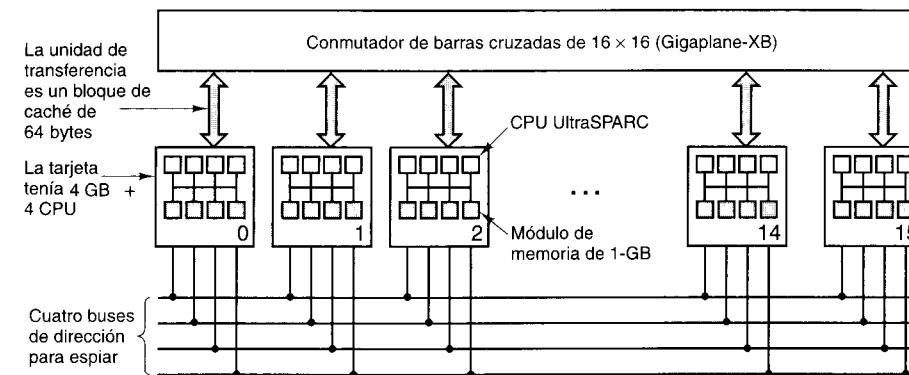


Figura 8-26. Multiprocesador simétrico Sun Enterprise 10000.

Es evidente que el Enterprise 10000 está en el límite; usa cuatro buses para espiar en paralelo y un conmutador de barras cruzadas muy ancho para transferir datos. Si queremos ir mucho más allá de 64 CPU, vamos a necesitar algo muy distinto.

### 8.3.4 Multiprocesadores UMA que usan redes de conmutación multietapas

Ese “algo muy distinto” puede basarse en el humilde conmutador de  $2 \times 2$  que se muestra en la figura 8-27(a). Este conmutador tiene dos entradas y dos salidas. Los mensajes que llegan por cualquiera de las líneas de entrada se pueden conmutar a cualquiera de las líneas de salida. Para nuestros fines, los mensajes contendrán hasta cuatro partes, como se muestra en la figura 8-27(b). El campo *Módulo* indica cuál memoria debe usarse. *Dirección* especifica una dirección dentro de un módulo. *Cód-op* da la operación, digamos **READ** o **WRITE**. Por último, el campo *Valor* opcional podría contener un operando, digamos una palabra de 32 bits que se escribirá con la instrucción **WRITE**. El conmutador inspecciona el campo *Módulo* y lo usa para determinar si el mensaje se debe enviar a *X* o a *Y*.

Nuestros conmutadores  $2 \times 2$  se pueden acomodar de muchas maneras para construir redes de conmutación multietapas más grandes (Adams *et al.*, 1987; Bhuyan *et al.*, 1989;



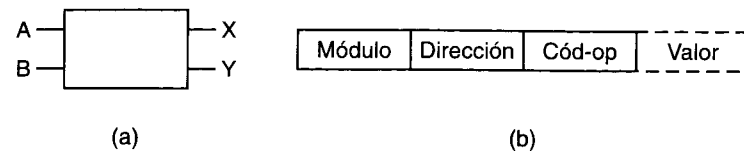


Figura 8-27. (a) Conmutador 2 x 2. (b) Un formato de mensaje.

Kumar y Reddy, 1987). Una posibilidad es la austera **red omega** que se ilustra en la figura 8-28. Aquí hemos conectado ocho CPU con ocho memorias empleando 12 conmutadores. De forma más general, para  $n$  CPU y  $n$  memorias necesitaríamos  $\log_2 n$  etapas, con  $n/2$  conmutadores por etapa, para un total de  $(n/2)\log_2 n$  conmutadores, que es mucho mejor que  $n^2$  puntos de cruce, sobre todo para valores grandes de  $n$ .

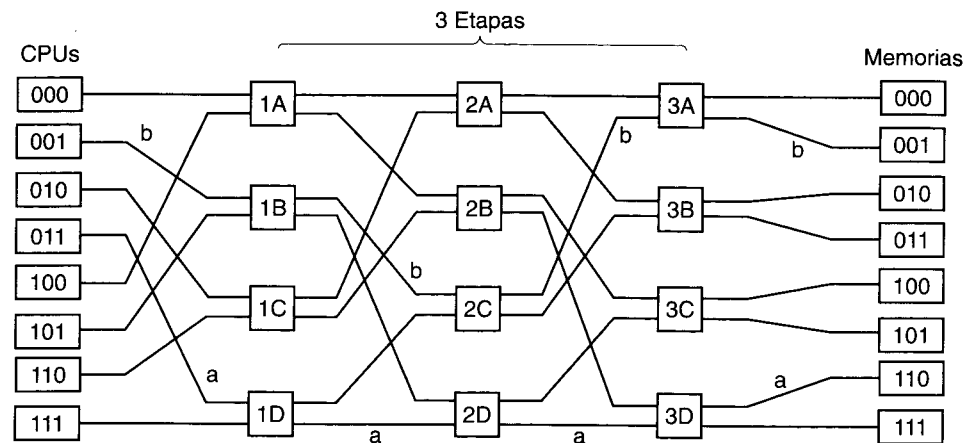


Figura 8-28. Red de conmutación omega.

El patrón de cableado para la red omega se conoce como **barajado perfecto**, ya que el mezclado de las señales en cada etapa se parece a un mazo de naipes que se divide a la mitad y luego se mezcla naip por naip. Para ver cómo funciona la red omega, suponga que la CPU 011 quiere leer una palabra del módulo de memoria 110. La CPU envía un mensaje READ al conmutador 1D que contiene 110 en el campo *Módulo*. El conmutador toma el primer bit (es decir, el de la extrema izquierda) de 110 y lo usa para enrutamiento. Un 0 dirige a la salida superior y un 1 dirige a la salida inferior. Puesto que este bit es 1, el mensaje se encamina por la salida inferior hacia 2D.

Todos los conmutadores de segunda etapa, incluido 2D, usan el segundo bit para enrutamiento. Éste también es 1, de modo que el mensaje ahora se reenvía por la salida inferior hacia 3D. Aquí se prueba el tercer bit y se ve que es 0. Por tanto, el mensaje sale por la salida superior y llega a la memoria 110, como se deseaba. El camino que este mensaje sigue se ha marcado con la letra *a* en la figura 8-28.

A medida que el mensaje avanza por la red de conmutación, los bits del extremo izquierdo del número de módulo dejan de necesitarse. Se pueden aprovechar para registrar el número de línea entrante, a fin de que la respuesta pueda encontrar el camino de regreso. En el caso del camino *a*, las líneas entrantes son 0 (entrada superior de 1D), 1 (entrada inferior de 2D) y 1 (entrada inferior de 3D), respectivamente. La respuesta se encamina de regreso usando 011, sólo que esta vez se lee de derecha a izquierda.

Al mismo tiempo que está pasando todo esto, la CPU 001 quiere escribir una palabra en el módulo de memoria 001. Aquí ocurre un proceso análogo, y el mensaje se encamina por las salidas superior, superior e inferior, respectivamente, camino que se ha marcado con la letra *b*. Cuando el mensaje llega, su campo *Módulo* es 001, lo que representa el camino que tomó. Puesto que estas dos solicitudes no usan los mismos conmutadores, líneas ni módulos de memoria, pueden ocurrir en paralelo.

Considere ahora qué sucedería si simultáneamente la CPU 000 quisiera acceder al módulo de memoria 000. Su solicitud entraría en conflicto con la solicitud de la CPU 001 en el conmutador 3A. Una de ellas tendría que esperar. A diferencia del conmutador de barras cruzadas, la red omega es una **red bloqueadora**. No todos los conjuntos de solicitudes se pueden procesar simultáneamente. Puede haber conflictos por el uso de un alambre o por el uso de un conmutador, y también entre solicitudes *a* la memoria y respuestas *de* la memoria.

Obviamente es deseable repartir las referencias a la memoria de manera uniforme entre los módulos. Una técnica común consiste en usar los bits de orden bajo como número de módulo. Considere, por ejemplo, un espacio de direcciones orientado a bytes para una computadora que por lo regular accede a palabras de 32 bits. Los dos bits de orden bajo normalmente son 00, pero los siguientes 3 bits tienen una distribución uniforme. Si se usan estos 3 bits como número de módulo, las palabras con direcciones consecutivas estarán en módulos consecutivos. Un sistema de memoria en el que palabras consecutivas están en diferentes módulos se describe como **intercalado**. Las memorias intercaladas maximizan el paralelismo porque casi todas las referencias a la memoria son a palabras consecutivas. También es posible diseñar redes de conmutación que sean no bloqueadoras y que ofrezcan varios caminos de cada CPU a cada módulo de memoria, a fin de distribuir el tráfico mejor.

### 8.3.5 Multiprocesadores NUMA

A estas alturas debe ser evidente que los multiprocesadores UMA de un solo bus generalmente están limitados a unas cuantas docenas de CPU, y los multiprocesadores de barras cruzadas o conmutados necesitan mucho hardware (costoso) y no son mucho más grandes. Si se quiere llegar a más de 100 CPU, algo tiene que ceder. Por lo regular, lo que cede es la idea de que todos los módulos de memoria tienen el mismo tiempo de acceso. Esta concesión da lugar a la idea de **multiprocesadores con acceso no uniforme a la memoria** (NUMA, *NonUniform Memory Access*). Al igual que sus parientes UMA, éstos ofrecen un solo espacio de direcciones para todas las CPU, pero a diferencia de las máquinas UMA, el acceso a los módulos de memoria local es más rápido que a los módulos remotos. Así pues, todos los programas UMA se pueden ejecutar sin cambios en las máquinas NUMA, pero el desempeño es más bajo que en una máquina UMA si la velocidad de reloj es la misma.

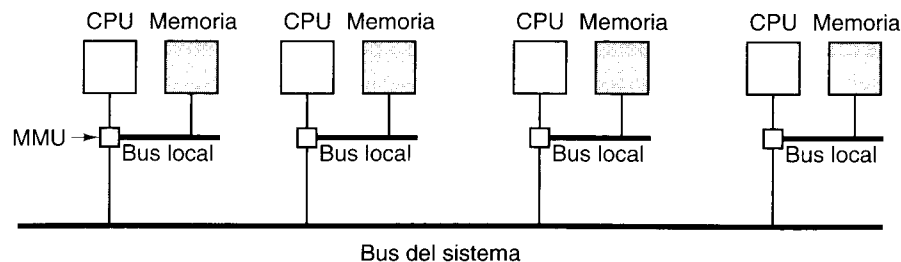


Las máquinas NUMA tienen tres características clave comunes a todas ellas que juntas las distinguen de otros multiprocesadores:

1. Hay un solo espacio de direcciones visible para todas las CPU.
2. El acceso a la memoria remota se efectúa con instrucciones **LOAD** y **STORE**.
3. El acceso a la memoria remota es más lento que el acceso a la memoria local.

Cuando el tiempo de acceso a la memoria remota no se oculta (porque no hay uso de cachés), el sistema se llama **NC-NUMA**. Si hay cachés coherentes, el sistema se llama **CC-NUMA** (al menos la gente de hardware lo llama así). La gente de software a menudo lo llama **DSM en hardware** porque es básicamente igual a la memoria compartida distribuida (DSM) pero implementado por el hardware utilizando páginas pequeñas.

Una de las primeras máquinas NC-NUMA (aunque todavía no se acuñaba ese nombre) fue la Cm\* de Carnegie-Mellon, que se ilustra en forma simplificada en la figura 8-29 (Swan *et al.*, 1977). La Cm\* consistía en una colección de CPU LSI-11, cada una con un poco de memoria que se direccionaba por un bus local. (El LSI-11 fue una versión en un solo chip del DEC PDP-11, una minicomputadora popular en los años setenta.) Además, los sistemas LSI-11 estaban conectados por un bus del sistema. Cuando una solicitud de memoria llegaba a la MMU (que tenía modificaciones especiales), se verificaba si la palabra requerida estaba en la memoria local. Si así era, se enviaba una solicitud por el bus local para obtener la palabra; si no, la solicitud se enviaba por el bus del sistema al sistema que contenía la palabra, el cual entonces respondía. Desde luego, lo segundo tardaba mucho más que lo primero. Si bien un programa podía ejecutarse sin problema usando memoria remota, tardaba 10 veces más en ejecutarse que el mismo programa usando memoria local.



**Figura 8-29.** Máquina NUMA basada en dos niveles de buses. El Cm\* fue el primer multiprocesador que usó este diseño.

La coherencia de memoria está garantizada en una máquina NC-NUMA porque no se usan cachés. Cada palabra de memoria reside en una y sólo una localidad, por lo que no hay peligro de que una copia tenga datos obsoletos: no hay copias. Desde luego, ahora es muy importante cuál página está en cuál memoria porque el menoscabo en el desempeño debido a que esté en el lugar equivocado es muy alto. Por ello, las máquinas NC-NUMA utilizan software complejo para colocar las páginas de modo que se maximice el desempeño.

Por lo regular existe un proceso demonio llamado **explorador de páginas** que se ejecuta cada cierto número de segundos. Su trabajo consiste en examinar las estadísticas de utilización y cambiar de lugar las páginas en un intento por mejorar el desempeño. Si una página parece estar en el lugar equivocado, el explorador de páginas anula su correspondencia con la memoria de modo que la siguiente referencia a ella cause un fallo de página. Cuando ocurra el fallo, se tomará una decisión acerca de dónde debe colocarse la página, posiblemente en una memoria distinta de aquella en la que estaba antes. Para evitar la hiperpaginación, generalmente hay una regla que dice que, una vez que una página se coloca, no podrá cambiarse de lugar durante un tiempo  $\Delta T$ . Se han estudiado diversos algoritmos, pero la conclusión es de que ningún algoritmo tiene un desempeño óptimo en todas las circunstancias (LaRowe y Ellis, 1991).

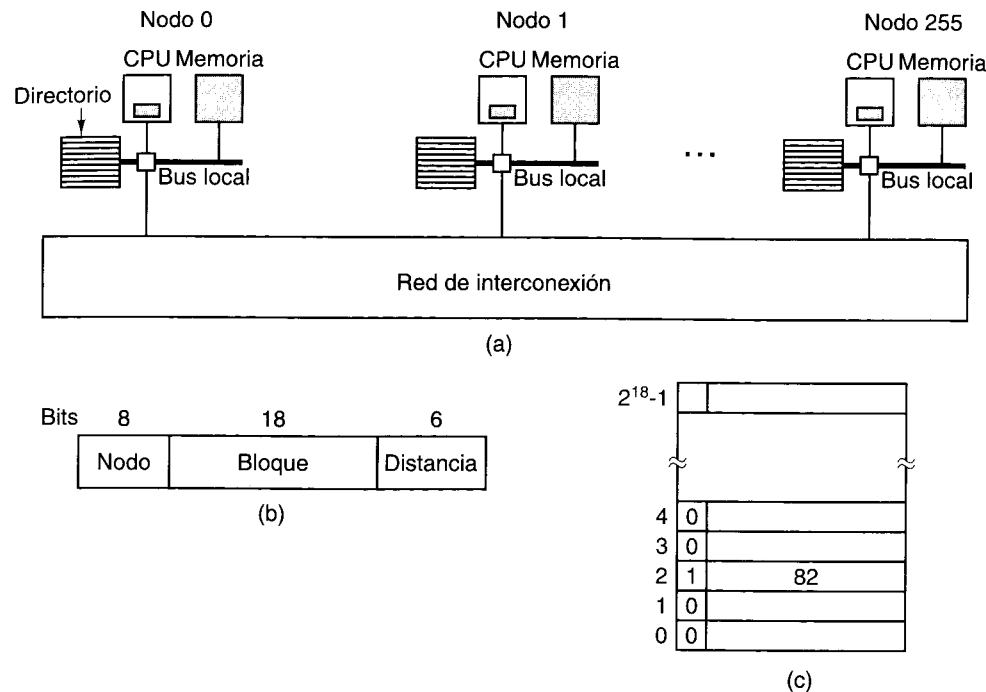
### 8.3.6 Multiprocesadores NUMA con coherencia de caché

Los diseños de multiprocesador como el de la figura 8-29 no se prestan a cambios de escala porque no usan cachés. Tener que acudir a la memoria remota cada vez que se accesa a una palabra de memoria no local es un freno importante para el desempeño. Sin embargo, si se añaden cachés, también hay que añadir coherencia de cachés. Una forma de proporcionar coherencia de cachés es espiar el bus del sistema. Técnicamente, no es difícil hacerlo, pero como vimos con el Enterprise 10000, incluso con cuatro buses para espiar y un conmutador de barras cruzadas de alta velocidad con 16 bytes de anchura para el tráfico de datos, 64 CPU es casi el límite de tamaño. Si queremos construir multiprocesadores en verdad grandes, se requiere un enfoque fundamentalmente distinto.

El enfoque más popular para construir multiprocesadores **NUMA con coherencia de caché** (CC-NUMA, *Cache Coherent NUMA*) es el **multiprocesador basado en directorios**. La idea es mantener una base de datos que dice dónde está cada línea de caché y cuál es su situación. Cuando se hace referencia a una línea de caché, se consulta la base de datos para averiguar dónde está y si está limpia o sucia (modificada). Puesto que esta base de datos se debe consultar en cada instrucción que hace referencia a la memoria, se tiene que mantener en hardware especial extremadamente rápido capaz de responder en una fracción de ciclo de bus.

A fin de hacer un poco más concreta la idea de un multiprocesador basado en directorios, consideremos un ejemplo (hipotético) sencillo, un sistema con 256 nodos, cada uno de los cuales consiste en una CPU y 16 MB de RAM conectada a la CPU por un bus local. La memoria total es de  $2^{32}$  bytes, dividida en  $2^{26}$  líneas de caché de 64 bytes cada una. La memoria se reparte estáticamente entre los nodos, con 0-16M en el nodo 0, 16M-32M en el nodo 1, y así. Los nodos se conectan con una red de interconexión, como se muestra en la figura 8-30(a). La red podría ser una cuadrícula, un hipercubo u otra topología. Cada nodo contiene también las entradas de directorio para las  $2^{18}$  líneas de caché de 64 bytes que constituyen su memoria de  $2^{24}$  bytes. Por el momento, supondremos que una línea puede estar contenida en cuando más una caché.

Para ver cómo funciona el directorio, sigamos la pista a una instrucción **LOAD** emitida por la CPU 20 que hace referencia a una línea que está en caché. Primero la CPU que emite la instrucción la presenta a su MMU, la cual la traduce en una dirección física, digamos 0x24000108. La MMU divide esta dirección en las tres partes que se muestran en la figura



**Figura 8-30.** (a) Multiprocesador de 256 nodos basado en directorios. (b) División de una dirección de memoria de 32 bits en campos. (c) El directorio del nodo 36.

8-30(b). En decimal, las tres partes son el nodo 36, línea 4 y distancia 8. La MMU ve que la palabra de memoria a la que se hizo referencia es del nodo 36, no del nodo 20, por lo que envía un mensaje de solicitud por la red de interconexión al nodo base de la línea, el 36, preguntándole si su línea 4 está en caché y, si así es, dónde está.

Cuando la solicitud llega al nodo 36 por la red de interconexión, se le encamina al hardware de directorio. El hardware busca la entrada 4 de su tabla de  $2^{18}$  entradas, una para cada una de sus líneas de caché, y la extrae. En la figura 8-30(c) vemos que la línea no está en la caché, por lo que el hardware trae la línea 4 de la RAM local, la devuelve al nodo 20 y actualiza la entrada 4 de su directorio de modo que indique que la línea ahora está en caché en el nodo 20.

Consideremos ahora una segunda solicitud, que ahora pide la línea 2 del nodo 36. En la figura 8-30(c) vemos que esta línea está en caché en el nodo 82. En este punto el hardware podría actualizar la entrada 2 del directorio de modo que diga que la línea ahora está en el nodo 20 y luego enviar al nodo 82 un mensaje pidiéndole que pase la línea al nodo 20 e invalide su caché. Observe que incluso en un supuesto “multiprocesador de memoria compartida” hay una gran cantidad de transferencia de mensajes oculta.

Como digresión rápida, calculemos qué tanta memoria están ocupando los directorios. Cada nodo tiene 16 MB de RAM y  $2^{18}$  entradas de 9 bits para mantenerse al tanto de esa RAM. Así, el gasto extra del directorio es de  $9 \times 2^{18}$  bits dividido entre 16 MB, o sea cerca del

1.76%, que generalmente es aceptable (aunque tiene que ser memoria de alta velocidad, lo que aumenta su costo). Aun con líneas de caché de 32 bytes el gasto extra sería de sólo un 4%. Con líneas de caché de 128 bytes, el gasto extra sería de menos del 1%.

Una limitación obvia de este diseño es que una línea se puede colocar en caché sólo en un nodo. Para poder colocar líneas en caché en varios nodos necesitaríamos alguna forma de localizarlas a todas, por ejemplo para invalidarlas o actualizarlas cuando se efectúa una escritura. Hay varias opciones que permiten colocar en caché en varios nodos al mismo tiempo.

Una posibilidad es dar a cada entrada de directorio  $k$  campos para especificar otros nodos, lo que permite a cada línea colocarse en caché en hasta  $k$  nodos. Una segunda posibilidad es sustituir el número de nodo de nuestro sencillo diseño por un mapa de bits, con un bit por nodo. Con esta opción no hay límite para el número de posibles copias, pero el gasto extra aumenta sustancialmente. Tener un directorio con 256 bits por cada línea de caché de 64 bytes (512 bits) implica un gasto extra de más del 50%. Una tercera posibilidad es mantener un campo de 8 bits en cada entrada de directorio y usarlo como cabeza de una lista enlazada que “ensarta” todas las copias de la línea de caché. Esta estrategia requiere almacenamiento extra en cada nodo para los apuntadores de la lista enlazada, y también requiere seguir una lista enlazada para encontrar todas las copias cuando esto es necesario. Cada posibilidad tiene sus ventajas y desventajas, y las tres se han usado en sistemas reales.

Otra mejora del diseño de directorios consiste en asentar si la línea de caché está limpia (la memoria base está actualizada) o sucia (la memoria base no está actualizada). Si llega una solicitud de lectura para una línea de caché limpia, el nodo base puede satisfacer la solicitud desde la memoria, sin tener que reenviarla a una caché. En cambio, una solicitud de lectura de una línea de caché sucia debe reenviarse al nodo que contiene la línea de caché porque sólo ella tiene una copia válida. Si sólo se permite una copia en caché, como en la figura 8-30, no resulta realmente ventajoso mantenerse al tanto de si está limpia o sucia, porque cualquier solicitud nueva requerirá el envío de un mensaje a la copia existente para invalidarla.

Desde luego, mantenerse al tanto de si cada línea de caché está limpia o sucia implica que cuando una línea de caché se modifica hay que informar de ello al nodo base, aunque sólo exista una copia en caché. Si hay varias copias, la modificación de una de ellas requiere invalidar todas las demás, por lo que se necesita algún protocolo para evitar condiciones de competencia. Por ejemplo, para modificar una línea de caché compartida, uno de los que la tienen podría tener que solicitar acceso exclusivo *antes* de modificarla. Semejante solicitud haría que todas las demás copias se invaliden antes de poder dar el permiso. En (Stenstrom *et al.*, 1997) se describen optimaciones del desempeño adicionales para máquinas CC-NUMA.

### El multiprocesador Stanford DASH

El primer multiprocesador CC-NUMA basado en directorios, **arquitectura de directorios para memoria compartida** (DASH, *Directory Architecture for SHared memory*), se construyó en la Stanford University (Lenoski *et al.*, 1992) como proyecto de investigación. Su diseño es fácil de explicar y ha influido mucho en varios productos comerciales, como el SGI Origin 2000, de